VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CAPSTONE PROJECT

STUDYING AND DEVELOPING NONBLOCKING DISTRIBUTED MPSC QUEUES

Major: Computer Science

THESIS COMMITTEE: 7 - COMPUTER SCIENCE
SUPERVISORS: DR. DIỆP THANH ĐĂNG
ASSOC. PROF. DR. THOẠI NAM
REVIEWER: DR. LÊ THANH VÂN

—ooo—

STUDENT: ĐỖ NGUYỄN AN HUY (2110193)

HO CHI MINH CITY, 7/10/2025

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

KHOA: KH & KT Máy tính
BỘ MÔN: HT & MMT

NHIỆM VỤ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP

Chú ý: Sinh viên phải dán tờ này vào trang nhất của bản thuyết trình

HỌ VÀ TÊN: ĐỖ NGUYỄN AN HUY MSSV: 2110193
NGÀNH: KHOA HỌC MÁY TÍNH LỚP:

1. Đầu đề luận văn/ đồ án tốt nghiệp:

- Tiếng Việt: Nghiên cứu và phát triển cấu trúc dữ liệu nonblocking MPSC queue phân tán
- Tiếng Anh: Studying and developing nonblocking distributed MPSC queues

2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

- Hiện thực các cấu trúc dữ liệu nonblocking MPSC queue được chọn trên hệ thống tính toán hiệu năng cao đang có.
- Thiết lập môi trường đánh giá các cấu trúc dữ liệu nonblocking MPSC queue.
- Xác định các tiêu chí đánh giá.
- Đề xuất các kịch bản đánh giá.
- Tiến hành đánh giá các kỹ thuật mã hóa dữ liệu lượng tử.
- Đề xuất cấu trúc dữ liệu nonblocking MPSC queue phân tán mới cho tính toán hiệu năng cao (bonus).

3. Ngày giao nhiệm vụ: 16/06/2025

4. Ngày hoàn thành nhiệm vụ: 10/10/2025

5. Họ tên giảng viên hướng dẫn:	**Phần hướng dẫn:**
1) TS. Diệp Thanh Đăng	Thảo luận, sửa lỗi trình bày
2) PGS. TS. Thoại Nam	Thảo luận

Nội dung và yêu cầu LVTN/ ĐATN đã được thông qua Bộ môn.

Ngày ..... tháng ..... năm .....

CHỦ NHIỆM BỘ MÔN
(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH
(Ký và ghi rõ họ tên)

Diệp Thanh Đăng

PHẦN DÀNH CHO KHOA, BỘ MÔN:
Người duyệt (chấm sơ bộ):
Đơn vị:
Ngày bảo vệ:
Điểm tổng kết:
Nơi lưu trữ LVTN/ĐATN:

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KH & KT MÁY TÍNH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

Ngày 03 tháng 10 năm 2025

PHIẾU ĐÁNH GIÁ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP

(Dành cho người hướng dẫn/phản biện)

1. Họ và tên SV: Đỗ Nguyễn An Huy
MSSV: 2110193 Ngành (chuyên ngành): Khoa Học Máy Tính
2. Đề tài: Nghiên cứu và phát triển cấu trúc dữ liệu nonblocking MPSC queue phân tán

3. Họ tên người hướng dẫn: Diệp Thanh Đăng

4. Tổng quát về bản thuyết minh:
Số trang: 95 Số chương: 6
Số bảng số liệu: 7 Số hình vẽ: 24
Số tài liệu tham khảo: 25 Phần mềm tính toán: 0
Hiện vật (sản phẩm): 0

5. Những ưu điểm chính của LV/ ĐATN:
- Phân tích và thảo luận chi tiết về các cấu trúc dữ liệu MPSC queue bộ nhớ chia sẻ và phân tán đang có trên thế giới dựa trên nhiều tiêu chí: ABA solution, safe memory reclamation, progress guarantees of enqueue & dequeue operations.
- Phát hiện được nhiều lỗi sai của các queue MPSC queue bộ nhớ chia sẻ và phân tán đang có và thậm chí đã được tác giả của các nghiên cứu đó xác nhận.
- Đề xuất 3 cấu trúc dữ liệu nonblocking MPSC queue phân tán mới, bao gồm: dLTQueue, Slot Queue, và dLTQueueV2.
- Chứng minh tính đúng đắn lý thuyết của các cấu trúc dữ liệu MPSC queue đề xuất.
- So sánh cả lý thuyết và thực nghiệm (về throughput và latency trên các hệ thống tính toán hiệu năng cao của Đức) giữa các queue đề xuất với một blocking MPSC queue bộ nhớ phân tán hiện đại để cho thấy được những ưu điểm của các queue mà ĐATN này đề xuất.
- 2 bài báo khoa học về 2 cấu trúc dữ liệu nonblocking MPSC queue phân tán đề xuất đã được chấp nhận xuất bản tại 2 hội nghị quốc tế (NPC'25 và FDSE'25).
- Khối lượng công việc được thực hiện trong ĐATN này vượt rất xa kì vọng của một đề tài ĐATN thông thường mà giảng viên hướng dẫn từng chứng kiến.

6. Những thiếu sót chính của LV/ĐATN:
- Nên xem xét thêm về việc đánh giá các cấu trúc dữ liệu nonblocking MPSC queue phân tán trên tiêu chí tiêu hao bộ nhớ. (Tuy nhiên, đây không hẳn là thiếu sót chính của ĐATN này.)

7. Đề nghị: Được bảo vệ ⊗ Bổ sung thêm để bảo vệ ○ Không được bảo vệ ○
8. Các câu hỏi SV phải trả lời trước Hội đồng:
a. Thách thức chính của ĐATN này là gì?
9. Đánh giá chung (bằng chữ: Xuất sắc, Giỏi, Khá, TB): Xuất sắc Điểm: 10/10

Ký tên (ghi rõ họ tên)

Diệp Thanh Đăng

viê TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KH & KT MÁY TÍNH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

Ngày 6 tháng 10 năm 2025

PHIẾU ĐÁNH GIÁ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP

(Dành cho người phản biện)

1. Họ và tên SV: Đỗ Nguyễn An Huy
MSSV: 2110193 Ngành (chuyên ngành): Khoa học máy tính
2. Đề tài: Nghiên cứu và phát triển cấu trúc dữ liệu nonblocking MPSC queue phân tán

3. Họ tên người phản biện: TS. Lê Thanh Vân

4. Tổng quát về bản thuyết minh:
Số trang: 95 Số chương: 7 (gồm 1 chương phụ lục)
Số bảng số liệu: 7 Số hình vẽ: 24
Số tài liệu tham khảo: 25 Phần mềm tính toán:
Hiện vật (sản phẩm)
5. Những ưu điểm chính của LV/ ĐATN:

Luận văn có đề tài mới mẻ, có giá trị học thuật rõ ràng và tập trung vào việc xây dựng hàng đợi MPSC phi chặn trong môi trường phân tán. Các ưu điểm chính của luận văn gồm có:

- Sinh viên chỉ ra hạn chế của AMQueue và đề xuất ba thuật toán mới (dLTQueue, Slotqueue, dLTQueueV2) với hiệu năng và độ tin cậy cao. Các thuật toán này vừa kế thừa ý tưởng từ các hàng đợi trong bộ nhớ chia sẻ, vừa điều chỉnh để thích nghi với môi trường phân tán dựa trên MPI-3. Các thuật toán này đều được thiết kế theo hướng wait-free hoặc lock-free, đảm bảo tiến độ (progress guarantee) và tính chịu lỗi (fault tolerance).
- Phần cơ sở lý thuyết bao quát đầy đủ các khái niệm nền tảng như linearizability, non-blocking algorithms, lock-free và wait-free, cũng như các vấn đề thường gặp như ABA và bộ nhớ reclamation. Sinh viên không chỉ dừng ở mức mô tả mà còn đi sâu vào chứng minh tính đúng đắn, đảm bảo cho các giải pháp được đề xuất có cơ sở vững chắc.
- Xây dựng đầy đủ các biến thể hàng đợi trong môi trường MPI-3 với BCL CoreX, sau đó tiến hành thử nghiệm với các chỉ số cụ thể như throughput, latency, và hiệu năng enqueue/dequeue. Kết quả benchmark được minh họa bằng các bảng số liệu và hình vẽ rõ ràng, cho thấy ưu thế của Slotqueue và dLTQueue so với các giải pháp trước.
- Lý giải được những vấn đề khi chuyển từ môi trường shared memory sang môi trường phân tán.
- Có 2 bài báo được chấp nhận trình bày tại hội nghị quốc tế FDSC 2025 và NPC 2025.

6. Những thiếu sót chính của LV/ĐATN:

- Cần trình bày luận văn theo đúng định dạng qui định (được công bố trên elearning của khoa Máy tính)

- Làm rõ sự khác biệt về interconnect của CoolMUC-4 vs SuperMUC-NG và sự khác biệt hiệu năng khi đánh giá.

7. Đề nghị: Được bảo vệ □ Bổ sung thêm để bảo vệ □ Không được bảo vệ □

8. Các câu hỏi SV phải trả lời trước Hội đồng:

a. Tại sao kết quả thực nghiệm cho thấy enqueue throughput giảm khi tăng số lượng node, mặc dù thuật toán lý thuyết bảo đảm constant time?

9. Đánh giá chung (bằng chữ: Xuất sắc, Giỏi, Khá, TB): Điểm : 10 /10

Ký tên (ghi rõ họ tên)



Lê Thanh Vân

Disclaimers

I affirm that this capstone project is the product of my original research and experimentation. Any references, resources, results which this project is based on have been given due citations and properly listed in the references section. All original contents presented are the culmination of my dedication and perserverance under the close guidance of my supervisors, Mr. Thoại Nam and Mr. Diệp Thanh Đăng, from the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology. I take full responsibility for the accuracy and authenticity of this document. Any misinformation, copyright infringement or plagiarism shall be faced with serious punishment.

Acknowledgements

This thesis is the culmination of joint efforts coming from not only myself, but also my professors, my family, my friends and other teachers of Ho Chi Minh University of Technology.

I want to first acknowledge my university, Ho Chi Minh University of Technology. Throughout my four years of pursuing education here, I have built a strong theoretical foundation and gained various practical experiences. These all lend themselves well to the completion of this thesis. However, I have to say that the one person who plays the cornerstone in my academic foundation must be Mr. Diệp Thanh Đăng. He has been the constant supervisor and reliable consultant through this capstone project, right from its inception, throughout its nurturing until the very end. The project could not have reached this stage of maturity without Mr. Đăng. Therefore, he has the right to share whatever yields this project has to offer. I know it is getting informal, but if you ever read this, I want you to know that you surely have become one of the most important people in my life. You are the right blend of intellect, wisdom, organization, empathy, open-mindedness that I always seek in an academic companion, which surely secures you a special place in my heart. Whatever the outcome of what is about to happen, whoever I become in the future, I will not ever forget you and our magical moments when we were brewing up this project. I want to credit you for anything I achieve in front of just anyone, including myself. I have always craved a companionship like Friedrich Engels and Karl Marx; maybe I have come quite close to it.

I want to thank Mr. Thoại Nam for providing all the facilities and infrastructure that this project necessitates. He essentially funded all of our idea-brewing grounds, where magic happens.

I also want to give my family the sincerest thanks for their emotional and financial support, without which I could not have wholeheartedly followed my research till the end.

Last but not least important, I want to thank my closest friends for their informal but ever-constant check-ups to make sure I did not miss the timeline for this capstone project, which I usually do not have the mental capacity for.

Abstract

A large class of distributed applications (e.g. the irregular applications) requires the use of distributed data structures. A subset of these applications (e.g. the distributed actor model, the fan-out/fan-in pattern) specifically requires distributed multi-producer single-consumer (MPSC) queues. For a distributed MPSC queue to be useful, it must provide high performance and fault tolerance (the notion of fault tolerance is closely linked to the notion of non-blocking). To date, there has been little effort into designing a performant and fault tolerant MPSC queue. In this thesis, we address this question, aiming to provide the programmers with a general-purpose, highly performant and fault tolerant MPSC queue. The key observation that guides us is that there are a lot non-blocking shared-memory MPSC queues in the literature. Therefore, our methodology is to use a bridging model, i.e. BCL CoreX, to adapt these into the distributed domain. We additionally make some optimizations to the resulting queues that take into account the constraint of distributed environments. As a result, we have obtained two fault tolerant (wait-free) MPSC queues that have good theoretical performance characteristics: dLTQueueV2 and Slotqueue. Based on our empirical evaluation against another blocking distributed MPSC queue (AMQueue), our queues perform better in terms of enqueue throughput and a little worse in terms of dequeue throughput. We believe that Slotqueue and dLTQueueV2 offer the right balance of fault tolerance and performance and can aid the programmer in implementing certain programming patterns.

Contents

Chapter I Introduction 14
1.1 Motivation 14
1.2 Objective 15
1.3 Scope 16
1.4 Research question 16
1.5 Contributions 17
1.6 Related publications 17
1.7 Structure 18
Chapter II Background 19
2.1 Irregular applications 19
2.1.1 Actor model as an irregular application 20
2.1.2 Fan-out/Fan-in pattern as an irregular application 21
2.2 MPSC queue 22
2.3 Correctness condition of concurrent algorithms 23
2.4 Fault-tolerance & progress guarantee of concurrent algorithms 24
2.4.1 Blocking algorithms 25
2.4.2 Non-blocking algorithms 26
2.4.2.1 Lock-free algorithms 26
2.4.2.2 Wait-free algorithms 26
2.5 Popular atomic instructions in designing non-blocking algorithms 27
2.5.1 Fetch-and-add (FAA) 27
2.5.2 Compare-and-swap (CAS) 28
2.5.3 Load-link/Store-conditional (LL/SC) 29
2.6 Common issues when designing non-blocking algorithms 30
2.6.1 ABA problem 30
2.6.2 Safe memory reclamation problem 32
2.7 MPI-3 - A popular distributed programming library interface specification .. 33
2.7.1 MPI-3 RMA 33
2.7.2 MPI-RMA communication operations 33
2.7.3 MPI-RMA synchronization 34
2.8 Pure MPI - A porting approach of shared memory algorithms to distributed algorithms 36
2.9 BCL CoreX 38
Chapter III Related works 40
3.1 Non-blocking shared-memory MPSC queues 40
3.1.1 LTQueue 40

3.1.2 DQueue 43
3.1.3 WRLQueue 45
3.1.4 Jiffy 46
3.1.5 Remarks 48
3.2 Distributed MPSC queues 48
Chapter IV Distributed MPSC queues 51
4.1 A simple baseline distributed SPSC 52
4.2 dLTQueue - Straightforward LTQueue adapted for distributed environment .55
4.2.1 Overview 55
4.2.2 Data structure 56
4.2.3 Algorithm 59
4.3 Slotqueue - dLTQueue-inspired distributed MPSC queue with all constant-time operations 66
4.3.1 Overview 66
4.3.2 Data structure 67
4.3.3 Algorithm 68
4.4 dLTQueueV2 72
Chapter V Evaluation 74
5.1 Benchmarking baselines 74
5.2 Benchmarking environment 74
5.3 Microbenchmark program 75
5.4 Benchmarking results 75
Chapter VI Conclusion 79
References 80
Appendix A Theoretical aspects 83
A.1 Terminology 83
A.2 Preliminaries 83
A.2.1 System model 83
A.2.2 Aspect-oriented linearizability proof 84
A.2.3 ABA-safety 85
A.3 Theoretical proofs of the distributed SPSC 85
A.3.1 ABA problem 85
A.3.2 Memory reclamation 86
A.3.3 Linearizability 86
A.3.4 Progress guarantee 89
A.3.5 Theoretical performance 89
A.4 Theoretical proofs of dLTQueue 90
A.4.1 Proof-specific notations 90

A.4.2 ABA problem 92
A.4.3 Memory reclamation 93
A.4.4 Linearizability 93
A.4.5 Progress guarantee 102
A.4.6 Theoretical performance 102
A.5 Theoretical proofs of Slotqueue 103
A.5.1 Proof-specific notations 103
A.5.2 ABA problem 105
A.5.3 Memory reclamation 109
A.5.4 Linearizability 109
A.5.5 Progress guarantee 113
A.5.6 Theoretical performance 113

List of Tables

Table 1 Specification of `MPI_Win_lock_all` and `MPI_Win_unlock_all`. 36
Table 2 Summary of existing shared memory MPSC queues. The cell marked with (*) indicates that our evaluation contradicts with the authors' claims. 40
Table 3 Characteristic summary of existing distributed MPSC queues. R stands for remote operations and L stands for local operations. (*) [1] claims that it is lock-free. 48
Table 4 Characteristic summary of our proposed distributed MPSC queues. (1) n is the number of processes. (2) R stands for **remote operation** and L stands for **local operation**. . . . 51
Table 5 Theoretical performance summary of our simple distributed SPSC. R means remote operations and L means local operations. 89
Table 6 Theoretical performance summary of dLTQueue. R means remote operations and L means local operations. 102
Table 7 Theoretical performance summary of Slotqueue. R means remote operations and L means local operations. 113

List of Images

Figure 1 Actor model visualization. 20
Figure 2 Fan-out/Fan-in pattern visualization. 21
Figure 3 Linearization points of method 1, method 2, method 3, method 4 happen at $t_1 < t_2 < t_3 < t_4$, therefore, their effects will be observed in this order as if we call method 1, method 2, method 3, method 4 sequentially. 24
Figure 4 Blocking algorithm: When a process is suspended, it can potentially block other processes from making further progress. 25
Figure 5 Lock-free algorithm: All the live processes together always finish in a finite amount of steps. 26
Figure 6 Wait-free algorithm: Any live process always finishes in a finite amount of steps. 27
Figure 7 ABA problem in a linked-list stack. 31
Figure 8 Unsafe memory reclamation in a LIFO stack. 32
Figure 9 An illustration of passive target communication. Dashed arrows represent synchronization (source: [2]). 35
Figure 10 An illustration of our synchronization approach in MPI RMA. 38
Figure 11 LTQueue's structure. 42
Figure 12 Intuition on how timestamp-refreshing works. 43
Figure 13 DQueue's structure. 44
Figure 14 WRLQueue's structure. 46
Figure 15 WRLQueue's dequeue operation 46
Figure 16 Jiffy's structure. 46
Figure 17 AMQueue's structure. 49
Figure 18 dLTQueue's structure. 56
Figure 19 Basic structure of Slotqueue. 67
Figure 20 Microbenchmark results for the enqueue operation. 76
Figure 21 Microbenchmark results for the dequeue operation. 77
Figure 22 Microbenchmark results for the total throughput. 77
Figure 23 dLTQueue's structure. 91
Figure 24 Basic structure of Slotqueue. 104

Chapter I Introduction

This chapter details the motivation for our research topic: "Studying and developing non-blocking distributed MPSC queues" (Section 1.1), based on which the objectives (Section 1.2) and scope (Section 1.3) of this study are set out. To summarize, we then come to the formulation of the research question (Section 1.4) and the overall contributions of the thesis are listed in Section 1.5. Related academic publications that are relevant to this thesis are listed in Section 1.6. This chapter is ended with a brief description of the structure of the rest of this document.

1.1 Motivation

The demand for computation power has been increasing relentlessly. Increasingly complex computation problems arise and accordingly more computation power is required to solve them. Much engineering effort has been put forth toward obtaining more computation power. A popular topic in this regard is distributed computing: The combined power of clusters of commodity hardware can surpass that of a single powerful machine [3].

To harness the power of distributed systems, specialized algorithms and data structures need to be devised. Two especially important properties of distributed systems are performance and fault tolerance [4]. Therefore, the algorithms and data structures running on distributed systems need to be highly efficient and fault tolerant. Regarding efficiency, we are concerned with the algorithms' throughput and latency, which are the two main metrics to measure performance. Considering fault tolerance, we are especially interested in the progress guarantee [5] characteristic of the algorithms. The progress guarantee criterion divides the algorithms into two groups: blocking and non-blocking. Blocking algorithms allow one faulty process to delay the other processes forever, which is not fault tolerant [6]. Non-blocking algorithms are safeguarded against this problem, exhibiting a higher degree of fault tolerance [7].

One of the algorithms that has seen applications in the distributed domain is the multi-producer, single-consumer (MPSC) queue algorithm [1]. Furthermore, there are applications and programming patterns in the shared-memory domain that can potentially see similar usage in the distributed domain, such as the actor model [8] or the fan-out fan-in pattern [9]. Although the more general multi-producer, multi-consumer (MPMC) queues suffice for the MPSC workloads, they are typically too expensive for these use cases [10, 11]. Therefore, supporting a specialized non-blocking distributed MPSC queue is still valuable.

However, currently in the literature, there is only one distributed MPSC queue, AMQueue [1]. Moreover, even though the author claims that AMQueue is non-blocking, we found that AMQueue is actually blocking (Section 3.2). This is unlike the shared-memory domain, where there are a lot more research on non-blocking MPSC queues [10–13]. This apparent gap between the two domains have been bridged by some recent research to adapt non-blocking shared-memory algorithms to distributed environments [14–17]. The work by [17] introduces a method for creating non-blocking distributed data structures within the partitioned global address space (PGAS) framework, particularly targeting the Chapel programming language. However, their methodology faces a significant limitation: it relies on double-word compare-and-swap (DCAS) or 128-bit compare-and-swap (CAS) operations to prevent ABA problems, which lack support from most remote direct memory access (RDMA) hardware systems [17]. The HCL framework [16] provides a distributed data structure library built on RPC over RDMA technology. While functional, this approach demands specialized hardware capabilities from contemporary network interface cards, limiting its portability [15]. BCL Core [14] presents a highly portable solution capable of interfacing with multiple distributed programming backends including MPI, SHMEM, and GASNet-EX. However, BCL Core's architecture incorporates 128-bit pointers, creating the same RDMA hardware compatibility issues as [17]. For our research, we have selected BCL CoreX [15] and adopted its design philosophy to adapt existing shared-memory MPSC queues for distributed computing environments. BCL CoreX [15] extends the original BCL [14] framework with enhanced features that simplify the development of non-blocking distributed data structures. A key innovation in their approach is the implementation of 64-bit pointers, which are compatible with virtually all large-scale computing clusters and supported by most RDMA hardware configurations. To address ABA problems without relying on specialized instructions like DCAS, they have developed a distributed hazard pointer mechanism. This generic solution provides sufficient portability and flexibility to accommodate the adaptation of most existing non-blocking shared-memory data structures to distributed environments.

In summary, we focus on the design of efficient non-blocking distributed MPSC queues using the BCL CoreX library as the main implementation framework. The next few sections will list the objectives in more details and sum them up in a research question.

1.2 Objective

Based on what we have listed out in Section 1.1, we aim to:

- Investigate the principles underpinning the design of fault-tolerant and performant shared-memory algorithms.

- Investigate state-of-the-art shared-memory MPSC queue algorithms as case studies to support our design of distributed MPSC queue algorithms.
- Investigate existing distributed MPSC algorithms to serve as a comparison baseline.
- Model and design distributed MPSC queue algorithms using techniques from the shared-memory literature, specifically the BCL CoreX library.
- Utilize the shared-memory programming model to evaluate various theoretical aspects of distributed MPSC queue algorithms: correctness and progress guarantee.
- Model the theoretical performance of distributed MPSC queue algorithms that are designed using techniques from the shared-memory literature.
- Collect empirical results on distributed MPSC queue algorithms and discuss important factors that affect these results.

1.3 Scope

The following narrows down what we are going to investigate in the shared-memory literature and which theoretical and empirical aspects we are interested in for our distributed algorithms:

- Regarding the investigation of the design principles in the shared-memory literature, we focus on fault-tolerant and performant concurrent algorithm design using atomic operations and common problems that often arise in this area, namely, ABA problem and safe memory reclamation problem.
- Regarding the investigation of shared-memory MPSC queues currently in the literature, we focus on linearizable MPSC queues that follow strict FIFO semantics and support at least lock-free `enqueue` and `dequeue` operations.
- Regarding correctness, we concern ourselves with the linearizability correctness condition.
- Regarding fault tolerance, we concern ourselves with the concept of progress guarantee, that is, the ability of the system to continue to make forward progress despite the failure of one or more components of the system.
- Regarding algorithm prototyping, benchmarking and optimizations, we assume an MPI-3 setting.
- Regarding empirical results, we focus on performance-related metrics, e.g. throughput and latency.

1.4 Research question

Any research effort in this thesis revolves around this research question:

"How to utilize shared-memory programming principles to model and design distributed MPSC queue algorithms in a correct, fault-tolerant and performant manner?"

This question is further decomposed into smaller subquestions:

1. How to model the correctness of a distributed MPSC queue algorithm?
2. Which factors contribute to the fault tolerance and performance of distributed MPSC queue algorithms?
3. Which shared-memory programming principles are relevant in modeling and designing distributed MPSC queue algorithms in a fault-tolerant and performant manner?
4. Which shared-memory programming principles need to be modified to more effectively model and design distributed MPSC queue algorithms in a fault-tolerant and performant manner?

1.5 Contributions

This research makes two primary contributions to the field of distributed programming:

- An application of a novel design technique for non-blocking distributed data structures - via adaptation of non-blocking shared-memory data structures. The thesis demonstrates the feasibility of this approach in designing new non-blocking distributed data structures.
- Three novel wait-free distributed MPSC queues: dLTQueue, Slotqueue and dLTQueueV2, which are all fault-tolerant. Slotqueue and dLTQueueV2 are especially optimized for performance.

In conclusion, this work establishes a foundation for future research in fault-tolerant distributed data structures while providing immediately usable implementations for practitioners.

1.6 Related publications

The following is a list of our academic publications that are relevant to the topic of this thesis:

1. Huy DNA, Nguyen M-T, Diep T-D, Thoai N (2025) dLTQueue: A Non-Blocking Distributed-Memory Multi-Producer Single-Consumer Queue. In: 2025 International Conference on Future Data and Security Engineering. To appear.
2. Huy DNA, Diep T-D, Fürlinger K, Thoai N (2025) Slotqueue: A Wait-Free Distributed Multi-Producer Single-Consumer Queue with Constant Remote Operations. In: 2025 The IFIP International Conference on Network and Parallel Computing. To appear.

1.7 Structure

The rest of this report is structured as follows:

Chapter II discusses the theoretical foundation this thesis is based on. As mentioned, this thesis investigates the principles of shared-memory programming and the existing state-of-the-art shared-memory MPSC queues. We then explore the utilities offered by MPI-3 and BCL CoreX to implement distributed algorithms modeled by shared-memory programming techniques.

Chapter III surveys the shared-memory literature for state-of-the-art queue algorithms, specifically MPSC queues. We specifically focus on non-blocking shared-memory algorithms that have the potential to be adapted efficiently for distributed environments. This chapter additionally surveys existing distributed MPSC algorithms to serve as a comparison baseline for our novel distributed MPSC queue algorithms.

Chapter IV introduces our novel distributed MPSC queue algorithms, designed using shared-memory programming techniques and inspired by the selected shared-memory MPSC queue algorithms surveyed in Chapter III. It specifically presents our adaptation efforts of existing algorithms in the shared-memory literature to make their distributed implementations feasible and efficient.

Chapter V details our benchmarking metrics and elaborates on our benchmarking setup. We aim to demonstrate results on how well our novel MPSC queue algorithms perform, additionally compared to existing distributed MPSC queues. Finally, we discuss important factors that affect the runtime properties of distributed MPSC queue algorithms.

Chapter VI concludes what we have accomplished in this thesis and considers future possible improvements to our research.

Chapter II Background

This chapter provides the background for the terminology referenced throughout this thesis. To motivate the discussion of MPSC queues in Section 2.2, the chapter first discusses two irregular applications in Section 2.1. Next, it establishes what it means for a concurrent algorithm to be correct in Section 2.3 and the progress guarantee characteristics of concurrent algorithms in Section 2.4. From there, the decision is to design linearizable non-blocking distributed MPSC queues. Therefore, this work focuses on the tools needed to design non-blocking algorithms in Section 2.5 and the issues that arise in this design process such as ABA problem and safe memory reclamation problem in Section 2.6. The chapter finally introduces the practical libraries to help realize non-blocking distributed MPSC queues in Section 2.7, Section 2.8 and Section 2.9.

2.1 Irregular applications

MPSC queues and their applications belong to the class of irregular applications. Designing irregular applications needs to take into account their special properties. Therefore, before we discuss MPSC queue in Section 2.2, we explain the term “irregular application” in this section.

Irregular applications [18] are a class of programs particularly interesting in distributed computing. They are characterized by:

- Unpredictable memory access: Before the program is actually run, we cannot know which data it will need to access. We can only know that at run time.
- Data-dependent control flow: The decision of what to do next (such as which data to access next) is highly dependent on the values of the data already accessed, hence the unpredictable memory access property because we cannot statically analyze the program to know which data it will access. The control flow is inherently engraved in the data, which is not known until runtime.

Irregular applications are interesting because they demand special techniques to achieve high performance [18]. One specific challenge is that this type of application is hard to model in traditional MPI APIs using the Send/Receive interface [19]. This is specifically because using this interface requires a programmer to have already anticipated communication within pairs of processes before runtime, which is difficult with irregular applications. The introduction of MPI remote memory access (RMA) in MPI-2 and its improvement in MPI-3 has significantly improved MPI’s capability to express irregular applications comfortably [20]. This will be explained further in Section 2.7.

2.1.1 Actor model as an irregular application

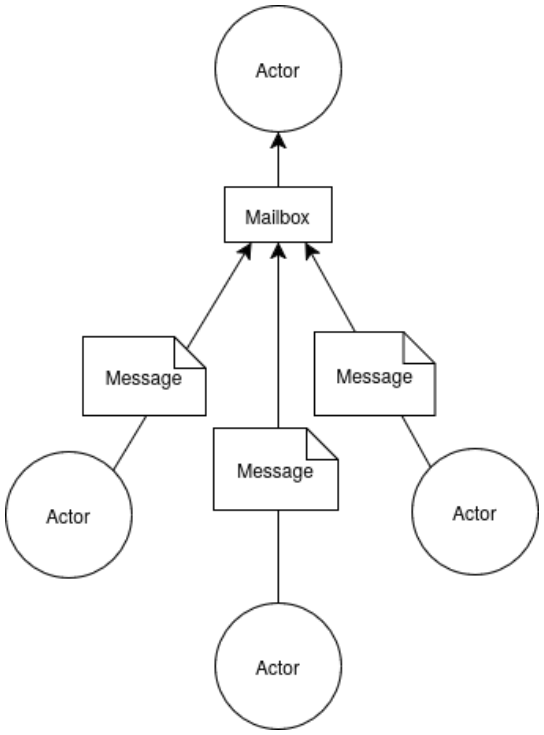


Figure 1: Actor model visualization.

The actor model [8] in actuality is a type of irregular application supported by the concurrent MPSC queue data structure.

Each actor can be a process or a compute node in the cluster, carrying out a specific responsibility in the system. From time to time, there is a need for the actors to communicate with each other. For this purpose, the actor model offers a mailbox local to each actor. This mailbox exhibits MPSC queue behavior: Other actors can send messages to the mailbox to notify the owner actor and the owner actor at their leisure repeatedly extracts messages from its mailbox. The actor model provides a simple programming model for concurrent processing.

The reasons why the actor model is an irregular application are straightforward to see:

- Unpredictable memory access: The cases in which one actor can anticipate which one of the other actors can send it a message are pretty rare and application-specific. As a general framework, in an actor model, the usual assumption is that any number of actors can try to communicate with an actor at some arbitrary time. By this nature, the communication pattern is unpredictable.
- Data-dependent control flow: If an actor A sends a message to another actor B, and when B reads this message, B decides to send another message to another actor

C. As we can see, the control flow is highly engraved in the messages, or in other words, the messages drive the program flow, which can only be known at runtime.

2.1.2 Fan-out/Fan-in pattern as an irregular application

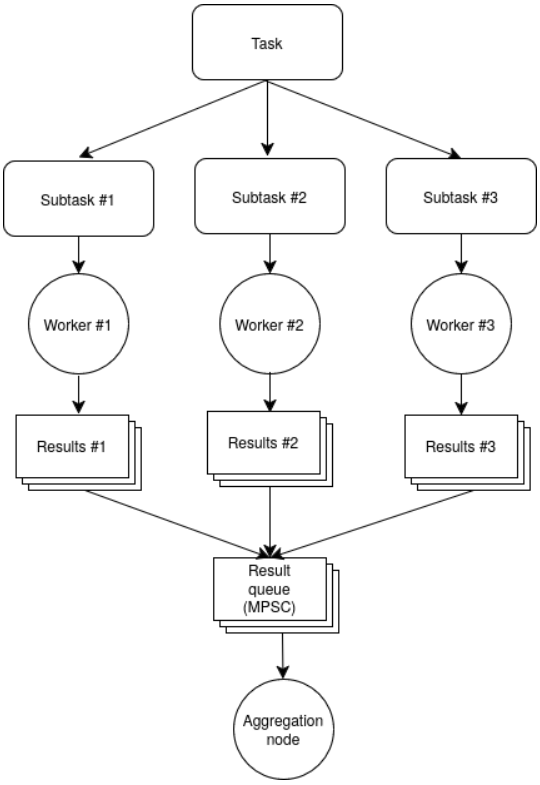


Figure 2: Fan-out/Fan-in pattern visualization.

The fan-out/fan-in pattern [9] is another type of irregular application supported by the concurrent MPSC queue data structure.

In this pattern, there is a big task that can be split into subtasks to be executed concurrently on some work nodes. In the execution process, each worker produces a result set, each enqueued back to a result queue located on an aggregation node. The aggregation node can then dequeue from this result queue to perform further processing. Clearly, this result queue exhibits MPSC behavior.

The fan-out/fan-in pattern exhibits less irregularity than the actor model, however. Usually, the worker nodes and the aggregation node are known in advance. The aggregation node can anticipate Send calls from the worker nodes. Still, there is a degree of irregularity that this pattern exhibits: How can the aggregation node know how many Send calls a worker node will issue? This is highly driven by the task and the data involved in this task, hence, we have the data-dependent control flow property. One can still statically calculate or predict how many Send calls a worker node will

issue. Nevertheless, this is problem-specific. Therefore, the memory access pattern is somewhat unpredictable. Notice that if supported by a concurrent MPSC queue data structure, the fan-out/fan-in pattern is free from this burden of organizing the right amount of Send/Receive calls. Thus, combining with the MPSC queue, the fan-out/fan-in pattern becomes more general and easier to program.

We have seen the role MPSC queues play in supporting irregular applications. It is important to understand what really comprises an MPSC queue data structure.

2.2 MPSC queue

With irregular applications defined in Section 2.1, the focus can now turn to the primary design goal of this thesis: MPSC queues.

Multi-producer, single-consumer (MPSC) queue is a specialized concurrent first-in first-out (FIFO) data structure. A FIFO is a container data structure where items can be inserted into or taken out of, with the constraint that the items that are inserted earlier are taken out earlier. Hence, it is also known as the queue data structure. The process that performs item insertion into the FIFO is called the producer and the process that performs item deletion (and retrieval) is called the consumer.

In concurrent queues, multiple producers and consumers can run concurrently. One class of concurrent FIFOs is the MPSC queue, where one consumer may run in parallel with multiple producers.

The reasons we are interested in MPSC queues instead of the more general multi-producer, multi-consumer (MPMC) queue data structures are that (1) high-performance and high-scalability MPSC queues are much simpler to design than MPMCs while (2) MPSC queues are powerful enough to solve certain problems, as demonstrated in Section 2.1. The MPSC queue in actuality is an irregular application in itself:

- Unpredictable memory access: As a general data structure, the MPSC queue allows any process to enqueue and dequeue at any time. By nature, its memory access pattern is unpredictable.
- Data-dependent control flow: The consumer's behavior is entirely dependent on whether and which data is available in the MPSC queue. The execution paths of MPSC queues can vary, based on the queue contention i.e. some processes may back off or retry some failed operations; this scenario often arises in lock-free data structures.

As an implication, some irregular applications can actually "push" the "irregularity burden" to the distributed MPSC queue, which is already designed for high perfor-

mance and fault tolerance. This provides a comfortable level of abstraction for programmers that need to deal with irregular applications.

2.3 Correctness condition of concurrent algorithms

During the design of MPSC queues, correctness must be carefully considered to ensure proper functionality in distributed environments. This section examines the correctness requirements that must be addressed when designing MPSC queues.

Correctness of concurrent algorithms is hard to define, regarding the semantics of concurrent data structures like MPSC queues. One effort to formalize the correctness of concurrent data structures is the definition of **linearizability** [21]. A method call on the FIFO can be visualized as an interval spanning two points in time. The starting point is called the **invocation event** and the ending point is called the **response event**. **Linearizability** informally states that each method call should appear to take effect instantaneously at some moment between its invocation event and response event [5]. The moment the method call takes effect is termed the **linearization point**. Specifically, suppose the following:

- We have n concurrent method calls m_1, m_2, ..., m_n.
- Each method call m_i starts with the **invocation event** happening at timestamp s_i and ends with the **response event** happening at timestamp e_i. We have $s_i < e_i$ for all $1 \leq i \leq n$.
- Each method call m_i has the **linearization point** happening at timestamp l_i, so that $s_i \leq l_i \leq e_i$.

Then, linearizability means that if we have $l_1 < l_2 < ... < l_n$, the effect of these n concurrent method calls m_1, m_2, ..., m_n must be equivalent to calling m_1, m_2, ..., m_n **sequentially**, one after the other in that order.

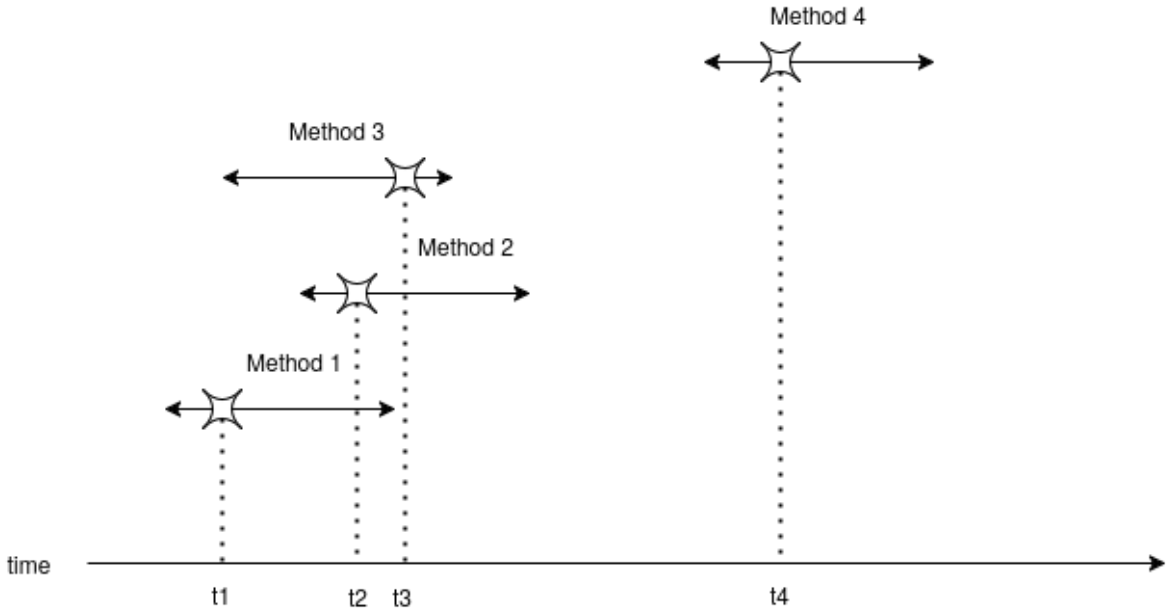


Figure 3: Linearization points of method 1, method 2, method 3, method 4 happen at $t_1 < t_2 < t_3 < t_4$, therefore, their effects will be observed in this order as if we call method 1, method 2, method 3, method 4 sequentially.

Linearizability is widely used as a correctness condition because of (1) its composability (if every component in the system is linearizable, the whole system is linearizable), which promotes modularity and ease of proof (2) its compatibility with human intuition, i.e. linearizability respects real-time order [21]. Naturally, we choose linearizability to be the only correctness condition for our algorithms.

2.4 Fault-tolerance & progress guarantee of concurrent algorithms

Correct algorithms can still be prone to faults at runtime, ranging from processes experiencing unexpected delays in execution to processes crashing indefinitely. Therefore, fault tolerance is also an important criterion for distributed MPSC queues (Section 2.2), in addition to correctness (Section 2.3). This section introduces the concept of progress guarantees, which are closely linked with fault tolerance.

Progress guarantee [5] is a criterion that only arises in the context of concurrent algorithms. Informally, it is the degree of hindrance one process imposes on another process from completing its task. In the context of sequential algorithms, this is irrelevant because there is only ever one process. Progress guarantee has an implication on an algorithm's performance and fault tolerance, especially in adverse situations, as we will explain in the following sections.

2.4.1 Blocking algorithms

Many concurrent algorithms are based on locks to create mutual exclusion, in which only some processes that have acquired the locks are able to act, while the others have to wait. While lock-based algorithms are simple to read, write and verify, these algorithms are said to be **blocking**: One slow process may slow down the other faster processes, for example, if the slow process successfully acquires a lock and then the operating system (OS) decides to suspend it to schedule another one, this means until the process is awakened, the other processes that contend for the lock cannot continue.

Blocking is the weakest progress guarantee one algorithm can offer; it allows one process to impose arbitrary impedance to any other processes, as shown in Figure 4.

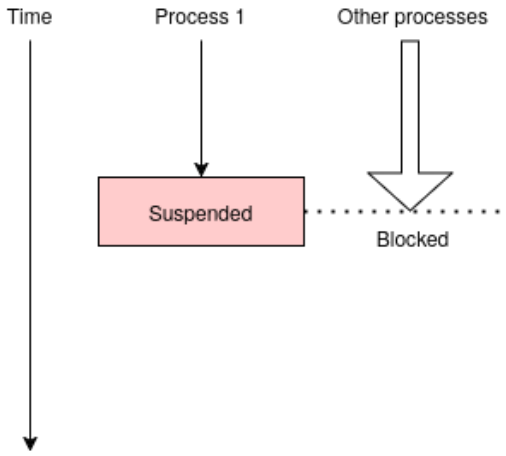


Figure 4: Blocking algorithm: When a process is suspended, it can potentially block other processes from making further progress.

Blocking algorithms introduce many problems such as:

- Deadlock: There is a circular lock-wait dependency among the processes, effectively preventing any processes from making progress.
- Convoy effect: One long process holding the lock will block other shorter processes contending for the lock.
- Priority inversion: A higher-priority process effectively has very low priority because it has to wait for another low priority process.

Furthermore, if a process that holds the lock dies, this will render the whole program unable to make any progress. This consideration holds even more weight in distributed computing because of a lot more failure modes, such as network failures, node failures, etc.

Therefore, while blocking algorithms, especially those using locks, are easy to write, they do not provide **progress guarantee** because **deadlock** or **livelock** can occur and their use of mutual exclusion is unnecessarily restrictive. Fortunately, there are other classes of algorithms which offer stronger progress guarantees.

2.4.2 Non-blocking algorithms

An algorithm is said to be **non-blocking** if a failure or slowdown in one process cannot cause the failure or slowdown in another process. Lock-free and wait-free algorithms are two especially interesting subclasses of non-blocking algorithms. Unlike blocking algorithms, they provide stronger degrees of progress guarantees.

2.4.2.1 Lock-free algorithms

Lock-free algorithms provide the following guarantee: Even if some processes are suspended, the remaining processes are ensured to make global progress and complete in bounded time. In other words, a process cannot cause hindrance to the global progress of the program. This property is invaluable in distributed computing; one dead or suspended process will not block the whole program, providing fault tolerance. Designing lock-free algorithms requires careful use of atomic instructions, such as Fetch-and-add (FAA), Compare-and-swap (CAS), etc which will be explained in Section 2.5.

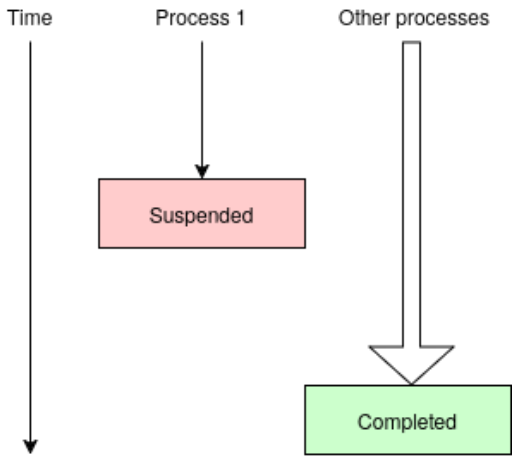


Figure 5: Lock-free algorithm: All the live processes together always finish in a finite amount of steps.

2.4.2.2 Wait-free algorithms

Wait-freedom offers the strongest degree of progress guarantee. It mandates that no process can cause constant hindrance to any running process. While lock-freedom ensures that at least one of the alive processes will make progress, wait-freedom guar-

antees that any alive process will finish in a finite number of steps. Wait-freedom can be desirable because it prevents starvation. Lock-freedom still allows the possibility of one process having to wait for another indefinitely, as long as some still make progress.

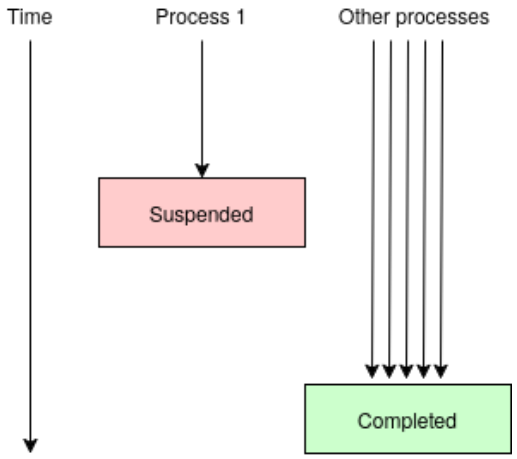


Figure 6: Wait-free algorithm: Any live process always finishes in a finite amount of steps.

2.5 Popular atomic instructions in designing non-blocking algorithms

As discussed in Section 2.4, blocking algorithms are not fault tolerant while non-blocking ones are, specifically lock-free and wait-free algorithms. Therefore, the design goal can be refined to linearizable non-blocking distributed MPSC queues. Techniques to achieve this are discussed next in this section.

In non-blocking algorithms, finer-grained synchronization primitives than simple locks are required, which manifest themselves as atomic instructions. Therefore, it is necessary to get familiar with the semantics of these atomic instructions and common programming patterns associated with them.

2.5.1 Fetch-and-add (FAA)

Fetch-and-add (FAA) is a simple atomic instruction with the following semantics: It atomically increments a value at a memory location x by a and returns the previous value just before the increment. Informally, FAA's effect is equivalent to the function in Procedure 1, assuming that the function is executed atomically.

Procedure 1: `int fetch_and_add(int* x, int a)`

```
1 old_value = *x
2 *x = *x + a
3 return old_value
```

Fetch-and-add can be used to create simple distributed counters.

2.5.2 Compare-and-swap (CAS)

Compare-and-swap (CAS) is probably the most popular atomic operation instruction. The reason for its popularity is (1) CAS is a **universal atomic instruction** with the **consensus number** of ∞, which means it is the most powerful atomic instruction [22] (2) CAS is implemented in most hardware (3) some concurrent lock-free data structures such as MPSC queues are more easily expressed using a powerful atomic instruction such as CAS.

The semantics of CAS is as follows. Given the instruction `CAS(memory location, old value, new value)`, atomically compares the value at `memory location` to see if it equals `old value`; if so, sets the value at `memory location` to `new value` and returns true; otherwise, leaves the value at `memory location` unchanged and returns false. Informally, its effect is equivalent to the function in Procedure 2.

Procedure 2: `bool compare_and_swap(int* x, int old_val, int new_val)`

```
1 if (*x == old_val)
2 |   *x = new_val
3 |   return true
4 return false
```

Compare-and-swap is very powerful and consequently, pervasive in concurrent algorithms and data structures.

Non-blocking concurrent algorithms often utilize CAS as follows. The steps 1-3 are retried until success.

1. Read the current value `old value = read(memory location)`.
2. Compute `new value` from `old value` by manipulating some resources associated with `old value` and allocating new resources for `new value`.

3. Call `CAS(memory location, old value, new value)`. If that succeeds, the new resources for `new value` remain valid because it was computed using valid resources associated with `old value`, which has not been modified since the last read. Otherwise, free up the resources we have allocated for `new value` because `old value` is no longer there, so its associated resources are not valid.

This scheme is, however, susceptible to the ABA problem, which will be discussed in Section 2.6.1.

2.5.3 Load-link/Store-conditional (LL/SC)

Load-link/Store-conditional is actually a pair of atomic instructions for synchronization.

Semantically, load-link returns a value currently located at a memory location x while store-conditional sets the memory location x to a value v if there is no other write to x since the last load-link call, otherwise, the store-conditional call would fail.

Intuitively, LL/SC provides an easier synchronization primitive than CAS: LL/SC ensures that a store-conditional can only succeed if there is no access to a memory location, while CAS can still succeed in this case if the value at the memory location does not change. Due to this property, LL/SC is not vulnerable to the ABA problem (see Section 2.6.1). However, CAS is in fact as powerful as LL/SC, considering that they can implement each other [22].

Practically, store-conditional can still fail even if thereis no write to the same memory location since the last load-link call. This is called a spurious failure. For example, consider the following generic sequence of events:

1. Thread X calls load-link on x and loads out v.
2. Thread X computes a new value v'.
3. Some *exceptional event* happens (discussed below). Assume that no other threads access x during this time.
4. Thread X calls store-conditional to store v' to x. It *should succeed* but *fails* anyway.

Exceptional events that can cause the store-conditional to fail spuriously include:

- Cache line flushing: If the cache line that caches the memory location x is written back to memory, logically, the memory location x has been accessed and therefore, the store-conditional fails.
- Context switch: If thread X is swapped out by the OS, cache lines may be invalidated and flushed out, which consequently leads to the first scenario.

LL/SC even though as powerful as CAS, is not as widespread as CAS; in fact, as of MPI-3, only CAS is supported.

2.6 Common issues when designing non-blocking algorithms

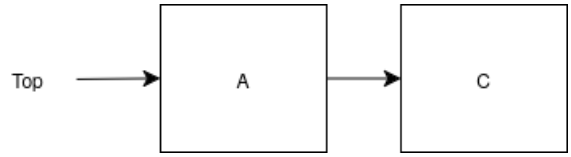
Atomic instructions represent the preferred approach for designing non-blocking algorithms (Section 2.5). However, this approach typically encounters two significant problems: the ABA problem (Section 2.6.1) and the safe memory reclamation problem (Section 2.6.2). Addressing these issues is essential for completing the design process outlined in the previous sections (Section 2.2, Section 2.3, Section 2.4, Section 2.5).

2.6.1 ABA problem

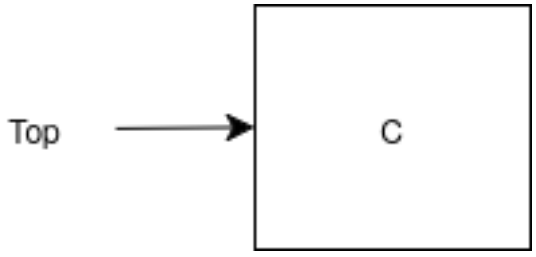
The ABA problem [23] is a notorious problem associated with the compare-and-swap atomic instruction. Because CAS is so widely used in non-blocking algorithms, the ABA problem almost has to always be accounted for.

As a reminder, here's how CAS is often utilized in non-blocking concurrent algorithms: The steps 1-3 are retried until success.

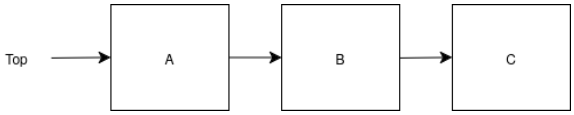
1. Read the current value `old value = read(memory location)`.
2. Compute `new value` from `old value` by manipulating some resources associated with `old value` and allocating new resources for `new value`.
3. Call `CAS(memory location, old value, new value)`. If that succeeds, the new resources for `new value` remain valid because it was computed using valid resources associated with `old value`, which has not been modified since the last read. Otherwise, free up the resources we have allocated for `new value` because `old value` is no longer there, so its associated resources are not valid.



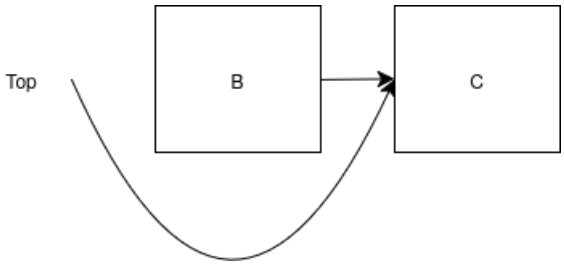
(a) Process X wants to pop a value, it observes Top = A and Top → next = C then suspends.



(b) Another process pops the value A and sets Top to C.



(c) Another process pushes two values B and A and sets Top to A.



(d) Process X successfully performs the pop by calling `CAS(&Top, A, C)`. Top no longer points to the top of the stack.

Figure 7: ABA problem in a linked-list stack.

As hinted, this scheme is susceptible to the notorious ABA problem. The following scenario illustrates an example of the ABA problem:

1. Process 1 reads the current value of `memory location` and reads out A.
2. Process 1 manipulates resources associated with A, and allocates resources based on these resources.
3. Process 1 suspends.
4. Process 2 reads the current value of `memory location` and reads out A.
5. Process 2 `CAS(memory location, A, B)` so that resources associated with A are no longer valid.
6. Process 3 `CAS(memory location, B, A)` and allocates new resources associated with A.
7. Process 1 continues and `CAS(memory location, A, new value)` relying on the fact that the old resources associated with A are still valid while in fact they aren't.

The ABA problem arises fundamentally because most algorithms assume a memory location is not accessed if its value is unchanged.

A specific case of the ABA problem is given in Figure 7.

To safeguard against the ABA problem, one must ensure that between the time a process reads out a value from a shared memory location and the time it calls CAS on

that location, there is no possibility another process has CAS-ed the memory location to the same value.

A simple scheme that is widely used practically and also in this thesis is the **unique timestamp** scheme. This scheme's idea is simple: for each shared memory location that is affected by CAS operations, we reserve some bits of this memory location for a monotonic counter. Each time a CAS operation is carried out, this counter is incremented. Theoretically, the ABA problem would never happen because combining with this counter, the value of this memory location is always unique, due to the counter never repeating itself. However, practically, the counter can overflow and wrap around to the same value and the ABA problem would happen in this case. Therefore, the counter's range must be big enough so that this scenario can't virtually happen. Empirically, a counter of 32-bit should be enough. The drawback of this approach is that we have wasted 32 meaningful bits to avoid the ABA problem.

2.6.2 Safe memory reclamation problem

The problem of safe memory reclamation [24] often arises in concurrent algorithms that dynamically allocate memory. In such algorithms, dynamically-allocated memory must be freed at some point. However, there is a good chance that while a process is freeing memory, other processes contending for the same memory are keeping a reference to that memory. Therefore, deallocated memory can potentially be accessed, which is erroneous.

An example of unsafe memory reclamation is given in Figure 8.



(a) Process X about to push a value onto the stack, already reading the top pointer but suspended.



(b) The top node is popped, the reference X holds is no longer valid. When X resumes, a freed memory location will be accessed.

Figure 8: Unsafe memory reclamation in a LIFO stack.

Solutions to this problem must ensure that memory is only freed when no other processes are holding references to it. In garbage-collected programming environments, this problem can be conveniently pushed to the garbage collector. In non-garbage-collected programming environments, however, custom schemes must be utilized.

2.7 MPI-3 - A popular distributed programming library interface specification

For implementing linearizable non-blocking distributed MPSC queues, MPI represents the most fundamental implementation choice and serves as the focus of this section. The discussion specifically examines the MPI-3 RMA API, which, as will be demonstrated, facilitates straightforward implementation of irregular applications such as MPSC queues. A pure MPI-3 RMA approach is detailed in Section 2.8, while more advanced implementation techniques are explored in Section 2.9.

MPI stands for message passing interface, which is a **message-passing library interface specification**. Design goals of MPI include high availability across platforms, efficient communication, thread safety, reliable and convenient communication interface while still allowing hardware-specific accelerated mechanisms to be exploited [2].

2.7.1 MPI-3 RMA

RMA in MPI RMA stands for remote memory access. RMA APIs were introduced in MPI-2 and their capabilities are further extended in MPI-3 to conveniently express irregular applications [20]. In general, RMA is intended to support applications with dynamically changing data access patterns where the data distribution is fixed or slowly changing [2]. This is very similar to the properties of irregular applications as discussed in Section 2.1. In such applications, one process, based on the data it needs, knowing the data distribution, can compute the nodes where the data is stored. However, because the data access pattern is not known, each process cannot know whether any other processes will access its data. Using the traditional Send/Receive interface, both sides need to issue matching operations by distributing appropriate transfer parameters. This is not suitable, as previously explained; only the side that needs to access the data knows all the transfer parameters while the side that stores the data cannot anticipate this.

2.7.2 MPI-RMA communication operations

RMA only requires one side to specify all the transfer parameters and thus only that side to participate in data communication [2].

To utilize MPI RMA, each process needs to open a memory window to expose a segment of its memory to RMA communication operations such as remote writes (`MPI_PUT`), remote reads (`MPI_GET`) or remote accumulates (`MPI_ACCUMULATE`, `MPI_GET_ACCUMULATE`, `MPI_FETCH_AND_OP`, `MPI_COMPARE_AND_SWAP`) [2]. These remote communication operations only require one side to specify.

2.7.3 MPI-RMA synchronization

Besides communication of data from the sender to the receiver, one also needs to synchronize the sender with the receiver. That is, there must be a mechanism to ensure the completion of RMA communication calls or that any remote operations have taken effect. For this purpose, MPI RMA provides **active target synchronization** and **passive target synchronization**. In this document, we are particularly interested in **passive target synchronization** as this mode of synchronization does not require the target process of an RMA operation to explicitly issue a matching synchronization call with the origin process, easing the expression of irregular applications [20].

In **passive target synchronization**, any RMA communication calls must be within a pair of `MPI_Win_lock`/`MPI_Win_unlock` or `MPI_Win_lock_all`/`MPI_Win_unlock_all`. After the unlock call, those RMA communication calls are guaranteed to have taken effect. One can also force the completion of those RMA communication calls without the need for the call to unlock using flush calls such as `MPI_Win_flush` or `MPI_Win_flush_local`.

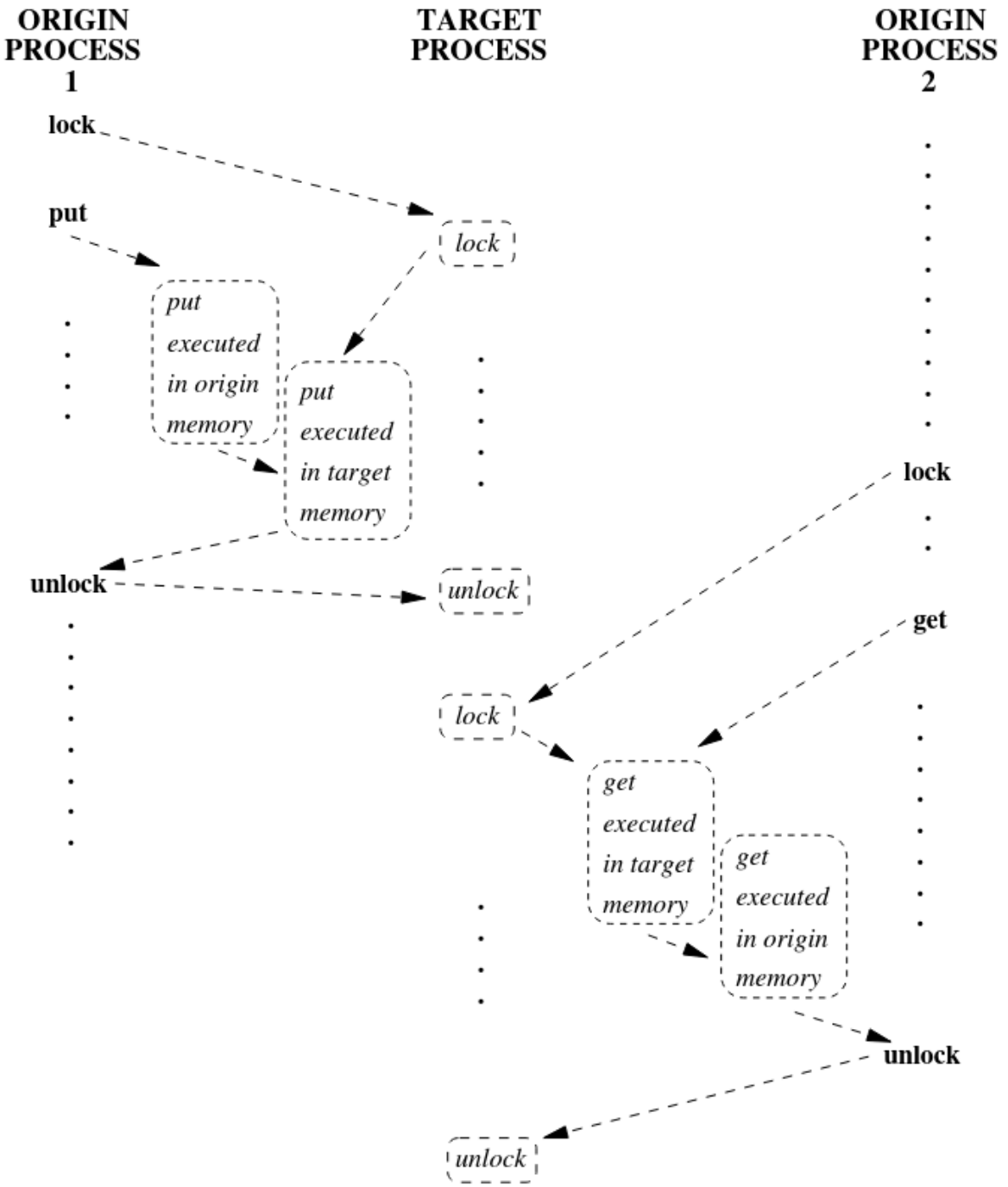


Figure 9: An illustration of passive target communication. Dashed arrows represent synchronization (source: [2]).

2.8 Pure MPI - A porting approach of shared memory algorithms to distributed algorithms

With MPI (Section 2.7), the most basic facility to adapt shared-memory algorithms to distributed algorithms is MPI-3 RMA. However, MPI-3 RMA offers a wide range of utilities, which may not be well-suited for implementing non-blocking distributed MPSC queues. This section introduces one technique utilizing MPI-3 RMA to implement non-blocking distributed algorithms. The BCL CoreX library (Section 2.9) is built on top of this approach.

In pure MPI, we use MPI exclusively for communication and synchronization. With MPI RMA, the communication calls that we utilize are [2]:

- Remote read: `MPI_Get`
- Remote write: `MPI_Put`
- Remote accumulation: `MPI_Accumulate`, `MPI_Get_accumulate`, `MPI_Fetch_and_op` and `MPI_Compare_and_swap`.

For lock-free synchronization, we choose to use **passive target synchronization** with `MPI_Win_lock_all`/`MPI_Win_unlock_all`.

In the MPI-3 specification [2], these functions are specified as in Table 1.

Operation	Usage
`MPI_Win_lock_all`	Starts an RMA access epoch to all processes in a memory window, with a lock type of `MPI_LOCK_SHARED`. The calling process can access the window memory on all processes in the memory window using RMA operations. This routine is not collective.
`MPI_Win_unlock_all`	Matches with an `MPI_Win_lock_all` to unlock a window previously locked by that `MPI_Win_lock_all`.

Table 1: Specification of `MPI_Win_lock_all` and `MPI_Win_unlock_all`.

The reason we choose this is 3-fold:

- Unlike **active target synchronization**, **passive target synchronization** does not require the process whose memory is being accessed by an MPI RMA communication call to participate. This is in line with our intention to use MPI RMA to easily model irregular applications like MPSC queues.
- Unlike **active target synchronization**, `MPI_Win_lock_all` and `MPI_Win_unlock_all` do not need to wait for a matching synchronization call in the target process, and thus, are not delayed by the target process.

- Unlike **passive target synchronization** with `MPI_Win_lock`/`MPI_Win_unlock`, multiple calls of `MPI_Win_lock_all` can succeed concurrently, so one process needing to issue MPI RMA communication calls does not block others.

An example of our pure MPI approach with `MPI_Win_lock_all`/`MPI_Win_unlock_all`, inspired by [20], is illustrated in the following:

```
MPI_Win_lock_all(0, win);

MPI_Get(...); // Remote get
MPI_Put(...); // Remote put
MPI_Accumulate(..., MPI_REPLACE, ...); // Atomic put
MPI_Get_accumulate(..., MPI_NO_OP, ...); // Atomic get
MPI_Fetch_and_op(...); // Remote fetch-and-op
MPI_Compare_and_swap(...); // Remote compare and swap
...

MPI_Win_flush(...); // Make previous RMA operations take effect
MPI_Win_flush_local(...); // Make previous RMA operations take effect
locally
...

MPI_Win_unlock_all(win);
```

Listing 3: An example snippet showcasing our synchronization approach in MPI RMA.

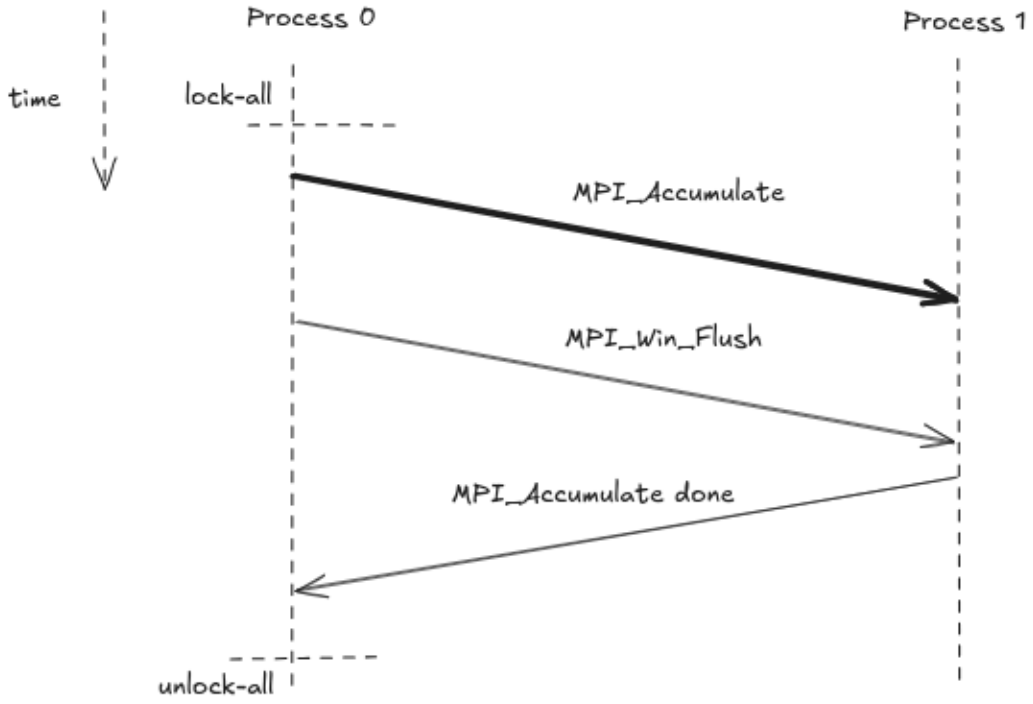


Figure 10: An illustration of our synchronization approach in MPI RMA.

2.9 BCL CoreX

BCL CoreX [15] is a high-level library built on top of MPI to facilitate the design of non-blocking algorithms for distributed-memory machines. In principle, it utilizes the pure MPI approach that we have covered in Section 2.8.

A subset of the primitives provided by BCL CoreX is presented below. We will utilize these primitives in our algorithm specification.

`gptr<T>`

> A global pointer that points to a variable of type `T`. A global pointer is one that can point to a variable outside of the current process's address space. The process whose address space a global pointer points to is called the *host*. Like normal pointers, pointer arithmetic also works with global pointers, which allows global pointers to point to remote arrays.

`T read(gptr<T> ptr, T* dest)`

> Issue a synchronous read on the location pointed to by `ptr` and stores the read value in `dest`.

`T write(gptr<T> ptr, T* src)`

> Issue a synchronous write on the location pointed to by `ptr` that writes the value stored in `src`.

`T faa(gptr<T> ptr, T inc)`

> Issue a synchronous fetch-and-add operation on the location pointed to by `ptr`. `T` must be an integral type of less than 64 bits.

`T cas(gptr<T> ptr, T old_val, T new_val)`

> Issue a synchronous compare-and-swap operation on the location pointed to by `ptr`. `T` must be a type of less than 64 bits.

A remote operation occurs when one of the primitive operations is applied on the global pointer that points to a non-local address space. Otherwise, a local operation occurs. Typically, remote operations are very expensive compared to local operations.

Chapter III Related works

Given the decision to adapt shared-memory data structures for creating non-blocking distributed MPSC queues with BCL CoreX (Section 2.9), the next step is to explore existing non-blocking shared-memory MPSC queues in Section 3.1. Additionally, current distributed MPSC queue implementations are analyzed in Section 3.2 to establish benchmarking baselines. Based on this analysis, LTQueue is selected as the candidate shared-memory MPSC queue for distributed adaptation and AMQueue is chosen as the primary benchmark reference.

3.1 Non-blocking shared-memory MPSC queues

There exists numerous research into the design of non-blocking shared memory MPSCs. Interestingly, research into non-blocking MPSC queues is noticeably scarce. In reality, we have only found 4 papers that are concerned with the direct support of non-blocking MPSC queues: LTQueue [12], DQueue [11], WRLQueue [10], and Jiffy [13]. Table 2 summarizes the characteristics of these algorithms.

MPSC queues	LTQueue [12]	DQueue [11]	WRLQueue [10]	Jiffy [13]
ABA solution	Load-link/ Store-conditional	Incorrect custom scheme (*)	Custom scheme	Custom scheme
Safe memory reclamation	Custom scheme	Incorrect custom scheme (*)	Custom scheme	Insufficient custom scheme
Progress guarantee of dequeue	Wait-free	Wait-free	Blocking (*)	Wait-free
Progress guarantee of enqueue	Wait-free	Wait-free	Wait-free	Wait-free

Table 2: Summary of existing shared memory MPSC queues. The cell marked with (*) indicates that our evaluation contradicts with the authors' claims.

3.1.1 LTQueue

To our knowledge, LTQueue [12] is the earliest paper that directly focuses on the design of a wait-free shared memory MPSC queue.

This algorithm is wait-free with $O(\log n)$ time complexity for both enqueues and dequeues, with n being the number of enqueuers due to a novel timestamp-update scheme and a tree-structure organization of timestamps.

The basic structure of LTQueue is given in Figure 11. In LTQueue, each enqueuer maintains an SPSC queue that only it and the dequeuer can access. This SPSC queue must additionally support the `readFront` operation, which returns the front element currently in the SPSC. The SPSC can be any implementation that conforms to this interface. In the original paper, the SPSC is represented as a simple linked list.

The rectangular nodes at the bottom in Figure 11 represent an enqueuer, whose SPSC contains items with 2 fields: `value` and `timestamp`. Every enqueuer has to timestamp its data before enqueueing. The timestamps can be obtained using a distributed counter shared by all the enqueuers.

The purpose of timestamping is to determine the order to dequeue the items from the local SPSCs. To efficiently maintain the timestamps and determine which SPSC to dequeue from first, a tree structure with a min-heap property is built upon the enqueuer nodes. The original algorithm leaves the exact representation of the tree open, for example, the arity of the tree, which is shown to be 2 in Figure 11. The circle-shaped nodes in this figure represent the nodes in this tree structure, which are shared by all processes. Each node stores the minimum timestamp along with the owner enqueuer's rank (an identifier given to a process) in the subtree rooted at that node. After every modification to the local SPSC, i.e., an enqueue or a dequeue, the changes must be propagated up to the root node.

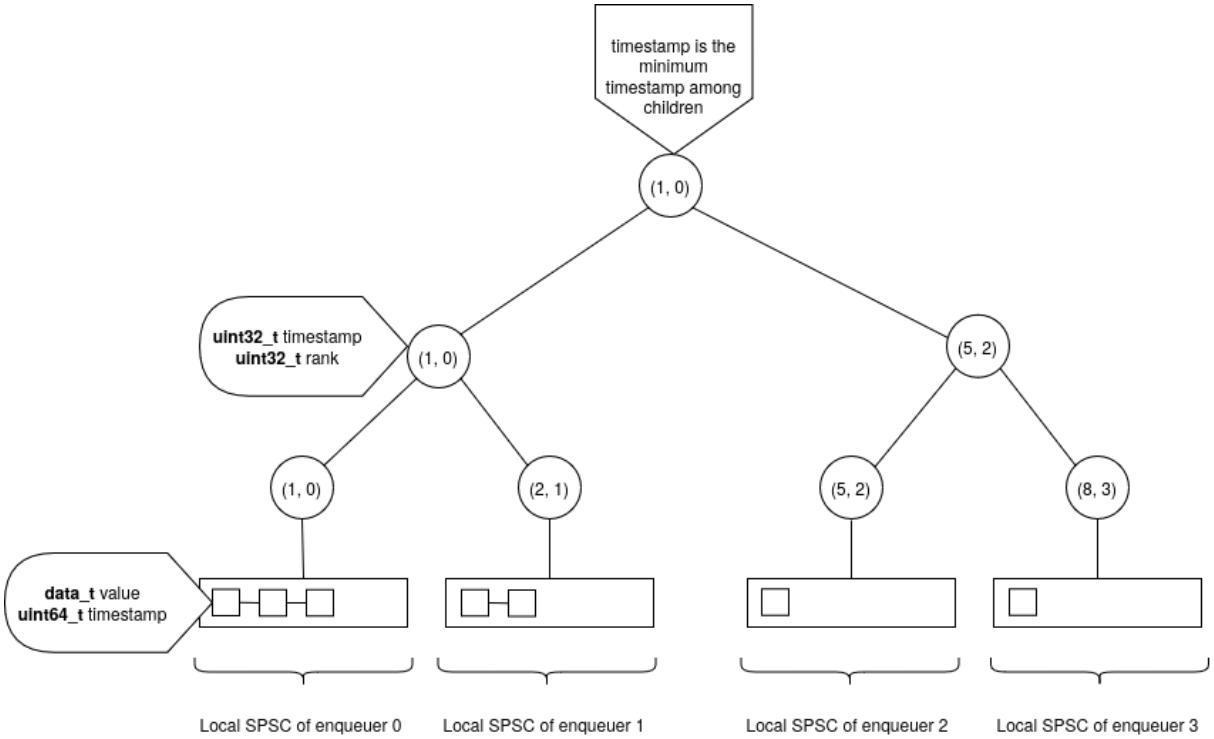


Figure 11: LTQueue's structure.

To dequeue, the dequeuer simply looks at the root node to determine the rank of the enqueuer to dequeue its SPSC.

The fundamental idea contributing to LTQueue's wait-freedom is the wait-free timestamp-propagation procedure. If there is a change to an enqueuer's SPSC, the timestamp of any nodes that lie on the path from the enqueuer to the root node is refreshed. The timestamp-refreshing procedure is simple:

- Call load-link on the node's `(timestamp, rank)`.
- Look at all the timestamps of the node's children and determine the minimum timestamp and its owner rank.
- Call store-conditional to store the new minimum timestamp and the new owner rank to the current node.

Notice that due to contention, the timestamp-refreshing procedure can fail. In that case, the timestamp-propagation procedure simply retries the timestamp-refreshing procedure one more time. This second call, again, can fail. However, after this second call, the node's timestamp is guaranteed to be up-to-date. The intuition behind this is demonstrated in Figure 12. Furthermore, because every node is refreshed at most twice, the timestamp-refresh procedure should finish in a finite number of steps.

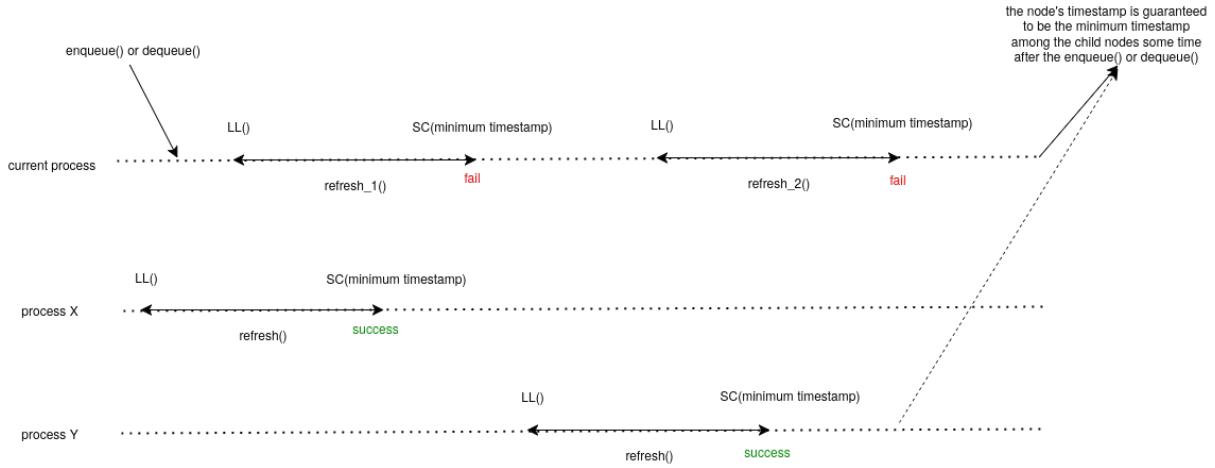


Figure 12: Intuition on how timestamp-refreshing works.

The LTQueue algorithm avoids ABA entirely by utilizing load-link/store-conditional. This represents a challenge to directly implementing this algorithm in a distributed environment.

The memory reclamation responsibility is handled by the SPSC structure, which is fairly trivial with a custom scheme.

The design of each enqueuer maintaining a separate SPSC allows multiple enqueuers to successfully enqueue their data in parallel without stepping on each other's toes. This can potentially scale well to a large number of processes. However, scalability may be limited due to potentially growing contention during timestamp propagation. The performance of LTQueue in shared-memory environments may still have a lot of room for improvement, i.e., more cache-aware design, avoiding unnecessary contention, etc. Nevertheless, its timestamp-refreshing scheme is interesting in and of itself and can potentially inspire the design of new algorithms. In fact, LTQueue's idea is core to one of our optimized distributed MPSC queue algorithms, Slotqueue (Section 4.3).

3.1.2 DQueue

DQueue [11] focuses on optimizing performance, aiming to be cache-friendly and avoid expensive atomic instructions such as CAS.

The basic structure of DQueue is demonstrated in Figure 13.

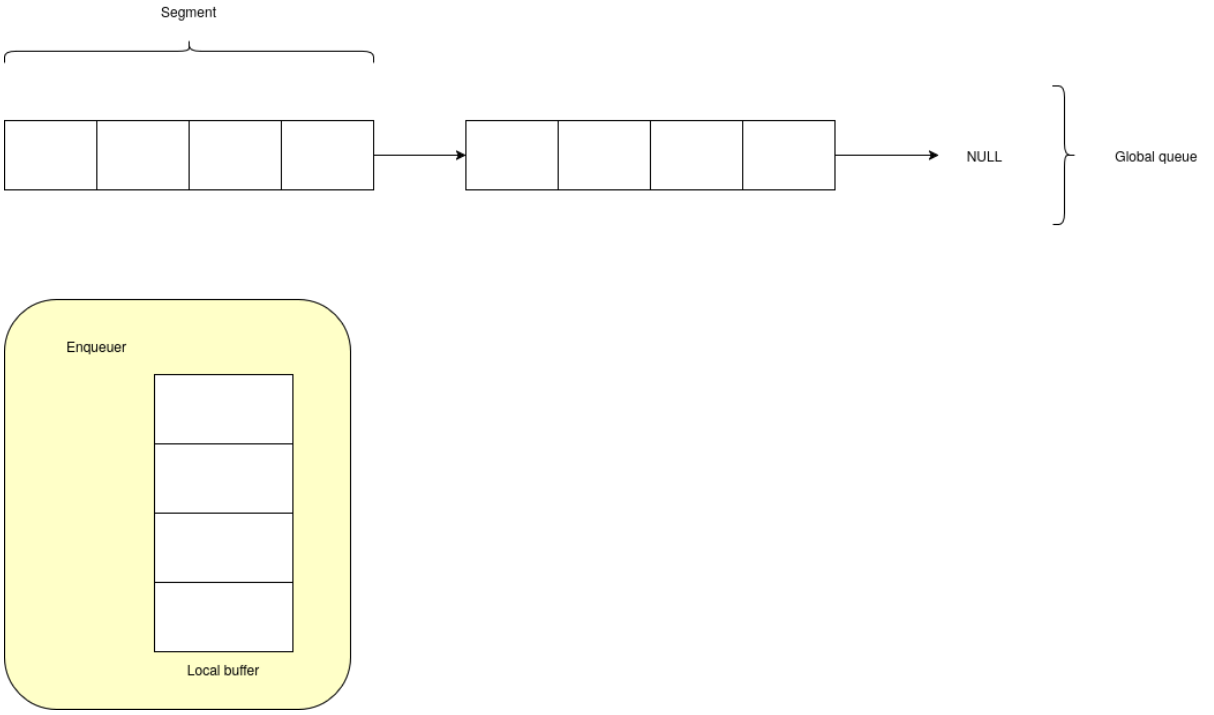


Figure 13: DQueue's structure.

The global queue where data is stored is represented as a linked list of segments. A segment is simply a contiguous array of data items. This design allows for unbounded queue capacity while still allowing a fair degree of random access within a segment. This allows us to use indices to index elements in the queue, thus permitting the use of inexpensive FAA instructions to swing the head and tail indices.

Each enqueuer maintains a local buffer to batch enqueue items into before flushing to the global queue. This helps prevent contention and plays nicely with the cache. To enqueue an item, an enqueuer simply FAA-s the head index to reserve a slot in the global queue; the obtained index is stored along with the data in the local buffer so that when flushing the local buffer, the enqueuer knows where to write the data into the global queue. Note that while flushing, an index may point to a not-yet-existent slot in the global queue. Therefore, new segments must be allocated on the fly and CAS-ed to the end of the queue.

The dequeuer dequeues the items by looking at the head index. If the queue is not empty but the slot at the head index is empty, the dequeuer utilizes a helping mechanism by looking at all enqueuers to help them flush out their local buffer. After this, the head slot is guaranteed to be non-empty, and the dequeuer can finally dequeue this value.

The ABA problem is solved by relying on its safe memory reclamation scheme. In DQueue, CAS is only used to update the tail pointer to point to a newly allocated segment. Therefore, the ABA problem in DQueue only involves internal manipulation of pointers to dynamically allocated memory. This means that if a proper memory reclamation scheme is used, the ABA problem cannot occur.

DQueue relies on a dedicated garbage collection thread to reclaim segments that have been exhausted by the dequeuer. However, this should be a careful process as even though some segments have been exhausted, some enqueuers can still hold an index that references one of these segments. DQueue implements this by reclaiming all exhausted segments if there is no enqueuer holding an index referencing these segments. Unfortunately, we believe DQueue's scheme is unsafe. Specifically, as described, DQueue allows the garbage collection thread to reclaim non-adjacent segments in the global queue without patching any of the next pointers. Any segment just before a reclaimed segment would point to a deallocated next segment. By definition, this segment was not reclaimed because it is referenced by an enqueuer. This means this enqueuer cannot traverse the next pointer chain to get to the end of the queue without accessing an already-deallocated segment.

If adapted to a distributed environment, the flushing may be expensive, both from the point of view of the enqueuer and the dequeuer. If the dequeuer has to help every enqueuer to flush their local buffer, which should always result in at least one remote operation, the cost would be prohibitively high. Similarly, each flush requires the enqueuer to issue at least one remote operation, but this is at least acceptable as flushing is infrequent.

Still, we can see that the pattern of maintaining a local buffer inside each enqueuer repeats throughout the literature, which we can definitely apply when designing distributed MPSC queues.

3.1.3 WRLQueue

WRLQueue [10] is a lock-free MPSC queue specifically designed for embedded real-time systems. Its main purpose is to avoid excessive modification of storage space.

WRLQueue is simply a pair of buffers: one is worked on by multiple enqueuers, and the other is worked on by the dequeuer. The structure of WRLQueue is shown in Figure 14.

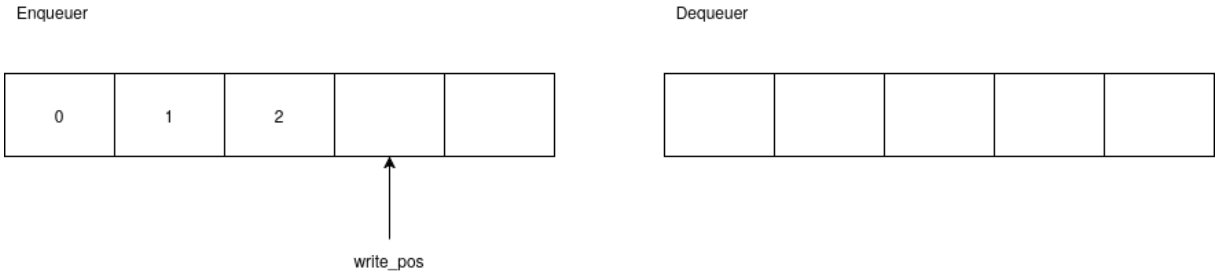


Figure 14: WRLQueue's structure.

The enqueuers batch their enqueues and write multiple elements onto the buffer at once. They use the usual scheme of FAA-ing the tail index (`write_pos` in Figure 14) to reserve their slots and write data items at their leisure.

The dequeuer, upon invocation, will swap its buffer with the enqueuers' buffer to dequeue from it, as in Figure 15. However, WRLQueue explicitly states that the dequeuer has to wait for all enqueue operations to complete in the other buffer before swapping. If an enqueue suspends or dies, the dequeuer will experience a slowdown; this clearly violates the property of non-blocking. Therefore, we believe that WRLQueue is blocking, concerning its dequeue operation.

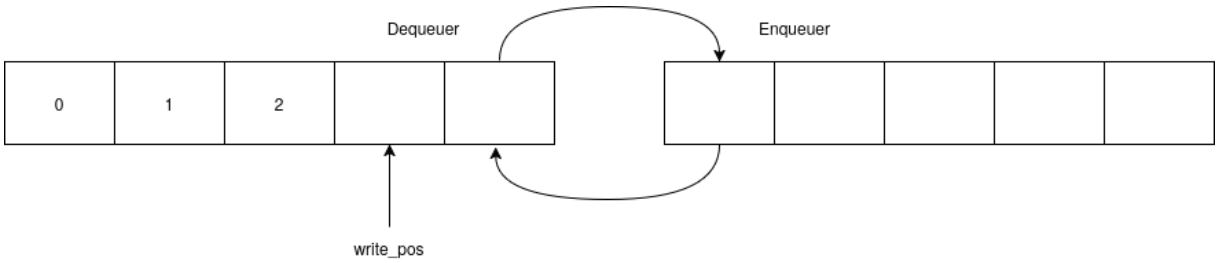


Figure 15: WRLQueue's dequeue operation

3.1.4 Jiffy

Jiffy [13] is a fast and memory-efficient wait-free MPSC queue by avoiding excessive allocation of memory.

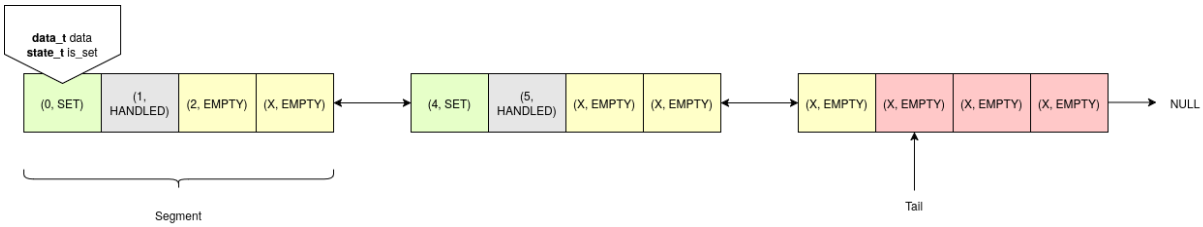


Figure 16: Jiffy's structure.

Like DQueue, Jiffy represents the queue as a doubly-linked list of segments as in Figure 16. This design again allows Jiffy to be unbounded while using head and tail indices to index elements. Each segment contains a pointer to a dynamically allocated array of slots, instead of directly storing the array. Each slot in the segment contains the data item and a state of that slot (`state_t` in the figure). There are 3 states: `SET`, `EMPTY`, and `HANDLED`. Initially, all slots are `EMPTY`. Instead of keeping a global head index, there are per-segment `Head` indices pointing to the first non-`HANDLED` slot. However, there is still one global `Tail` index shared by all the processes.

To enqueue, each enqueuer would FAA the `Tail` to reserve a slot. If the slot isn't in the linked list yet, it tries to allocate new segments and CAS them at the end of the linked list until the slot is available. It then traverses to the desired segment by following the previous pointers starting from the last segment. It then writes the data and sets the slot's state to `SET`. Notice that `EMPTY` slots actually have two substates. If an `EMPTY` slot is before the `Tail` index, that slot is actually reserved by an enqueuer but has not been set yet, while the `EMPTY` slots after the `Tail` index are truly empty.

To dequeue, the dequeuer would start from the Head index of the first segment, scanning until it finds the first non-`HANDLED` slot before the end of the queue. If there is no such slot, the queue is empty, and the dequeuer would return nothing. If this slot is `SET`, it simply reads the data item in this slot and sets it to `HANDLED`. If this slot is `EMPTY`, that means this slot has been reserved by an enqueuer that has not finished. In this case, the dequeuer performs a scan forward to find the first `SET` slot. If not found, the dequeuer returns nothing. Otherwise, it continues to repeatedly scan all slots between the first non-`HANDLED` and the last found `SET` slot until the first `SET` slot in this interval is unchanged between 2 scans. Only then, the dequeuer would return the data item in this `SET` slot and mark it as `HANDLED`.

Similar to DQueue, CAS is only used when appending new segments at the end of the queue. Therefore, the ABA problem only involves internal manipulation of pointers to dynamically allocated memory. Consequently, if a proper memory reclamation scheme is utilized, the ABA problem is also properly solved.

Regarding memory reclamation, Jiffy does not specify a sufficient scheme: If one enqueuer is delayed forever, no memory is ever reclaimed. As a consequence, if an enqueuer is delayed for too long, the system will run out of memory, causing other enqueuers to fail without making any progress. Effectively, Jiffy is not wait-free.

3.1.5 Remarks

Out of the 4 investigated MPSC queue algorithms, we quickly eliminate DQueue, WRLQueue, and Jiffy as potential candidates for porting to a distributed environment because they either do not provide a sufficient progress guarantee or protection against the ABA problem and memory reclamation problem. Therefore, we will only adapt LTQueue for distributed environments in the next section. LTQueue also presents some challenges, though, as it utilizes LL/SC for the ABA solution, which does not exist in distributed environments. Consequently, to adapt LTQueue, we have to work around LTQueue's usage of LL/SC.

3.2 Distributed MPSC queues

This section summarizes, to the best of our knowledge, existing MPSC queue algorithms, which is reflected in Section 3.2.

The only paper we have found so far that either mentions directly or indirectly the design of an MPSC queue is [1]. [1] introduces a hosted blocking (the original paper claims that it is lock-free) bounded distributed MPSC queue called active-message queue (AMQueue) that bears resemblance to WRLQueue in [10].

FIFO queues	Active-message queue (AMQueue) [1]
Progress guarantee of dequeue	Blocking (*)
Progress guarantee of enqueue	Wait-free
ABA solution	No compare-and-swap usage
Safe memory reclamation	Custom scheme

Table 3: Characteristic summary of existing distributed MPSC queues.
R stands for remote operations and L stands for local operations.
(*) [1] claims that it is lock-free.

The structure of AMQueue is given in Figure 17. The MPSC is split into 2 queues, each maintaining its own set of control variables:

- `WriterCnt`: The number of enqueuers currently writing in this queue.
- `Offset`: The index to the first empty entry in the queue. Note that any shared data and control variables are hosted on the dequeuer.

To determine which queue to read and write, the `QueueNum` binary variable is used. If `QueueNum` is `0`, then the first queue is being actively written by enqueuers, and the second queue is being reserved for the dequeuer, and vice versa.

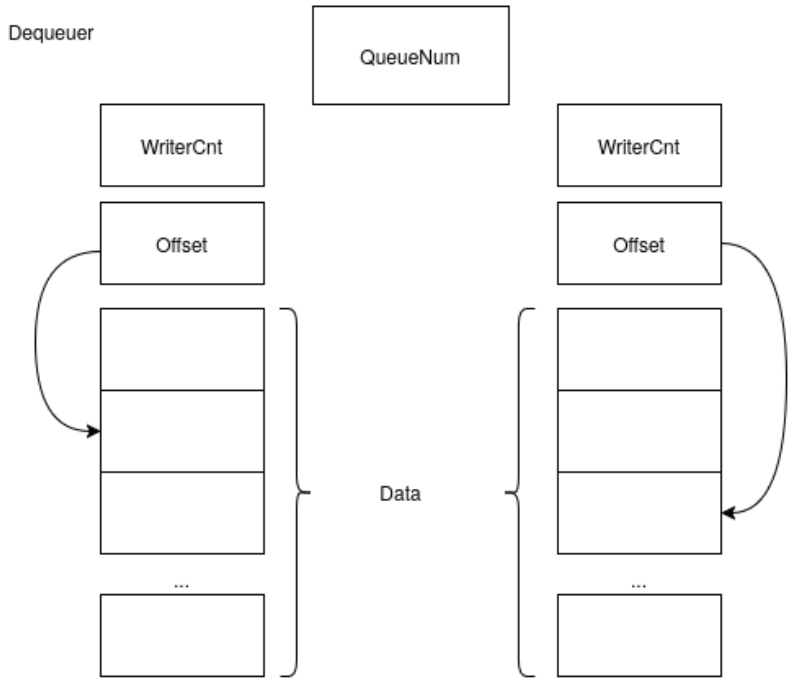


Figure 17: AMQueue's structure.

To enqueue, the enqueuer first reads the `QueueNum` variable to see which queue is active. The enqueuer then registers for that queue by atomically FAA-ing the corresponding `WriterCnt` variable. If the fetched value is negative, though, the `QueueNum` queue is being swapped for dequeuing, and the enqueuer has to decrement the `WriterCnt` variable and repeat the process until `WriterCnt` is positive. After a successful registration, the enqueuer then reserves an entry in the data array by FAA-ing the `Offset` variable. After that, the enqueuer can enqueue data at its leisure. Upon success, the enqueuer has to decrement `WriterCnt` before returning.

To dequeue, the dequeuer inverts `QueueNum` to direct future enqueuers to the other queue. The dequeuer then subtracts a sufficiently large number from `WriterCnt` to signal to other enqueuers that it has started processing. The dequeuer has to wait for all current enqueuers in the queue to finish by repeatedly checking the `WriterCnt` variable, hence the blocking property. After all enqueuers have finished, the dequeuer then batch-dequeues all data in the queue, resetting the `Offset` and `WriterCnt` variables to 0.

Based on our discussion, there is currently no non-blocking distrbuted MPSC queue in the literature. This makes our research the first one of its kind to be about non-

blocking distributed MPSC queues. AMQueue will serve as a benchmarking baseline for our MPSC queues in Chapter V.

Chapter IV Distributed MPSC queues

Based on the MPSC queue algorithms we have surveyed in Chapter III, we propose three wait-free distributed MPSC queue algorithms:

- dLTQueue (Section 4.2) is a direct modification of the original LTQueue [12] without any usage of LL/SC, adapted for distributed environment.
- Slotqueue (Section 4.3) is inspired by the timestamp-refreshing idea of dLTQueue [12] and repeated-rescan of Jiffy [13]. Although it still bears some resemblance to LTQueue, we believe that it is more optimized for distributed context.
- dLTQueueV2 (Section 4.4) introduces some lightweight optimization upon dLTQueue. Therefore, dLTQueueV2 does not deviate much from dLTQueue. However, theory-wise, dLTQueueV2 has the best performance characteristics.

The characteristics of these algorithms are discussed in Table 4.

MPSC queues	**dLTQueue**	**Slotqueue**	**dLTQueueV2**
Correctness	Linearizable	Linearizable	Linearizable
Progress guarantee of dequeue	Wait-free	Wait-free	Wait-free
Progress guarantee of enqueue	Wait-free	Wait-free	Wait-free
Dequeue time-complexity	$4\log_2(n)R + 6\log_2(n)L$	$3R + 2nL$	$3R + 10\log(n)L$
Enqueue time-complexity	$6\log_2(n)R + 4\log_2(n)L$	$4R + 3L$	$4R + 4L$
ABA solution	Unique timestamp	No hazardous ABA problem	Unique timestamp
Safe memory reclamation	No dynamic memory allocation	No dynamic memory allocation	No dynamic memory allocation

Table 4: Characteristic summary of our proposed distributed MPSC queues.
(1) n is the number of processes.
(2) R stands for **remote operation** and L stands for **local operation**.

The rest of this chapter is organized as follows. Section 4.1 describes a simple baseline distributed SPSC that is utilized as the underlying SPSC in our MPSC queues.

Section 4.2, Section 4.3 and Section 4.4 introduces three wait-free MPSC queues: dLTQueue, Slotqueue and dLTQueueV2, respectively.

In these next few descriptions, we assume that each process in our program is assigned a unique number as an identifier, which is termed as its **rank**. The numbers are taken from the range of `[0, size - 1]`, with `size` being the number of processes in our program.

4.1 A simple baseline distributed SPSC

The three MPSC queue algorithms proposed in Section 4.2, Section 4.3 and Section 4.4 all utilize a baseline distributed SPSC data structure, which we will present in this section.

For implementation simplicity, we present a bounded SPSC, effectively make our proposed algorithms support only a bounded number of elements. However, one can trivially substitute another distributed unbounded SPSC to make our proposed algorithms support an unbounded number of elements, as long as this SPSC supports the same interface as ours.

The SPSC queue uses a circular array `Data` with a fixed `Capacity`. It maintains two indices: `First` marks the oldest item not yet removed, and `Last` marks the next available slot for insertion. Both indices use modulo arithmetic (`First % Capacity` and `Last % Capacity`) to wrap around the array.

For performance optimization, each process maintains local cached copies of these indices in `First_buf` and `Last_buf`. All indices start at zero. The memory layout is distributed between processes: the dequeuer hosts the `First` and `Last` indices, while the enqueuer hosts the `Data` array itself.

Placement-wise, all queue data in this SPSC is hosted on the enqueuer while the control variables i.e. `First` and `Last`, are hosted on the dequeuer.

Shared variables

> `Data`: `gptr<data_t>`
>
> `First`: `gptr<uint64_t>`
>
> `Last`: `gptr<uint64_t>`

Enqueuer-local variables

> `First_buf`: `uint64_t`
>
> `Last_buf`: `uint64_t`
>
> `Capacity`: `uint64_t`

Dequeuer-local variables

> `First_buf`: `uint64_t`
>
> `Last_buf`: `uint64_t`
>
> `Capacity`: `uint64_t`

The procedures of the enqueuer are given as follows.

Procedure 4: `bool spsc_enqueue(data_t v)`

```
1 new_last = Last_buf + 1
2 if (new_last - First_buf > Capacity)
3 |  read(First, &First_buf)
4 |  if (new_last - First_buf > Capacity)
5 |  |  return false
6 write(Data + Last_buf % Capacity, &v)
7 write(Last, &new_last)
8 Last_buf = new_last
9 return true
```

`spsc_enqueue` first computes the new `Last` value (Line 1). If the queue is full as indicated by the difference the new `Last` value and `First_buf` (Line 2), there can still be the possibility that some elements have been dequeued but `First_buf` has not been synced with `First` yet, therefore, we first refresh the value of `First_buf` by fetching from `First` (Line 3). If the queue is still full (Line 4), we signal failure (Line 5). Otherwise, we proceed to write the enqueued value to the entry at `Last_buf % Capacity` (Line 6), increment `Last` (Line 7), update the value of `Last_buf` (Line 8) and signal success (Line 9).

Procedure 5: `bool spsc_readFront`$_e$`(data_t* output)`

```
10 if (First_buf >= Last_buf)
11 |  return false
12 read(First, &First_buf)
13 if (First_buf >= Last_buf)
14 |  return false
15 read(Data + First_buf % Capacity, output)
16 return true
```

`spsc_readFront`$_e$ first checks if the SPSC is empty based on the difference between `First_buf` and `Last_buf` (Line 10). Note that if this check fails, we signal failure immediately (Line 11) without refetching either `First` or `Last`. This suffices because

`Last` cannot be out-of-sync with `Last_buf` as we are the enqueuer and `First` can only increase since the last refresh of `First_buf`, therefore, if we refresh `First` and `Last`, the condition on Line 10 would return `false` anyways. If the SPSC is not empty, we refresh `First` and re-perform the empty check (Line 13 - Line 14). If the SPSC is again not empty, we read the queue entry at `First_buf % Capacity` into `output` (Line 15) and signal success (Line 16).

The procedures of the dequeuer are given as follows.

Procedure 6: `bool spsc_dequeue(data_t* output)`

```
17 new_first = First_buf + 1
18 if (new_first > Last_buf)
19 |  read(Last, &Last_buf)
20 |  if (new_first > Last_buf)
21 |  |  return false
22 read(Data + First_buf % Capacity, output)
23 write(First, &new_first)
24 First_buf = new_first
25 return true
```

`spsc_dequeue` first computes the new `First` value (Line 17). If the queue is empty as indicated by the difference the new `First` value and `Last_buf` (Line 18), there can still be the possibility that some elements have been enqueued but `Last_buf` has not been synced with `Last` yet, therefore, we first refresh the value of `Last_buf` by fetching from `Last` (Line 19). If the queue is still empty (Line 20), we signal failure (Line 21). Otherwise, we proceed to read the top value at `First_buf % Capacity` (Line 22) into `output`, increment `First` (Line 23) - effectively dequeue the element, update the value of `First_buf` (Line 24) and signal success (Line 25).

Procedure 7: `bool spsc_readFront_d(data_t* output)`

```
26 if (First_buf >= Last_buf)
27 |  read(Last, &Last_buf)
28 |  if (First_buf >= Last_buf)
29 |  |  return false
30 read(Data + First_buf % Capacity, output)
31 return true
```

`spsc_readFront_d` first checks if the SPSC is empty based on the difference between `First_buf` and `Last_buf` (Line 26). If this check fails, we refresh `Last_buf` (Line 27) and recheck (Line 28). If the recheck fails, signal failure (Line 29). If the SPSC is not empty, we read the queue entry at `First_buf % Capacity` into `output` (Line 30) and signal success (Line 31).

4.2 dLTQueue - Straightforward LTQueue adapted for distributed environment

This algorithm presents our most straightforward effort to port LTQueue [12] to distributed context. The main challenge is that LTQueue uses LL/SC as the universal atomic instruction and also an ABA solution, but LL/SC is not available in distributed programming environments. We have to replace any usage of LL/SC in the original LTQueue algorithm. We use compare-and-swap and the well-known monotonic timestamp scheme to guard against ABA problem.

4.2.1 Overview

The structure of our dLTQueue is shown as in Figure 18.

We differentiate between 2 types of nodes: *enqueuer nodes* (represented as the rectangular boxes at the bottom of Figure 18) and normal *tree nodes* (represented as the circular boxes in Figure 18).

Each enqueuer node corresponds to an enqueuer. Each time the local SPSC is enqueued with a value, the enqueuer timestamps the value using a distributed counter shared by all enqueuers. An enqueuer node stores the SPSC local to the corresponding enqueuer and a `min_timestamp` value which is the minimum timestamp inside the local SPSC.

Each tree node stores the rank of an enqueuer process. This rank corresponds to the enqueuer node with the minimum timestamp among the node's children's ranks. The

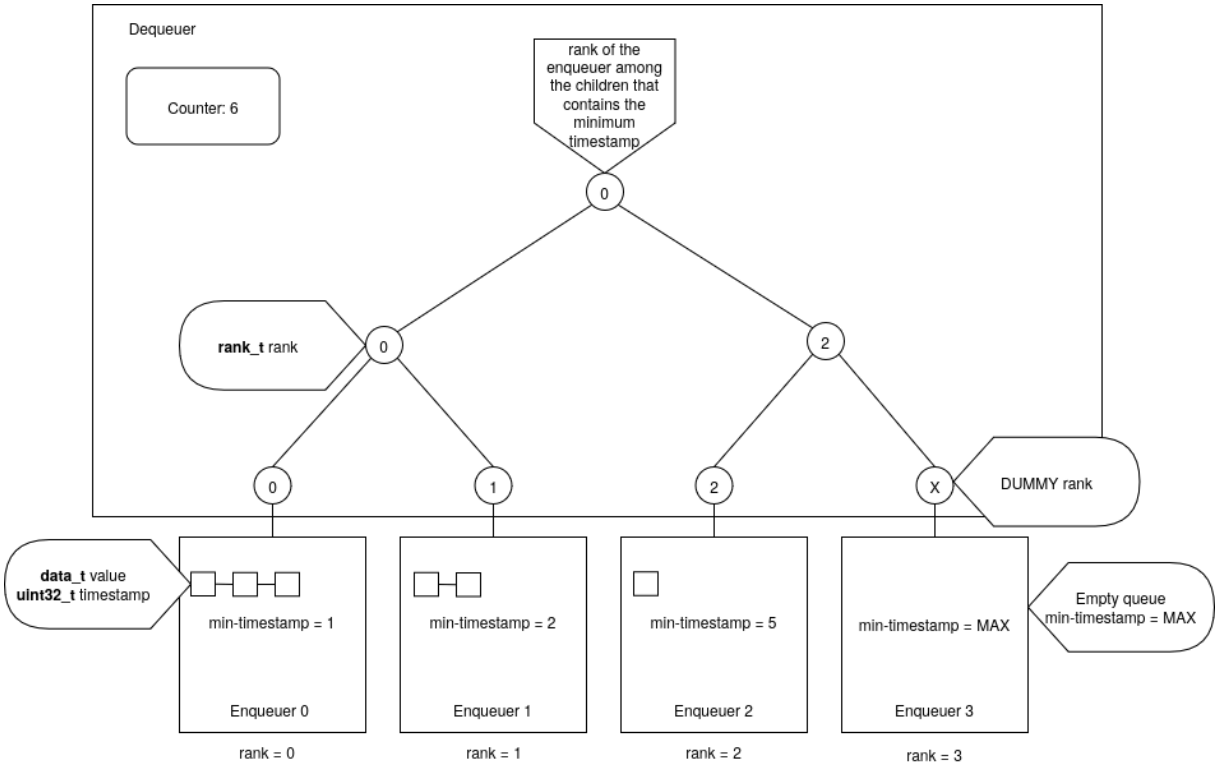


Figure 18: dLTQueue's structure.

tree node that is attached to an enqueuer node is called a *leaf node*, otherwise, it is called an *internal node*.

Note that if a local SPSC is empty, the `min_timestamp` variable of the corresponding enqueuer node is set to `MAX_TIMESTAMP` and the corresponding leaf node's rank is set to `DUMMY_RANK`.

Placement-wise:

- The enqueuer nodes are hosted at the corresponding enqueuer.
- All the tree nodes are hosted at the dequeuer.
- The distributed counter, which the enqueuers use to timestamp their enqueued value, is hosted at the dequeuer.

4.2.2 Data structure

Below is the types utilized in dLTQueue.

Types

> `data_t` = The type of the data to be stored.
>
> `spsc_t` = The type of the SPSC, this is assumed to be the distributed SPSC in Section 4.1.

`rank_t` = The type of the rank of an enqueuer process tagged with a unique timestamp (version) to avoid ABA problem.

> **struct**
>
> > `value`: `uint32_t`
> >
> > `version`: `uint32_t`
>
> **end**

`timestamp_t` = The type of the timestamp tagged with a unique timestamp (version) to avoid ABA problem.

> **struct**
>
> > `value`: `uint32_t`
> >
> > `version`: `uint32_t`
>
> **end**

`node_t` = The type of a tree node.

> **struct**
>
> > `rank`: `rank_t`
>
> **end**

The shared variables in our LTQueue version are as follows.

Note that we have described a very specific and simple way to organize the tree nodes in dLTQueue in a min-heap-like array structure hosted on the sole dequeuer. We will resume our description of the related tree-structure procedures `parent()` (Procedure 8), `children()` (Procedure 9), `leafNodeIndex()` (Procedure 10) with this representation in mind. However, our algorithm does not strictly require this representation and can be substituted with other more-optimized representations & distributed placements, as long as the similar tree-structure procedures are supported.

Shared variables

`Counter`: `gptr<uint64_t>`

> A distributed counter shared by the enqueuers. Hosted at the dequeuer.

`Tree_size`: `uint64_t`

> A read-only variable storing the number of tree nodes present in the dLTQueue.

`Nodes`: `gptr<node_t>`

> An array with `Tree_size` entries storing all the tree nodes present in the dLTQueue shared by all processes.
>
> Hosted at the dequeuer.

> This array is organized in a similar manner as a min-heap: At index `0` is the root node. For every index $i > 0$, $\lfloor \frac{i-1}{2} \rfloor$ is the index of the parent of node i. For every index $i > 0$, $2i + 1$ and $2i + 2$ are the indices of the children of node i.

`Dequeuer_rank`: `uint32_t`

> The rank of the dequeuer process. This is read-only.

`Timestamps`: A read-only **array** `[0..size - 1]` of `gptr<timestamp_t>`, with `size` being the number of processes.

> The entry at index i corresponds to the `Min_timestamp` distributed variable at the process with a rank of i.

Enqueuer-local variables

`Process_count`: `uint64_t`

> The number of processes.

`Self_rank`: `uint32_t`

> The rank of the current enqueuer process.

`Min_timestamp`: `gptr<timestamp_t>`

`Spsc`: `spsc_t`

> This SPSC is synchronized with the dequeuer.

Dequeuer-local variables

`Process_count`: `uint64_t`

> The number of processes.

`Spscs`: An **array** of `spsc_t` with `Process_count` entries.

> The entry at index i corresponds to the `Spsc` at the process with a rank of i.

Initially, the enqueuers and the dequeuer are initialized as follows:

Enqueuer initialization

Initialize `Process_count`, `Self_rank` and `Dequeuer_rank`.

Initialize `Spsc` to the initial state.

Initialize `Min_timestamp` to `timestamp_t {MAX_TIMESTAMP, 0}`.

Dequeuer initialization

Initialize `Process_count`, `Self_rank` and `Dequeuer_rank`.

Initialize `Counter` to `0`.

Initialize `Tree_size` to `Process_count * 2`.

Initialize `Nodes` to an array with `Tree_size` entries. Each entry is initialized to `node_t {DUMMY_RANK}`.

Initialize `Spscs`, synchronizing each entry with the corresponding enqueuer.

> Initialize `Timestamps`, synchronizing each entry with the corresponding enqueuer.

4.2.3 Algorithm

We first present the tree-structure utility procedures that are shared by both the enqueuer and the dequeuer:

Procedure 8: `uint32_t parent(uint32_t index)`

```
1 return (index - 1) / 2
```

`parent` returns the index of the parent tree node given the node with index `index`. These indices are based on the shared `Nodes` array. Based on how we organize the `Nodes` array, the index of the parent tree node of `index` is `(index - 1) / 2`.

Procedure 9: `vector<uint32_t> children(uint32_t index)`

```
2  left_child = index * 2 + 1
3  right_child = left_child + 1
4  res = vector<uint32_t>()
5  if (left_child >= Tree_size)
6  |  return res
7  res.push(left_child)
8  if (right_child >= Tree_size)
9  |  return res
10 res.push(right_child)
11 return res
```

Similarly, `children` returns all indices of the child tree nodes given the node with index `index`. These indices are based on the shared `Nodes` array. Based on how we organize the `Nodes` array, these indices can be either `index * 2 + 1` or `index * 2 + 2`.

Procedure 10: `uint32_t leafNodeIndex(uint32_t enqueuer_rank)`

```
12 return Tree_size + enqueuer_rank
```

`leafNodeIndex` returns the index of the leaf node that is logically attached to the enqueuer node with rank `enqueuer_rank` as in Figure 18.

The followings are the enqueuer procedures.

Procedure 11: `bool enqueue(data_t value)`

```
13 timestamp = faa(Counter, 1)
14 if (!spsc_enqueue(&Spsc, (value, timestamp)))
15  | return false
16 propagate_e()
17 return true
```

To enqueue a value, `enqueue` first obtains a count by FAA-ing the distributed counter `Counter` (Line 13). Then, we enqueue the data tagged with the timestamp into the local SPSC (Line 14). Then, `enqueue` propagates the changes by invoking `propagate_e()` (Line 16) and returns `true`.

Procedure 12: `void propagate_e()`

```
18 if (!refreshTimestamp_e())
19  | refreshTimestamp_e()
20 if (!refreshLeaf_e())
21  | refreshLeaf_e()
22 current_node_index = leafNodeIndex(Self_rank)
23 repeat
24  | current_node_index = parent(current_node_index)
25  | if (!refresh_e(current_node_index))
26  |  | refresh_e(current_node_index)
27 until current_node_index == 0
```

The `propagate_e` procedure is responsible for propagating SPSC updates up to the root node as a way to notify other processes of the newly enqueued item. It is split into 3 phases: Refreshing of `Min_timestamp` in the enqueuer node (Line 18 - Line 19), refreshing of the enqueuer's leaf node (Line 20 - Line 21), refreshing of internal nodes (Line 23 - Line 27). On Line 20 - Line 27, we refresh every tree node that lies between the enqueuer node and the root node.

Procedure 13: `bool refreshTimestamp`$_e$`()`

```
28 min_timestamp = timestamp_t {}
29 read(Min_timestamp, &min_timestamp)
30 {old_timestamp, old_version} = min_timestamp
31 front = (data_t {}, timestamp_t {})
32 is_empty = !spsc_readFront(&Spsc, &front)
33 if (is_empty)
       return cas(Min_timestamp,
34     timestamp_t {old_timestamp, old_version},
       timestamp_t {MAX_TIMESTAMP, old_version + 1})
35 else
       return cas(Min_timestamp,
36     timestamp_t {old_timestamp, old_version},
       timestamp_t {front.timestamp, old_version + 1})
```

The `refreshTimestamp`$_e$ procedure is responsible for updating the `Min_timestamp` of the enqueuer node. It simply looks at the front of the local SPSC (Line 32) and CAS `Min_timestamp` accordingly (Line 33 - Line 36).

Procedure 14: `bool refreshNode_e(uint32_t current_node_index)`

```
37 current_node = node_t {}
38 read(Nodes, current_node_index, &current_node)
39 {old_rank, old_version} = current_node.rank
40 min_rank = DUMMY_RANK
41 min_timestamp = MAX_TIMESTAMP
42 for child_node_index in children(current_node)
43     child_node = node_t {}
44     read(Nodes + child_node_index, &child_node)
45     {child_rank, child_version} = child_node
46     if (child_rank == DUMMY_RANK) continue
47     child_timestamp = timestamp_t {}
48     read(Timestamps + child_rank, &child_timestamp)
49     if (child_timestamp < min_timestamp)
50         min_timestamp = child_timestamp
51         min_rank = child_rank
   return cas(Nodes + current_node_index,
52 node_t {rank_t {old_rank, old_version}},
   node_t {rank_t {min_rank, old_version + 1}})
```

The `refreshNode_e` procedure is responsible for updating the ranks of the internal nodes affected by the enqueue. It loops over the children of the current internal nodes (Line 42). For each child node, we read the rank stored in it (Line 45), if the rank is not `DUMMY_RANK`, we proceed to read the value of `Min_timestamp` of the enqueuer node with the corresponding rank (Line 48). At the end of the loop, we obtain the rank stored inside one of the child nodes that has the minimum timestamp stored in its enqueuer node (Line 50 - Line 51). We then try to CAS the rank inside the current internal node to this rank (Line 52).

Procedure 15: `bool refreshLeaf_e()`

```
53 leaf_node_index = leafNodeIndex(Self_rank)
54 leaf_node = node_t {}
55 read(Nodes + leaf_node_index, &leaf_node)
56 {old_rank, old_version} = leaf_node.rank
57 min_timestamp = timestamp_t {}
58 read(Min_timestamp, &min_timestamp)
59 timestamp = min_timestamp.timestamp
   return cas(Nodes + leaf_node_index,
60 node_t {rank_t {old_rank, old_version}},
   node_t {timestamp == MAX ? DUMMY_RANK : Self_rank, old_version + 1})
```

The `refreshLeaf_e` procedure is responsible for updating the rank of the leaf node affected by the enqueue. It simply reads the value of `Min_timestamp` of the enqueuer node it is logically attached to (Line 58) and CAS the leaf node's rank accordingly (Line 60).

The followings are the dequeuer procedures.

Procedure 16: `bool dequeue(data_t* output)`

```
61 root_node = node_t {}
62 read(Nodes, &root_node)
63 {rank, version} = root_node.rank
64 if (rank == DUMMY_RANK) return false
65 output_with_timestamp = (data_t {}, timestamp_t {})
66 if (!spsc_dequeue(&Spscs[rank]),
       &output_with_timestamp))
67   return false
68 *output = output_with_timestamp.data
69 propagate_d(rank)
70 return true
```

To dequeue a value, `dequeue` reads the rank stored inside the root node (Line 63). If the rank is `DUMMY_RANK`, the MPSC queue is treated as empty and failure is signaled (Line 64). Otherwise, we invoke `spsc_dequeue` on the SPSC of the enqueuer with the obtained rank (Line 66). We then extract out the real data and set it to `output` (Line 68).

We finally propagate the dequeue from the enqueuer node that corresponds to the obtained rank (Line 69) and signal success (Line 70).

Procedure 17: `void propagate_d(uint32_t enqueuer_rank)`

```
71 if (!refreshTimestamp_d(enqueuer_rank))
72  | refreshTimestamp_d(enqueuer_rank)
73 if (!refreshLeaf_d(enqueuer_rank))
74  | refreshLeaf_d(enqueuer_rank)
75 current_node_index = leafNodeIndex(enqueuer_rank)
76 repeat
77  | current_node_index = parent(current_node_index)
78  | if (!refresh_d(current_node_index))
79  |  | refresh_d(current_node_index)
80 until current_node_index == 0
```

The `propagate_d` procedure is similar to `propagate_e`, with appropriate changes to accommodate the dequeuer.

Procedure 18: `bool refreshTimestamp_d(uint32_t enqueuer_rank)`

```
81 min_timestamp = timestamp_t {}
82 read(Timestamps + enqueuer_rank, &min_timestamp)
83 {old_timestamp, old_version} = min_timestamp
84 front = (data_t {}, timestamp_t {})
85 is_empty = !spsc_readFront(&Spscs[enqueuer_rank], &front)
86 if (is_empty)
    | return cas(Timestamps + enqueuer_rank,
87  | timestamp_t {old_timestamp, old_version},
    | timestamp_t {MAX_TIMESTAMP, old_version + 1})
88 else
    | return cas(Timestamps + enqueuer_rank,
89  | timestamp_t {old_timestamp, old_version},
    | timestamp_t {front.timestamp, old_version + 1})
```

The `refreshTimestamp_d` procedure is similar to `refreshTimestamp_e`, with appropriate changes to accommodate the dequeuer.

Procedure 19: `bool refreshNode_d(uint32_t current_node_index)`

```
91  current_node = node_t {}
92  read(Nodes + current_node_index, &current_node)
93  {old_rank, old_version} = current_node.rank
94  min_rank = DUMMY_RANK
95  min_timestamp = MAX_TIMESTAMP
96  for child_node_index in children(current_node)
97      child_node = node_t {}
98      read(Nodes + child_node_index, &child_node)
99      {child_rank, child_version} = child_node
100     if (child_rank == DUMMY_RANK) continue
101     child_timestamp = timestamp_t {}
102     read(Timestamps + child_rank, &child_timestamp)
103     if (child_timestamp < min_timestamp)
104         min_timestamp = child_timestamp
105         min_rank = child_rank
    return cas(Nodes + current_node_index,
106 node_t {rank_t {old_rank, old_version}},
    node_t {rank_t {min_rank, old_version + 1}})
```

The `refreshNode_d` procedure is similar to `refreshNode_e`, with appropriate changes to accommodate the dequeuer.

Procedure 20: `bool refreshLeaf_d(uint32_t enqueuer_rank)`

```
107 leaf_node_index = leafNodeIndex(enqueuer_rank)
108 leaf_node = node_t {}
109 read(Nodes + leaf_node_index, &leaf_node)
110 {old_rank, old_version} = leaf_node.rank
111 min_timestamp = timestamp_t {}
112 read(Timestamps + enqueuer_rank, &min_timestamp)
113 timestamp = min_timestamp.timestamp
    return cas(Nodes + leaf_node_index,
114 node_t {rank_t {old_rank, old_version}},
    node_t {timestamp == MAX ? DUMMY_RANK : Self_rank, old_version + 1})
```

The `refreshLeaf`$_d$ procedure is similar to `refreshLeaf`$_e$, with appropriate changes to accommodate the dequeuer.

4.3 Slotqueue - dLTQueue-inspired distributed MPSC queue with all constant-time operations

The straightforward dLTQueue algorithm we have ported in Section 4.2 pretty much preserves the original algorithm's characteristics, i.e. wait-freedom and time complexity of $\Theta(\log n)$ for `dequeue` and `enqueue` operations. We note that in shared-memory systems, this logarithmic growth is fine. However, in distributed systems, this increase in remote operations would present a bottleneck in enqueue and dequeue latency. Upon closer inspection, this logarithmic growth is due to the propagation process because it has to traverse every level in the tree. Intuitively, this is the problem of us trying to maintain the tree structure. Therefore, to be more suitable for distributed context, we propose a new algorithm Slotqueue inspired by LTQueue, which uses a slightly different structure. The key point is that both `enqueue` and `dequeue` only perform a constant number of remote operations, at the cost of `dequeue` having to perform $\Theta(n)$ local operations, where n is the number of enqueuers. Because remote operations are much more expensive, this might be a worthy tradeoff.

4.3.1 Overview

The structure of Slotqueue is shown as in Figure 19.

Each enqueuer hosts a distributed SPSC as in dLTQueue (Section 4.2). The enqueuer when enqueues a value to its local SPSC will timestamp the value using a distributed counter hosted at the dequeuer.

Additionally, the dequeuer hosts an array whose entries each corresponds with an enqueuer. Each entry stores the minimum timestamp of the local SPSC of the corresponding enqueuer.

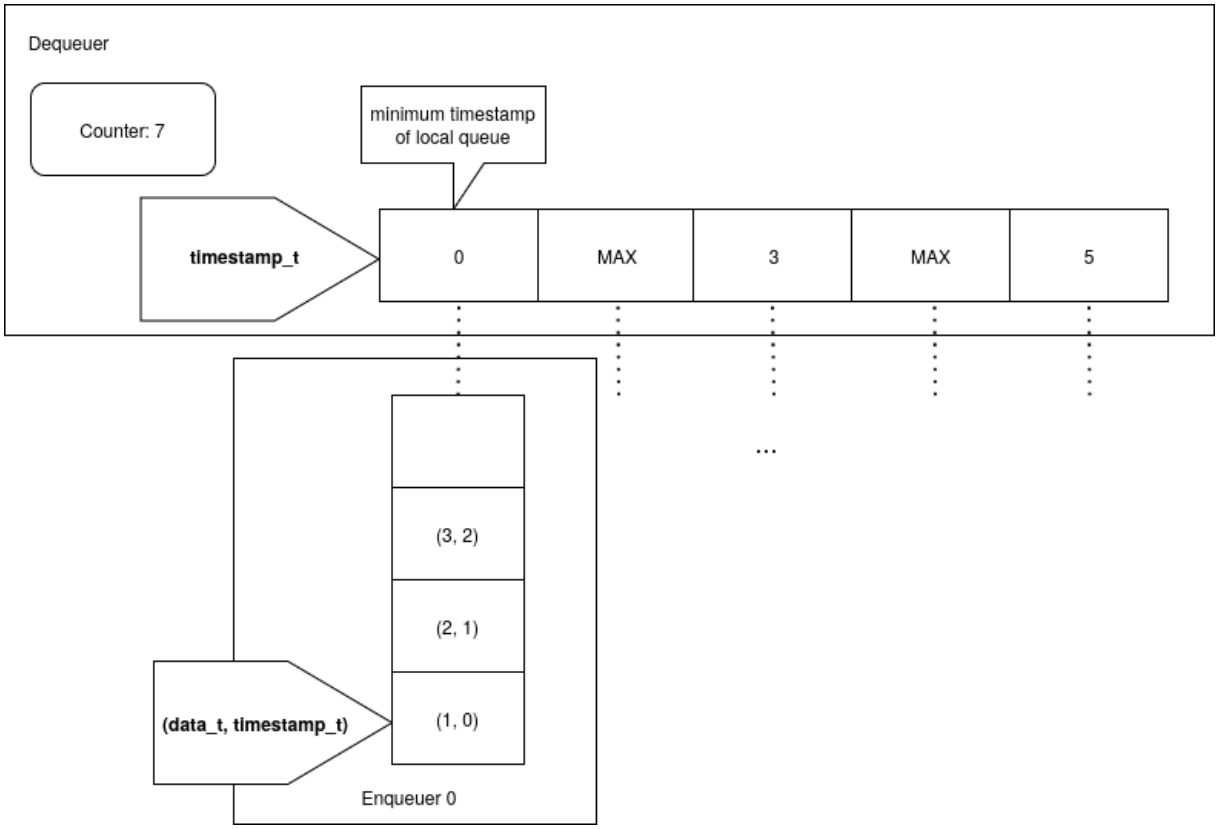


Figure 19: Basic structure of Slotqueue.

4.3.2 Data structure

We first introduce the types and shared variables utilized in Slotqueue.

Types

> `data_t` = The type of data stored.
>
> `timestamp_t` = `uint64_t`
>
> `spsc_t` = The type of the SPSC each enqueuer uses, this is assumed to be the distributed SPSC in Section 4.1.

Shared variables

> `Slots`: `gptr<timestamp_t*>`
>
> > An array of `timestamp_t` with the number of entries equal to the number of enqueuers.
> >
> > Hosted at the dequeuer.
>
> `Counter`: `gptr<uint64_t>`
>
> > A distributed counter.

Hosted at the dequeuer.

Enqueuer-local variables

`Dequeuer_rank`: `uint64_t`

The rank of the dequeuer.

`Process_count`: `uint64_t`

The number of enqueuers.

`Self_rank`: `uint32_t`

The rank of the current enqueuer process.

`Spsc`: `spsc_t`

This SPSC is synchronized with the dequeuer.

Dequeuer-local variables

`Dequeuer_rank`: `uint64_t`

The rank of the dequeuer.

`Process_count`: `uint64_t`

The number of enqueuers.

`Spscs`: An **array** of `spsc_t` with `Process_count` entries.

The entry at index i corresponds to the `Spsc` at the process with a rank of i.

Initially, the enqueuer and the dequeuer are initialized as follows.

Enqueuer initialization

Initialize `Dequeuer_rank`.

Initialize `Process_count`.

Initialize `Self_rank`.

Initialize the local `Spsc` to its initial state.

Dequeuer initialization

Initialize `Dequeuer_rank`.

Initialize `Process_count`.

Initialize `Counter` to 0.

Initialize the `Slots` array with size equal to the number of enqueuers and every entry is initialized to `MAX_TIMESTAMP`.

Initialize the `Spscs` array, the `i`-th entry corresponds to the `Spsc` variable of the process of rank `i`.

4.3.3 Algorithm

The enqueuer operations are given as follows.

Procedure 21: `bool enqueue(data_t v)`

```
1 timestamp = faa(Counter, 1)
2 if (!spsc_enqueue(&Spsc, (v, timestamp))) return false
3 if (!refreshEnqueue(timestamp))
4  | refreshEnqueue(timestamp)
5 return true
```

To enqueue a value, `enqueue` first obtains a timestamp by FAA-ing the distributed counter (Line 1). It then tries to enqueue the value tagged with the timestamp (Line 2). At Line 3 - Line 4, the enqueuer tries to refresh its slot's timestamp.

Procedure 22: `bool refreshEnqueue(timestamp_t ts)`

```
6  front = (data_t {}, timestamp_t {})
7  success = spsc_readFront(&Spsc, &front)
8  new_timestamp = success ? front.timestamp : MAX_TIMESTAMP
9  if (new_timestamp != ts)
10  | return true
11 old_timestamp = timestamp_t {}
12 read(Slots + Self_rank, &old_timestamp)
13 success = spsc_readFront(&Spsc, &front)
14 new_timestamp = success ? front.timestamp : MAX_TIMESTAMP
15 if (new_timestamp != ts)
16  | return true
   return cas(Slots + Self_rank,
17     old_timestamp,
       new_timestamp)
```

`refreshEnqueue`'s responsibility is to refresh the timestamp stores in the enqueuer's slot to potentially notify the dequeuer of its newly-enqueued element. It first reads the current front element (Line 7). If the SPSC is empty, the new timestamp is set to `MAX_TIMESTAMP`, otherwise, the front element's timestamp (Line 8). If it finds that the front element's timestamp is different from the timestamp `ts` it returns `true` immediately (Line 9 - Line 10). Otherwise, it reads its slot's old timestamp (Line 12) and re-reads the current front element in the SPSC (Line 13) to update the new timestamp. Note that similar to Line 10, `refreshEnqueue` immediately succeeds if the new timestamp

is different from the timestamp `ts` of the element it enqueues (Line 16). Otherwise, it tries to CAS its slot's timestamp with the new timestamp (Line 17).

The dequeuer operations are given as follows.

Procedure 23: `bool dequeue(data_t* output)`

```
19 rank = readMinimumRank()
20 if (rank == DUMMY_RANK)
21  | return false
22 output_with_timestamp = (data_t {}, timestamp_t {})
23 if (!spsc_dequeue(&Spscs[rank], &output_with_timestamp))
24  | return false
25 *output = output_with_timestamp.data
26 if (!refreshDequeue(rank))
27  | refreshDequeue(rank)
28 return true
```

To dequeue a value, `dequeue` first reads the rank of the enqueuer whose slot currently stores the minimum timestamp (Line 19). If the obtained rank is `DUMMY_RANK`, failure is signaled (Line 20 - Line 21). Otherwise, it tries to dequeue the SPSC of the corresponding enqueuer (Line 23). It then tries to refresh the enqueuer's slot's timestamp to potentially notify the enqueuer of the dequeue (Line 26 - Line 27). It then signals success (Line 28).

Procedure 24: `uint64_t readMinimumRank()`

```
29 buffered_slots = timestamp_t[Process_count] {}
30 for index in 0..Process_count
31  | read(Slots + index, &bufferred_slots[index])
32 if every entry in bufferred_slots is MAX_TIMESTAMP
33  | return DUMMY_RANK
34 let rank be the index of the first slot that contains the minimum timestamp among bufferred_slots
35 for index in 0..rank
36  | read(Slots + index, &bufferred_slots[index])
37 min_timestamp = MAX_TIMESTAMP
38 for index in 0..rank
39  | timestamp = buffered_slots[index]
40  | if (min_timestamp < timestamp)
41  |  | min_rank = index
42  |  | min_timestamp = timestamp
43 return min_rank
```

`readMinimumRank`'s main responsibility is to return the rank of the enqueuer from which we can safely dequeue next. It first creates a local buffer to store the value read from `Slots` (Line 29). It then performs 2 scans of `Slots` and read every entry into `buffered_slots` (Line 30 - Line 36). If the first scan finds only `MAX_TIMESTAMP`s, `DUMMY_RANK` is returned (Line 33). From there, based on `bufferred_slots`, it returns the rank of the enqueuer whose bufferred slot stores the minimum timestamp (Line 38 - Line 43).

Procedure 25: `refreshDequeue(rank: int)` **returns** `bool`

```
46 old_timestamp = timestamp_t {}
47 read(Slots + rank, &old_timestamp)
48 front = (data_t {}, timestamp_t {})
49 success = spsc_readFront(&Spscs[rank], &front)
50 new_timestamp = success ? front.timestamp : MAX_TIMESTAMP
   return cas(Slots + rank,
51     old_timestamp,
       new_timestamp)
```

`refreshDequeue`'s responsibility is to refresh the timestamp of the just-dequeued enqueuer to notify the enqueuer of the dequeue. It first reads the old timestamp of the slot (Line 47) and the front element (Line 49). If the SPSC is empty, the new timestamp is set to `MAX_TIMESTAMP`, otherwise, it is the front element's timestamp (Line 50). It finally tries to CAS the slot with the new timestamp (Line 51).

4.4 dLTQueueV2

The structure of dLTQueueV2 is the same as in dLTQueue. There are two key differences from dLTQueue:

- The `min-timestamp` variables are no longer hosted on the enqueuers, but on the sole dequeuer. This helps bring the number of remote operations of the dequeue operation to a constant number.
- The enqueuer only performs the propagation procedure when its enqueued item is the only item in the local SPSC queue. This avoids unnecessary propogations in most cases, which help bring the number of remote operations of the enqueue operation to a constant number most of the time.

The new enqueuer process is presented in Procedure 26. Otherwise, dLTQueueV2 is similar to dLTQueue, as in Section 4.2.

Procedure 26: `bool enqueue(data_t value)`

```
1 timestamp = faa(Counter, 1)
2 if (!spsc_enqueue(&Spsc, (value, timestamp)))
3 |  return false
4 front = (data_t {}, timestamp_t {})
5 is_empty = !spsc_readFront(Spsc, &front)
6 if (!is_empty && front.timestamp.value != timestamp)
7 |  return true
8 propagate_e()
9 return true
```

Procedure 26, compared to Procedure 11, adds a small check on line Line 4 - Line 7. This check reads the front element (Line 5) and compares its timestamp with the enqueued timestamp (Line 6). If they are equal, then the propagation process can be safely skipped (Line 7).

Chapter V Evaluation

This section introduces our benchmarking process, including the benchmarking baselines (Section 5.1), the benchmarking environment (Section 5.2) and the microbenchmark program (Section 5.3). Most importantly, we showcase the results on how well our algorithms perform and conclude with a discussion about the implications of these results in Section 5.4.

5.1 Benchmarking baselines

Three MPSC queue algorithms are used as benchmarking baselines:

- dLTQueueV2 + our custom SPSC: Our most optimized version of LTQueue while still keeping the core algorithm intact.
- Slotqueue + our custom SPSC: Our modification to dLTQueue to obtain a more optimized distributed version of LTQueue.
- AMQueue [1]: A hosted bounded MPSC queue algorithm, already detailed in Section 3.2.

5.2 Benchmarking environment

We conduct all benchmark evaluations using computing resources at the Leibniz Supercomputing Center[1], specifically leveraging both the SuperMUC-NG[2] and CoolMUC-4[3] systems.

SuperMUC-NG provides extensive computational capacity through its configuration of over 6,000 compute nodes. Each node features 48 processing cores powered by Intel Xeon Platinum 8174 processors and is equipped with a minimum of 96GB of memory. Inter-node communication is facilitated by a high-performance OmniPath network delivering 100GBit/s bandwidth. The platform operates on SUSE Linux Enterprise Server 15.3 and employs Intel MPI Version 2019 Update 12 Build 20210429 for parallel processing coordination.

For additional computational resources, we utilize the CoolMUC-4 infrastructure, which comprises over 100 compute nodes totaling approximately 12,000 processing cores. These nodes are powered by Intel Xeon Platinum 8480+ processors, with each node providing 112 cores of processing capability. Node-to-node connectivity is established through an Infiniband network architecture. The system environment consists

[1] https://www.lrz.de/
[2] https://doku.lrz.de/supermuc-ng-10745965.html
[3] https://doku.lrz.de/coolmuc-4-1082337877.html

of SUSE Linux Enterprise Server 15.6 with Intel MPI Version 2021.12 Build 20240213 handling message passing operations.

5.3 Microbenchmark program

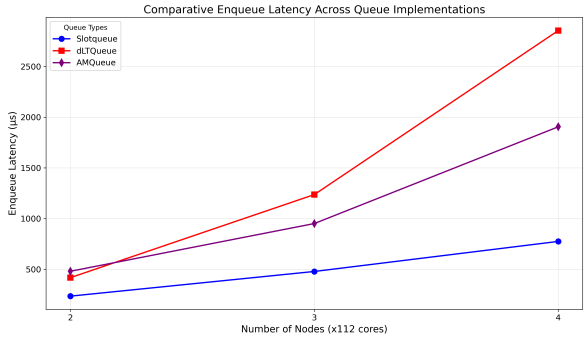
Our microbenchmark is as follows:

- All processes share a single MPSC; one of the processes is a dequeuer, and the rest are enqueuers.
- The enqueuers enqueue a total of 10^4 elements.
- The dequeuer dequeues 10^4 elements.
- The MPSC is warmed up before the dequeuer starts.

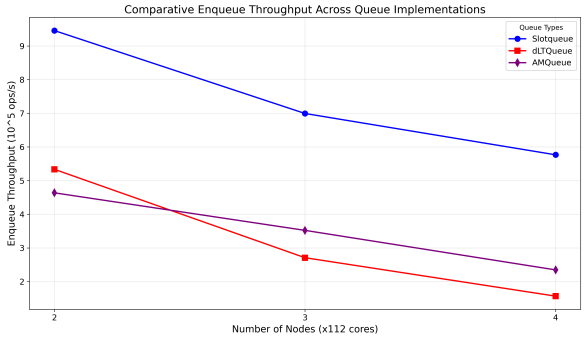
We measure the latency and throughput of the enqueue and dequeue operations. This microbenchmark is repeated 5 times for each algorithm, and we take the mean of the results.

5.4 Benchmarking results

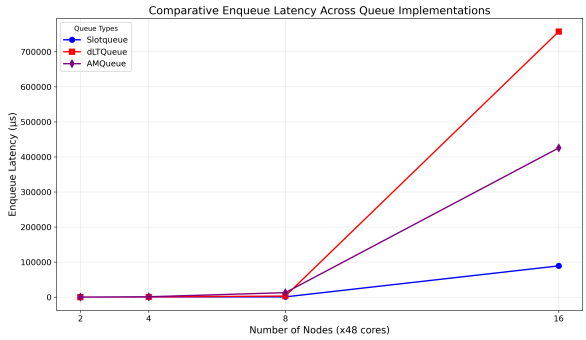
Figure 20, Figure 21, and Figure 22 showcase our benchmarking results.



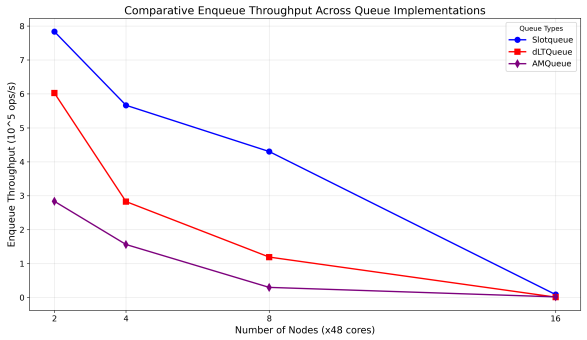
(a) Enqueue latency benchmark results on CoolMUC-4.



(b) Enqueue throughput benchmark results on CoolMUC-4.

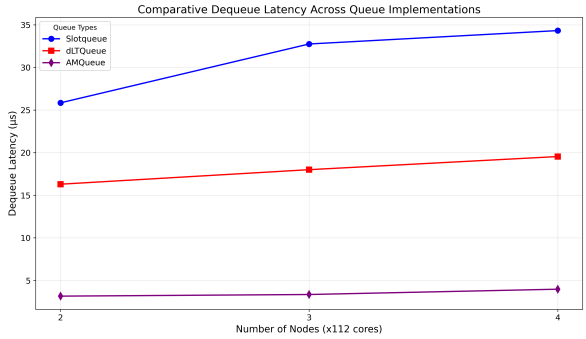


(c) Enqueue latency benchmark results on SuperMUC.

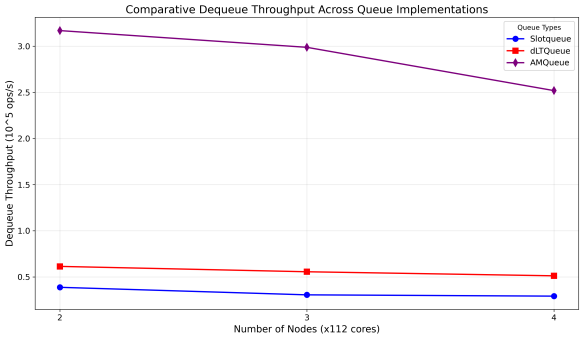


(d) Enqueue throughput benchmark results on SuperMUC.

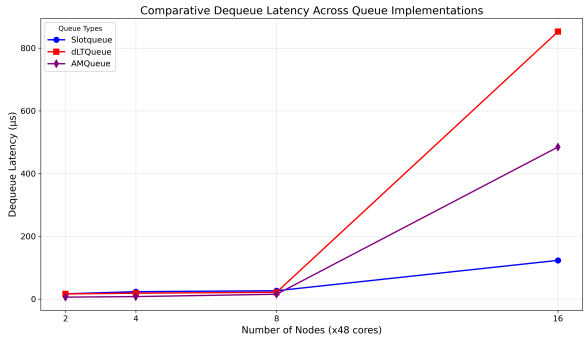
Figure 20: Microbenchmark results for the enqueue operation.



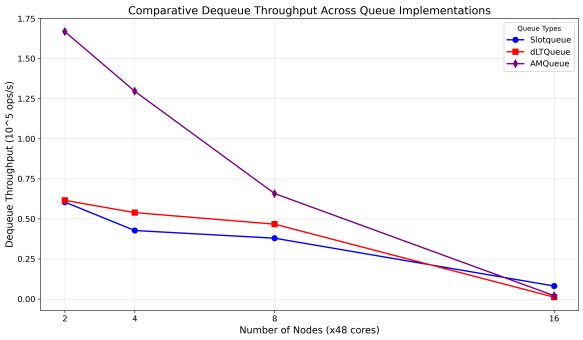
(a) Dequeue latency benchmark results on CoolMUC-4.



(b) Dequeue throughput benchmark results on CoolMUC-4.

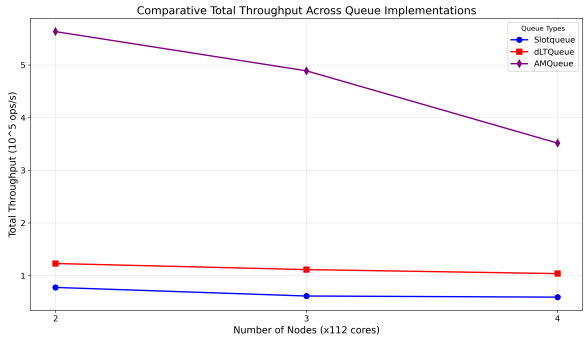


(c) Dequeue latency benchmark results on SuperMUC.

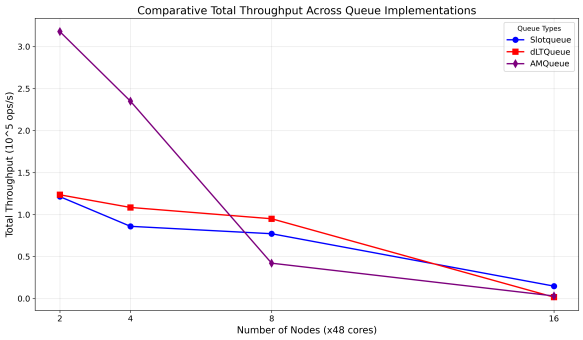


(d) Dequeue throughput benchmark results on SuperMUC.

Figure 21: Microbenchmark results for the dequeue operation.



(a) Total throughput benchmark results on CoolMUC-4.



(b) Total throughput benchmark results on SuperMUC.

Figure 22: Microbenchmark results for the total throughput.

The most evident thing is that the trends in total throughput and dequeue throughput are almost identical. This supports our claim that in an MPSC queue, the performance is bottlenecked by the dequeuer.

Slotqueue demonstrates superior enqueue performance compared to AMQueue on both CoolMUC-4 and SuperMUC-NG systems, achieving roughly double the throughput. Meanwhile, dLTQueueV2 performs the worst enqueue-wise. The performance advantage of Slotqueue results from reduced contention between concurrent enqueuers and minimal interference between enqueue and dequeue operations in Slotqueue's architecture, unlike AMQueue's design. However, despite Slotqueue and dLTQueueV2′s theoretical guarantee of constant-time enqueue operations, practical measurements reveal declining enqueue throughput as cluster size increases. This performance degradation occurs due to heightened competition for the shared counter resource among multiple enqueuers, which eventually overwhelms the interconnect infrastructure's capacity.

In contrast, Slotqueue and dLTqueue's dequeue performance significantly lags behind AMQueue, showing 10-fold slower performance on CoolMUC-4 and 3-fold slower performance on SuperMUC-NG. While Slotqueue and dLTQueueV2 execute only a limited number of remote operations per dequeue compared to AMQueue's potentially unlimited remote operations, AMQueue's batch processing capability dramatically enhances its dequeue throughput. Additionally, AMQueue's architecture stores all data locally on the dequeuer node, eliminating remote operations during dequeue processes entirely. Consequently, Slotqueue and dLTQueueV2′s benefits become less pronounced when measured against AMQueue's optimized approach. Similar to enqueue operations, dequeue throughput also declines with increasing node count, as more processes competing for access to the dequeuer node intensify contention at that critical bottleneck.

While Slotqueue and dLTQueueV2 offer superior fault tolerance compared to AMQueue and maintains competitive performance across both evaluated systems, it faces challenges under high-contention scenarios. In large-scale cluster deployments, the dequeuer node becomes overwhelmed by numerous remote memory access requests. Therefore, Slotqueue and dLTQueueV2 would benefit from additional optimizations focused on alleviating contention pressure at the dequeuer node to realize its full potential in distributed environments.

Chapter VI Conclusion

In this thesis, we introduced Slotqueue and dLTQueueV2, two high-performance non-blocking distributed MPSC (Multi-Producer Single-Consumer) queues that achieve constant-time complexity for both enqueue and dequeue operations through a fixed number of remote operations. Our research encompassed both theoretical analysis of Slotqueue and dLTQueueV2′s properties—including fault tolerance capabilities and performance characteristics—and comprehensive empirical benchmarking against the existing AMQueue implementation.

Our findings demonstrate that Slotqueue and dLTQueueV2 achieve wait-free operation with superior fault tolerance guarantees, maintaining constant remote operation counts for all queue operations. The experimental results reveal mixed performance outcomes: Slotqueue and dLTQueueV2 surpass AMQueue in enqueue throughput while experiencing 3-10 times lower performance for dequeue operations compared to AMQueue.

However, our evaluation identified two significant limitations in Slotqueue and dLTQueueV2′s current designs. First, the system experiences substantial contention degradation as the node count increases, resulting in poor scalability characteristics under large-scale deployments. Second, our existing performance model inadequately captures contention effects, leading to inaccurate throughput predictions across varying cluster sizes.

Based on these findings, our future research directions will focus on two primary objectives: developing contention mitigation strategies to prevent the dequeuer node from becoming a performance bottleneck, and creating more sophisticated performance models that accurately incorporate contention dynamics to provide reliable scalability predictions.

References

1. Schuchart J, Bouteiller A, Bosilca G (2019) Using MPI-3 RMA for active messages. In: 2019 IEEE/ACM Workshop on Exascale MPI (ExaMPI). pp 47–56
2. MPI Forum S (2012) MPI: A message-passing interface standard. Version 3.1
3. Baker M, Buyya R (1999) Cluster computing: the commodity supercomputer. Software: Practice and Experience 29:551–576
4. Mullender S (1990) Distributed systems. ACM
5. Herlihy M, Shavit N, Luchangco V, Spear M (2020) The art of multiprocessor programming. Newnes
6. Herlihy M, Shavit N (2011) On the nature of progress. In: International Conference On Principles Of Distributed Systems. pp 313–328
7. Ellen F, Brown T (2016) Concurrent data structures. In: Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing. pp 151–153
8. Hewitt C, Bishop P, Steiger R (1973) Session 8 formalisms for artificial intelligence a universal modular actor formalism for artificial intelligence. In: Advance papers of the conference. p 235
9. Malhotra A (2025) Concurrency Patterns in Golang: Real-World Use Cases and Performance Analysis. Journal of Computer Science and Technology Studies 7:782–788
10. Yang Q, Tang L, Guo Y, et al (2022) WRLqueue: A Lock-Free Queue For Embedded Real-Time System. In: 2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys). pp 1255–1260
11. Wang J, Jin Q, Fu X, et al (2019) Accelerating wait-free algorithms: Pragmatic solutions on cache-coherent multicore architectures. IEEE Access 7:74653–74669
12. Jayanti P, Petrovic S (2005) Logarithmic-time single deleter, multiple inserter wait-free queues and stacks. In: International Conference on Foundations of Software Technology and Theoretical Computer Science. pp 408–419
13. Adas D, Friedman R (2022) A fast wait-free multi-producers single-consumer queue. In: Proceedings of the 23rd International Conference on Distributed Computing and Networking. pp 77–86

14. Brock B, Buluç A, Yelick K (2019) BCL: A cross-platform distributed data structures library. In: Proceedings of the 48th International Conference on Parallel Processing. pp 1–10

15. Diep T-D, Ha PH, Fürlinger K (2023) A general approach for supporting non-blocking data structures on distributed-memory systems. Journal of Parallel and Distributed Computing 173:48–60

16. Devarajan H, Kougkas A, Bateman K, Sun X-H (2020) Hcl: Distributing parallel data structures in extreme scales. In: 2020 IEEE International Conference on Cluster Computing (CLUSTER) . pp 248–258

17. Dewan G, Jenkins L (2020) Paving the way for distributed non-blocking algorithms and data structures in the partitioned global address space model. In: 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). pp 659–666

18. Feo J, Villa O, Tumeo A, Secchi S (2011) Irregular applications: architectures & algorithms. In: Proceedings of the 1st Workshop on Irregular Applications: Architectures and Algorithms. pp 1–2

19. Gropp W, Lusk R, Ross R, Thakur R (2006) Advanced MPI: I/O and one-sided communication. In: Proc. ACM/IEEE Conf. Supercomput.. p 202

20. Dinan J, Balaji P, Buntinas D, et al (2016) An implementation and evaluation of the MPI 3.0 one-sided communication interface. Concurrency and Computation: Practice and Experience 28:4385–4404

21. Herlihy MP, Wing JM (1990) Linearizability: A correctness condition for concurrent objects. ACM Transactions on Programming Languages and Systems (TOPLAS) 12:463–492

22. Herlihy M (1991) Wait-free synchronization. ACM Transactions on Programming Languages and Systems (TOPLAS) 13:124–149

23. Michael MM, Scott ML (1996) Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In: Proceedings of the fifteenth annual ACM symposium on Principles of distributed computing. pp 267–275

24. Michael MM (2002) Safe memory reclamation for dynamic lock-free objects using atomic reads and writes. In: Proceedings of the twenty-first annual symposium on Principles of distributed computing. pp 21–30

25. Henzinger TA, Sezgin A, Vafeiadis V (2013) Aspect-oriented linearizability proofs. In: International Conference on Concurrency Theory. pp 242–256

Appendix A Theoretical aspects

This section discusses the correctness and progress guarantee properties of the distributed MPSC queue algorithms introduced in Chapter IV. Because dLTQueueV2 is very similar to dLTQueue, we skip the discussion of its theoretical aspects. Furthermore, we also provide a theoretical performance model of these algorithms to predict how well they scale to multiple nodes.

A.1 Terminology

Definition A.1.1 In an SPSC/MPSC queue, an enqueue operation e is said to **match** a dequeue operation d if d returns the value that e enqueues. Similarly, d is said to **match** e. In this case, both e and d are said to be **matched**.

Definition A.1.2 In an SPSC/MPSC queue, an enqueue operation e is said to be **unmatched** if no dequeue operation **matches** it.

Definition A.1.3 In an SPSC/MPSC queue, a dequeue operation d is said to be **unmatched** if no enqueue operation **matches** it, in other word, d returns `false`.

A.2 Preliminaries

In this section, we first formalize the system model in Section A.2.1. Based on this, we describe the aspect-oriented linearizability proof technique for queue to demonstrate dLTQueue and Slotqueue's correctness in Section A.2.2. Additionally, we formulate harmless ABA problem in Section A.2.3. We will base our proofs on these formalisms to prove their correctness.

A.2.1 System model

Here, we introduce the system model used in our correctness proofs that is compatible with the one in [25].

A *data structure* D is a list of methods. An *object* o is an instance of a data structure D. Each *event* e is a tuple of the form (o, M, d_i, d_o), where M is a method of the object o and d_i, d_o are data inputs and outputs, respectively.

An *invocation action* and a *response action* are generated by an event. For an event e, we denote $e_{i(d_i)}$ (or e_i in short) as its invocation action along with the data input and $e_{r(d_o)}$ (or e_r in short) as its response action along with the data output. A *history* h is a sequence of invocation actions and response actions generated by some data structure. An event e precedes another event e' in h, written $e \underset{h}{\prec} e'$, if the response of e occurs

before the invocation of e' in h. A history h is called *complete* if it does not have any pending events.

Consider a history h. Suppose every enqueue event inserts a unique value. Consider an enqueue event e and a dequeue event d. e is said to *match* d at time t if e_i and d_r both happen before t and e_i's input is d_r's output. In this case, e and d are said to be *matched* at time t. e is said to be *unmatched* at time t if e_i happens after t or no d exists that d_r happens before t and e_i's input is d_r's output.

A.2.2 Aspect-oriented linearizability proof

Consider a history h. Suppose every enqueue event inserts a unique value. Consider an enqueue event e and a dequeue event d. e is said to *match* d at time t if e_i and d_r both happen before t and e's input is d's output. In this case, e and d are said to be *matched* at time t. e is said to be *unmatched* at time t if e_i happens after t or no d exists that d_r happens before t and e's input is d's output.

We suppose that every enqueue inserts a unique value. In a complete history h of an MPSC queue, there are 4 types of violation, based on [25].

- (`VFresh`): A dequeue returns a value not previously inserted by any enqueue. Formally, there exists a dequeue event that returns `true` at time t but no enqueue event matches it at time t.
- (`VRepet`): Two dequeues return the value inserted by the same enqueue. Formally, there exists an enqueue event that matches two dequeue events at some time t.
- (`VOrd`): Two values are enqueued in a certain order, and a dequeue returns the later value before any dequeue of the earlier value starts. Formally, at some time t, there exist enqueue events e_1, e_2 such that $e_1 \underset{h}{\prec} e_2$, e_2 matches d_2 at time t but e_1 is unmatched at time t.
- (`VWit`): A dequeue returning `false` even though the queue is never empty during the execution of the dequeue. Formally, there exists a dequeue d starting at time t and returning `false` but there is an enqueue e that finishes before t and is still unmatched at t. This notion of `VWit` has been simplified for MPSC and SPSC queues. In the original paper, this violation is more complex to formulate.

We can derive the important Theorem A.2.2.1 from [25].

Theorem A.2.2.1 Every wait-free MPSC queue implementation is linearizable if all its complete histories have none of the `VFresh`, `VRepet`, `VOrd` and `VWit` violations.

A.2.3 ABA-safety

Not every ABA problem is unsafe. In this section, we formalize which ABA problem is safe and which is not.

Definition A.2.3.1 A **modification instruction** on a variable v is an atomic instruction that may change the value of v e.g. a store or a CAS.

Definition A.2.3.2 A **successful modification instruction** on a variable v is an atomic instruction that changes the value of v e.g. a store or a successful CAS.

Definition A.2.3.3 A **CAS-sequence** on a variable v is a sequence of instructions of a method m such that:

- The first instruction is a load $v_0 = \texttt{load}(v)$.
- The last instruction is a $\texttt{CAS}(\&v, v_0, v_1)$.
- There's no modification instruction on v between the first and the last instruction.

Definition A.2.3.4 A **successful CAS-sequence** on a variable v is a **CAS-sequence** on v that ends with a successful CAS.

Definition A.2.3.5 Consider a method m on a concurrent object S. m is said to be **ABA-safe** if and only if for any history of method calls produced from S, we can reorder any successful CAS-sequences inside an invocation of m in the following fashion:

- If a successful CAS-sequence is part of an invocation of m, after reordering, it must still be part of that invocation.
- If a successful CAS-sequence by an invocation of m precedes another by that invocation, after reordering, this ordering is still respected.
- Any successful CAS-sequence by an invocation of m after reordering must not overlap with a successful modification instruction on the same variable.
- After reordering, all method calls' response events on the concurrent object S stay the same.

A.3 Theoretical proofs of the distributed SPSC

In this section, we focus on the correctness and progress guarantee of the simple distributed SPSC established in Section 4.1.

A.3.1 ABA problem

There is no CAS instruction in our simple distributed SPSC, so there's no potential for ABA problem.

A.3.2 Memory reclamation

There is no dynamic memory allocation and deallocation in our simple distributed SPSC, so it is memory-safe.

A.3.3 Linearizability

We prove that our simple distributed SPSC is linearizable.

Theorem A.3.3.1 *(Linearizability of the simple distributed SPSC)* The distributed SPSC given in Section 4.1 is linearizable.

Proof We claim that the following are the linearization points of our SPSC's methods:

- The linearization point of an `spsc_enqueue` call (Procedure 4) that returns `false` is Line 3.
- The linearization point of an `spsc_enqueue` call (Procedure 4) that returns `true` is Line 7.
- The linearization point of an `spsc_dequeue` call (Procedure 6) that returns `false` is Line 19.
- The linearization point of an `spsc_dequeue` call (Procedure 6) that returns `true` is Line 23.
- The linearization point of $\texttt{spsc_readFront}_\texttt{e}$ call (Procedure 5) that returns `false` is Line 11 or Line 12 if Line 11 is passed.
- The linearization point of $\texttt{spsc_readFront}_\texttt{e}$ call (Procedure 5) that returns `true` is Line 12.
- The linearization point of $\texttt{spsc_readFront}_\texttt{d}$ call (Procedure 7) that returns `false` is Line 27.
- The linearization point of $\texttt{spsc_readFront}_\texttt{d}$ call (Procedure 7) that returns `true` is right after Line 27 or right before Line 30 if Line 27 is never executed.

We define a total ordering $<$ on the set of completed method calls based on these linearization points: If the linearization point of a method call A is before the linearization point of a method call B, then $A < B$.

If the distributed SPSC is linearizable, $<$ would define a equivalent valid sequential execution order for our SPSC method calls.

A valid sequential execution of SPSC method calls would possess the following characteristics.

An enqueue can only be matched by one dequeue: Each time an `spsc_dequeue` is executed, it advances the `First` index. Because only one dequeue can happen at a time, it is guaranteed that each dequeue proceeds with one unique `First` index. Two dequeues

can only dequeue out the same entry in the SPSC's array if their `First` indices are congurent modulo `Capacity`. However, by then, this entry must have been overwritten. Therefore, an enqueue can only be dequeued at most once.

A dequeue can only be matched by one enqueue: This is trivial, as based on how Procedure 6 is defined, a dequeue can only dequeue out at most one value.

The order of item dequeues is the same as the order of item enqueues: To put more precisely, if there are 2 `spsc_enqueues` e_1, e_2 such that $e_1 < e_2$, then either e_2 is unmatched or e_1 matches d_1 and e_2 matches d_2 such that $d_1 < d_2$. If e_2 is unmatched, the statement holds. Suppose e_2 matches d_2. Because $e_1 < e_2$, based on how Procedure 4 is defined, e_1 corresponds to a value i_1 of `Last` and e_2 corresponds to a value i_2 of `Last` such that $i_1 < i_2$. Based on how Procedure 6 is defined, each time a dequeue happens successfully, `First` would be incremented. Therefore, for e_2 to be matched, e_1 must be matched first because `First` must surpass i_1 before getting to i_2. In other words, e_1 matches d_1 such that $d_1 < d_2$.

An enqueue can only be matched by a later dequeue: To put more precisely, if an `spsc_enqueue` e matches an `spsc_dequeue` d, then $e < d$. If e hasn't executed its linearization point at Line 7, there's no way d's Line 22 can see e's value. Therefore, d's linearization point at Line 23 must be after e's linearization point at Line 7. Therefore, $e < d$.

A dequeue would return `false` when the queue is empty: To put more precisely, for an `spsc_dequeue` d, if by d's linearization point, every successful `spsc_enqueue` e' such that $e' < d$ has been matched by d' such that $d' < d$, then d would be unmatched and return `false`. By this assumption, any `spsc_enqueue` e that has executed its linearization point at Line 7 before d's Line 18 has been matched. Therefore, `First = Last` at Line 18, or `First >= Last_buf`, therefore, the if condition at Line 18 - Line 21 is entered. Also by the assumption, any `spsc_enqueue` e that has executed its linearization point at Line 7 before d's Line 20 has been matched. Therefore, `First = Last` at Line 20. Then, Line 21 is executed and d returns `false`.

A dequeue would return `true` and match an enqueue when the queue is not empty: To put more precisely, for an `spsc_dequeue` d, if there exists a successful `spsc_enqueue` e' such that $e' < d$ and has not been matched by a dequeue d' such that $d' < e'$, then d would be match some e and return `true`. By this assumption, some e' must have executed its linearization point at Line 7 but is still unmatched by the time d starts. Then, `First < Last`, so d must match some enqueue e and returns `true`.

An enqueue would return `false` when the queue is full: To put more precisely, for an `spsc_enqueue` e, if by e's linearization point, the number of unmatched successful `spsc_enqueue` $e' < e$ by the time e starts equals `Capacity`, then e returns `false`. By this assumption, any d' that matches e' must satisfy $e < d'$, or d' must execute its synchronization point at Line 23 after Line 1 and Line 4 of e, then e's Line 5 must have executed and return `false`.

An enqueue would return `true` when the queue is not full and the number of elements should increase by one: To put more precisely, for an `spsc_enqueue` e, if by e's linearization point, the number of unmatched successful `spsc_enqueue` $e' < e$ by the time e starts is fewer than `Capacity`, then e returns `true`. By this assumption, `First < Last` at least until e's linearization point and because Line 7 must be executed, which means the number of elements should increase by one.

A read-front would return `false` when the queue is empty: To put more precisely, for a read-front r, if by r's linearization point, every successful `spsc_enqueue` e' such that $e' < r$ has been matched by d' such that $d' < r$, then r would return `false`. That means any unmatched successful `spsc_enqueue` e must have executed its linearization point at Line 7 after r's, or `First = Last` before r's linearization point

- For an enqueuer's read-front, if r doesn't pass Line 10, the statement holds. If r passes Line 10, by the assumption, r would execute Line 14, because r sees that `First = Last`.
- For a dequeuer's read-front, r must enter Line 27 because `First_buf >= Last_buf`, which is due to from the dequeuer's point of view, `First_buf = First` and `Last_buf <= Last`. Similarly, r must execute Line 29 and return `false`.

A read-front would return `true` and the first element in the queue is read out: To put more precisely, for a read-front r, if before r's linearization point, there exists some unmatched successful `spsc_enqueue` e' such that $e' < r$, then r would read out the same value as the first d such that $r < d$. By this assumption, any d' that matches some of these successful `spsc_enqueue` e' must execute its linearization point at Line 23 after r's linearization point. Therefore, `First < Last` until r's linearization point.

- For an enqueuer's read-front, r must not execute Line 11 and Line 14. Therefore, Line 15 is executed, and `First_buf` at this point is the same as `First_buf` of the first d such that $r < d$, because we have just read it at Line 12, and any successful $d' > r$ must execute Line 23 after Line 15, therefore, `First` has no chance to be incremented between Line 12 and Line 15.
- For a dequeuer's read-front, r must not execute Line 27 - Line 29 and execute Line 30 instead. It's trivial that r reads out the same value as the first dequeue d such that $r < d$ because there can only be one dequeuer.

In conclusion, for any completed history of method calls our SPSC can produce, we have defined a way to sequentially order them in a way that conforms to SPSC's sequential specification. Therefore, our SPSC is linearizable. □

A.3.4 Progress guarantee

Our simple distributed SPSC is wait-free:

- `spsc_dequeue` (Procedure 6) does not execute any loops or wait for any other method calls.
- `spsc_enqueue` (Procedure 4) does not execute any loops or wait for any other method calls.
- $\texttt{spsc_readFront}_{\texttt{e}}$ (Procedure 5) does not execute any loops or wait for any other method calls.
- $\texttt{spsc_readFront}_{\texttt{d}}$ (Procedure 7) does not execute any loops or wait for any other method calls.

A.3.5 Theoretical performance

A summary of the theoretical performance of our simple SPSC is provided in Table 5. In the following discussion, R means remote operations and L means local operations.

Operations	Time-complexity
`spsc_enqueue`	$R + L$
`spsc_dequeue`	$R + L$
$\texttt{spsc_readFront}_{\texttt{e}}$	$R + L$
$\texttt{spsc_readFront}_{\texttt{d}}$	R

Table 5: Theoretical performance summary of our simple distributed SPSC. R means remote operations and L means local operations.

For `spsc_enqueue`, we consider the procedure Procedure 4. In the usual case, the remote operation on Line 3 is skipped and so only 2 remote puts are performed on Line 6 and

Line 7. The Data array on Line 6 is hosted on the enqueuer, so this is actually a local operation, while the control variable is hosted on the dequeuer, so Line 7 is truly a remote operation. Therefore, theoretically, it is one remote operation plus a local one.

For `spsc_dequeue`, we consider the procedure Procedure 6. Similarly, in the usual case, the remote operation on Line 19 is skipped and only the 2 lines Line 22 and Line 23 are executed always. Here, it is the other way around, the access to the `Data` array on Line 22 is a truly remote operation while the access to the First control variable is a local one. Therefore, theoretically, it is one remote operation plus a local one.

For $\texttt{spsc_readFront}_{\texttt{e}}$, we consider the procedure Procedure 5. The operation on Line 12 is a truly remote operation, as the `First` control variable is hosted on the dequeuer. The operation on Line 15 is a remote operation, as the `Data` array is hosted on the enqueuer. This means, theoretically, it also takes one remote operation plus a local one.

For $\texttt{spsc_readFront}_{\texttt{d}}$, we consider the procedure Procedure 7. Only the operation on Line 30 is executed always, which results in a truly remote operation as the Data array is hosted on the enqueuer. Therefore, it only takes one remote operation.

A.4 Theoretical proofs of dLTQueue

In this section, we provide proofs covering all of our interested theoretical aspects in dLTQueue.

A.4.1 Proof-specific notations

The structure of dLTQueue is presented again in Figure 23.

As a reminder, the bottom rectangular nodes are called the **enqueuer nodes** and the circular node are called the **tree nodes**. Tree nodes that are attached to an enqueuer node are called **leaf nodes**, otherwise, they are called **internal nodes**. Each **enqueuer node** is hosted on the enqueuer that corresponds to it. The enqueuer nodes accomodate an instance of our distributed SPSC in Section 4.1 and a `Min_timestamp` variable representing the minimum timestamp inside the SPSC. Each **tree node** stores a rank of a enqueuer that is attached to the subtree which roots at the **tree node**.

We will refer $\texttt{propagate}_{\texttt{e}}$ and $\texttt{propagate}_{\texttt{d}}$ as `propagate` if there's no need for discrimination. Similarly, we will sometimes refer to $\texttt{refreshNode}_{\texttt{e}}$ and $\texttt{refreshNode}_{\texttt{d}}$ as `refreshNode`, $\texttt{refreshLeaf}_{\texttt{e}}$ and $\texttt{refreshLeaf}_{\texttt{d}}$ as `refreshLeaf`, $\texttt{refreshTimestamp}_{\texttt{e}}$ and $\texttt{refreshTimestamp}_{\texttt{d}}$ as `refreshTimestamp`.

Definition A.4.1.1 For a tree node n, the rank stored in n at time t is denoted as $rank(n, t)$.

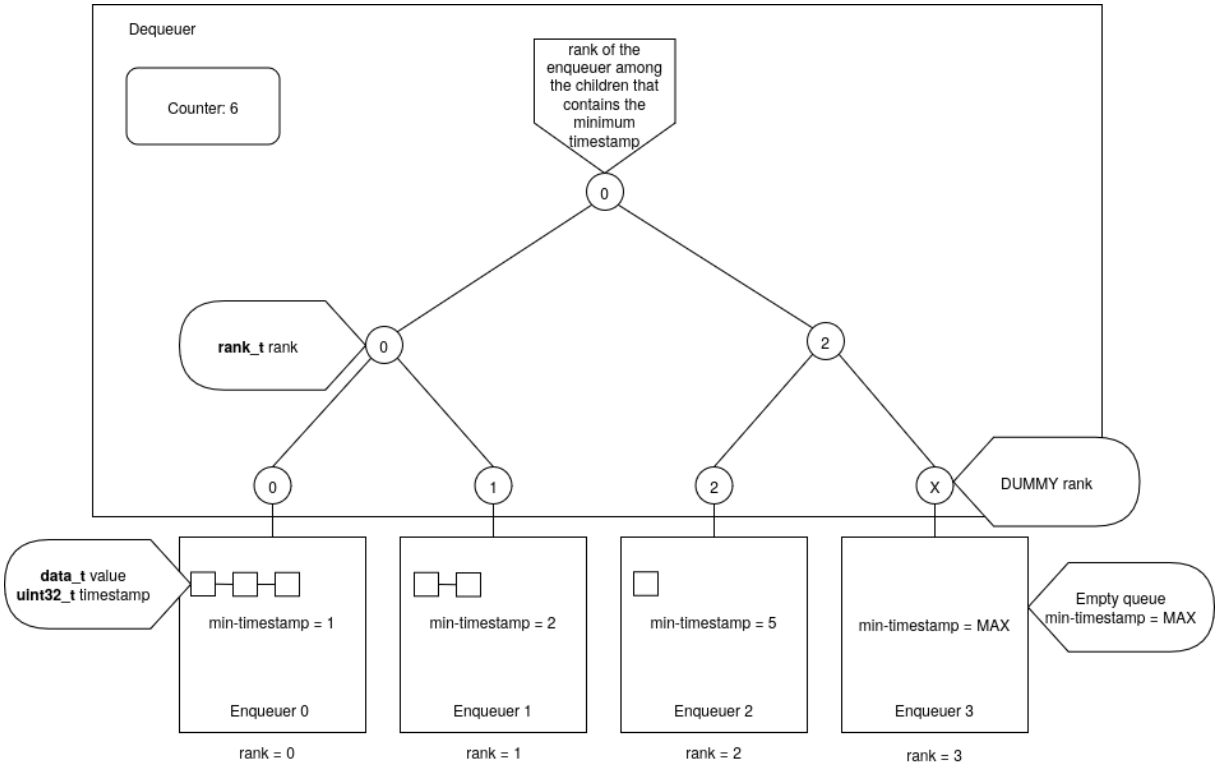


Figure 23: dLTQueue's structure.

Definition A.4.1.2 For an enqueue or a dequeue `op`, the rank of the enqueuer it affects is denoted as $rank(\texttt{op})$.

Definition A.4.1.3 For an enqueuer whose rank is r, the `Min_timestamp` value stored in its enqueuer node at time t is denoted as $min\text{-}ts(r, t)$. If r is `DUMMY_RANK`, $min\text{-}ts(r, t)$ is `MAX_TIMESTAMP`.

Definition A.4.1.4 For an enqueuer with rank r, the minimum timestamp among the elements between `First` and `Last` in its SPSC at time t is denoted as $min\text{-}spsc\text{-}ts(r, t)$. If r is dummy, $min\text{-}spsc\text{-}ts(r, t)$ is `MAX`.

Definition A.4.1.5 For an enqueue or a dequeue `op`, the set of nodes that it calls `refreshNode` (Procedure 14 or Procedure 19) or `refreshLeaf` (Procedure 15 or Procedure 20) on is denoted as $path(\texttt{op})$.

Definition A.4.1.6 For an enqueue or a dequeue, **timestamp-refresh phase** refer to its execution of Line 18 - Line 19 in $\texttt{propagate}_\texttt{e}$ (Procedure 12) or Line 71 - Line 72 in $\texttt{propagate}_\texttt{d}$ (Procedure 17).

Definition A.4.1.7 For an enqueue `op`, and a node $n \in path(\texttt{op})$, **node-n-refresh phase** refer to its execution of:

- Line 20 - Line 21 of $\texttt{propagate}_\texttt{e}$ (Procedure 12) if n is a leaf node.
- Line 25 - Line 26 of $\texttt{propagate}_\texttt{e}$ (Procedure 12) to refresh n's rank if n is a non-leaf node.

Definition A.4.1.8 For a dequeue op, and a node $n \in path(\texttt{op})$, **node-n-refresh phase** refer to its execution of:

- Line 73 - Line 74 of $\texttt{propagate}_\texttt{d}$ (Procedure 17) if n is a leaf node.
- Line 78 - Line 79 of $\texttt{propagate}_\texttt{d}$ (Procedure 17) to refresh n's rank if n is a non-leaf node.

Definition A.4.1.9 $\texttt{refreshTimestamp}_\texttt{e}$ (Procedure 13) is said to start its **CAS-sequence** if it finishes Line 29. $\texttt{refreshTimestamp}_\texttt{e}$ is said to end its **CAS-sequence** if it finishes Line 34 or Line 36.

Definition A.4.1.10 $\texttt{refreshTimestamp}_\texttt{d}$ (Procedure 18) is said to start its **CAS-sequence** if it finishes Line 82. $\texttt{refreshTimestamp}_\texttt{d}$ is said to end its **CAS-sequence** if it finishes Line 87 or Line 89.

Definition A.4.1.11 $\texttt{refreshNode}_\texttt{e}$ (Procedure 14) is said to start its **CAS-sequence** if it finishes Line 38. $\texttt{refreshNode}_\texttt{e}$ is said to end its **CAS-sequence** if it finishes Line 52.

Definition A.4.1.12 $\texttt{refreshNode}_\texttt{d}$ (Procedure 19) is said to start its **CAS-sequence** if it finishes Line 92. $\texttt{refreshNode}_\texttt{d}$ is said to end its **CAS-sequence** if it finishes Line 106.

Definition A.4.1.13 $\texttt{refreshLeaf}_\texttt{e}$ (Procedure 15) is said to start its **CAS-sequence** if it finishes Line 55. $\texttt{refreshLeaf}_\texttt{e}$ is said to end its **CAS-sequence** if it finishes Line 60.

Definition A.4.1.14 $\texttt{refreshLeaf}_\texttt{d}$ (Procedure 20) is said to start its **CAS-sequence** if it finishes Line 109. $\texttt{refreshLeaf}_\texttt{d}$ is said to end its **CAS-sequence** if it finishes Line 114.

A.4.2 ABA problem

We use CAS instructions on:

- Line 34 and Line 36 of $\texttt{refreshTimestamp}_\texttt{e}$ (Procedure 13).
- Line 52 of $\texttt{refreshNode}_\texttt{e}$ (Procedure 14).
- Line 60 of $\texttt{refreshLeaf}_\texttt{e}$ (Procedure 15).
- Line 87 and Line 89 of $\texttt{refreshTimestamp}_\texttt{d}$ (Procedure 18).
- Line 106 of $\texttt{refreshNode}_\texttt{d}$ (Procedure 19).
- Line 114 of $\texttt{refreshLeaf}_\texttt{d}$ (Procedure 20).

Notice that at these locations, we increase the associated version tags of the CAS-ed values. These version tags are 32-bit in size, therefore, practically, ABA problem can't virtually occur. It's safe to assume that there's no ABA problem in dLTQueue.

A.4.3 Memory reclamation

Notice that dLTQueue pushes the memory reclamation problem to the underlying SPSC. If the underlying SPSC is memory-safe, dLTQueue is also memory-safe.

A.4.4 Linearizability

We assume all enqueues succeed in this section. Note that a failed enqueue only causes the counter to increment, and does not change the queue state in any other ways.

Theorem A.4.4.1 In dLTQueue, an enqueue can only match at most one dequeue.

Proof A dequeue indirectly performs a value dequeue through `spsc_dequeue`. Because `spsc_dequeue` can only match one `spsc_enqueue` by another enqueue, the theorem holds. □

Theorem A.4.4.2 In dLTQueue, a dequeue can only match at most one enqueue.

Proof This is trivial as a dequeue can only read out at most one value, so it can only match at most one enqueue. □

Theorem A.4.4.3 Only the dequeuer and one enqueuer can operate on an enqueuer node.

Proof This is trivial based on how the algorithm is defined. □

We immediately obtain the following result.

Corollary A.4.4.4 Only one dequeue operation and one enqueue operation can operate concurrently on an enqueuer node.

Theorem A.4.4.5 The SPSC at an enqueuer node contains items with increasing timestamps.

Proof Each enqueue would FAA the distributed counter (Line 13 in Procedure 11) and enqueue into the SPSC an item with the timestamp obtained from that counter. Applying Corollary A.4.4.4, we know that items are enqueued one at a time into the SPSC. Therefore, later items are enqueued by later enqueues, which obtain increasing values by FAA-ing the shared counter. The theorem holds. □

Theorem A.4.4.6 For an enqueue or a dequeue op, if op modifies an enqueuer node and this enqueuer node is attached to a leaf node l, then $path(\texttt{op})$ is the set of nodes lying on the path from l to the root node.

Proof This is trivial considering how $\texttt{propagate}_\texttt{e}$ (Procedure 12) and $\texttt{propagate}_\texttt{d}$ (Procedure 17) work. □

Theorem A.4.4.7 For any time t and a node n, $rank(n, t)$ can only be DUMMY_RANK or the rank of an enqueuer that is attached to the subtree rooted at n.

Proof This is trivial considering how $\texttt{refreshNode}_\texttt{e}$, $\texttt{refreshNode}_\texttt{d}$ and $\texttt{refreshLeaf}_\texttt{e}$, $\texttt{refreshLeaf}_\texttt{d}$ works. □

Theorem A.4.4.8 If an enqueue or a dequeue op begins its **timestamp-refresh phase** at t_0 and finishes at time t_1, there's always at least one successful call to $\texttt{refreshTimestamp}_\texttt{e}$ (Procedure 13) or $\texttt{refreshTimestamp}_\texttt{d}$ (Procedure 18) that affects the enqueuer node corresponding to $rank(\texttt{op})$ and this successful call starts and ends its **CAS-sequence** between t_0 and t_1.

Proof Suppose the interested **timestamp-refresh phase** affects the enqueuer node n.

Notice that the **timestamp-refresh phase** of both enqueue and dequeue consists of at most 2 refreshTimestamp calls affecting n.

If one of the two refreshTimestamps of the **timestamp-refresh phase** succeeds, then the theorem obviously holds.

Consider the case where both fail.

The first refreshTimestamp fails because there's another refreshTimestamp on n ending its **CAS-sequence** successfully after t_0 but before the end of the first refreshTimestamp's **CAS-sequence**.

The second refreshTimestamp fails because there's another refreshTimestamp on n ending its **CAS-sequence** successfully after t_0 but before the end of the second refreshTimestamp's **CAS-sequence**. This another refreshTimestamp must start its **CAS-sequence** after the end of the first successful refreshTimestamp, otherwise, it would overlap with the **CAS-sequence** of the first successful refreshTimestamp, but successful **CAS-sequences** on the same enqueuer node cannot overlap as ABA problem does not occur. In other words, this another refreshTimestamp starts and successfully ends its **CAS-sequence** between t_0 and t_1.

We have proved the theorem. □

Theorem A.4.4.9 If an enqueue or a dequeue begins its **node-n-refresh phase** at t_0 and finishes at t_1, there's always at least one successful `refreshNode` or `refreshLeaf` calls affecting n and this successful call starts and ends its **CAS-sequence** between t_0 and t_1.

Proof This is similar to the above proof. □

Theorem A.4.4.10 Consider a node n. If within t_0 and t_1, any dequeue d where $n \in path(d)$ has finished its **node-n-refresh phase**, then $min\text{-}ts\big(rank(n, t_x), t_y\big)$ is monotonically decreasing for $t_x, t_y \in [t_0, t_1]$.

Proof We have the assumption that within t_0 and t_1, all dequeue where $n \in path(d)$ has finished its **node-n-refresh phase**. Notice that if n satisfies this assumption, any child of n also satisfies this assumption.

We will prove a stronger version of this theorem: Given a node n, time t_0 and t_1 such that within $[t_0, t_1]$, any dequeue d where $n \in path(d)$ has finished its **node-n-refresh phase**. Consider the last dequeue's **node-n-refresh phase** before t_0 (there maybe none). Take $t_s(n)$ and $t_e(n)$ to be the starting and ending time of the CAS-sequence of the last successful ***n*-refresh call** during this phase, or if there is none, $t_s(n) = t_e(n) = 0$. Then, $min\text{-}ts\big(rank(n, t_x), t_y\big)$ is monotonically decreasing for $t_x, t_y \in [t_e(n), t_1]$.

Consider any enqueuer node of rank r that is attached to a satisfied leaf node. For any n' that is a descendant of n, during $t_s(n')$ and t_1, there's no call to `spsc_dequeue`. Because:

- If an `spsc_dequeue` starts between t_0 and t_1, the dequeue that calls it hasn't finished its **node-n'-refresh phase**.
- If an `spsc_dequeue` starts between $t_s(n')$ and t_0, then a dequeue's **node-n'-refresh phase** must start after $t_s(n')$ and before t_0, but this violates our assumption of $t_s(n')$.

Therefore, there can only be calls to `spsc_enqueue` during $t_s(n')$ and t_1. Thus, $min\text{-}spsc\text{-}ts(r, t_x)$ can only decrease from `MAX_TIMESTAMP` to some timestamp and remain constant for $t_x \in [t_s(n'), t_1]$. (1)

Similarly, there can be no dequeue that hasn't finished its **timestamp-refresh phase** during $t_s(n')$ and t_1. Therefore, $min\text{-}ts(r, t_x)$ can only decrease from `MAX_TIMESTAMP` to some timestamp and remain constant for $t_x \in [t_s(n'), t_1]$. (2)

Consider any satisfied leaf node n_0. There can't be any dequeue that hasn't finished its **node-n_0-refresh phase** during $t_e(n_0)$ and t_1. Therefore, any successful `refreshLeaf`

affecting n_0 during $[t_e(n_0), t_1]$ must be called by an enqueue. Because there's no `spsc_dequeue`, this `refreshLeaf` can only set $rank(n_0, t_x)$ from `DUMMY_RANK` to r and this remains r until t_1, which is the rank of the enqueuer whose node it is attached to. Therefore, combining with (1), $min\text{-}ts\big(rank(n_0, t_x), t_y\big)$ is monotonically decreasing for $t_x, t_y \in [t_e(n_0), t_1]$. (3)

Consider any satisfied non-leaf node n' that is a descendant of n. Suppose during $[t_e(n'), t_1]$, we have a sequence of successful **n'-refresh calls** that start their CAS-sequences at $t_{start\text{-}0} < t_{start\text{-}1} < t_{start\text{-}2} < ... < t_{start\text{-}k}$ and end them at $t_{end\text{-}0} < t_{end\text{-}1} < t_{end\text{-}2} < ... < t_{end\text{-}k}$. By definition, $t_{end\text{-}0} = t_e(n')$ and $t_{start\text{-}0} = t_s(n')$. We can prove that $t_{end\text{-}i} < t_{start\text{-}(i+1)}$ because successful CAS-sequences cannot overlap.

Due to how `refreshNode` (Procedure 14 and Procedure 19) is defined, for any $k \geq i \geq 1$:

- Suppose $t_{rank\text{-}i}(c)$ is the time `refreshNode` reads the rank stored in the child node c, so $t_{start\text{-}i} \leq t_{rank\text{-}i}(c) \leq t_{end\text{-}i}$.
- Suppose $t_{ts\text{-}i}(c)$ is the time `refreshNode` reads the timestamp stored in the enqueuer with the rank read previously, so $t_{start\text{-}i} \leq t_{ts\text{-}i}(c) \leq t_{end\text{-}i}$.
- There exists a child c_i such that $rank(n', t_{end\text{-}i}) = rank(c_i, t_{rank\text{-}i}(c_i))$. (4)
- For every child c of n',
  $min\text{-}ts(rank(n', t_{end\text{-}i}), t_{ts\text{-}i}(c_i))$
  $\leq min\text{-}ts(rank(c, t_{rank\text{-}i}(c)), t_{ts\text{-}i}(c))$. (5)

Suppose the stronger theorem already holds for every child c of n'. (6)

For any $i \geq 1$, we have $t_e(c) \leq t_s(n') \leq t_{start\text{-}(i-1)} \leq t_{rank\text{-}(i-1)}(c) \leq t_{end\text{-}(i-1)} \leq t_{start\text{-}i} \leq t_{rank\text{-}i}(c) \leq t_1$. Combining with (5), (6), we have for any $k \geq i \geq 1$,
$min\text{-}ts(rank(n', t_{end\text{-}i}), t_{ts\text{-}i}(c_i))$
$\leq min\text{-}ts(rank(c, t_{rank\text{-}i}(c)), t_{ts\text{-}i}(c))$
$\leq min\text{-}ts\Big(rank\Big(c, t_{rank\text{-}(i-1)}(c)\Big), t_{ts\text{-}i}(c)\Big)$.

Choose $c = c_{i-1}$ as in (4). We have for any $k \geq i \geq 1$,
$min\text{-}ts(rank(n', t_{end\text{-}i}), t_{ts\text{-}i}(c_i))$
$\leq min\text{-}ts(rank\Big(c_{i-1}, t_{rank\text{-}(i-1)}(c_{i-1})\Big), t_{ts\text{-}i}(c_{i-1}))$
$= min\text{-}ts(rank\Big(n', t_{end\text{-}(i-1)}\Big), t_{ts\text{-}i}(c_{i-1})$.

Because $t_{ts\text{-}i}(c_i) \leq t_{end\text{-}i}$ and $t_{ts\text{-}i}(c_{i-1}) \geq t_{end\text{-}(i-1)}$ and (2), we have for any $k \geq i \geq 1$,
$min\text{-}ts(rank(n', t_{end\text{-}i}), t_{end\text{-}i})$
$\leq min\text{-}ts\Big(rank\Big(n', t_{end\text{-}(i-1)}\Big), t_{end\text{-}(i-1)}\Big)$. (*)

$rank(n', t_x)$ can only change after each successful `refreshNode`, therefore, the sequence of its value is $rank(n', t_{end\text{-}0})$, $rank(n', t_{end\text{-}1})$, ..., $rank(n', t_{end\text{-}k})$. $(**)$

Note that if `refreshNode` observes that an enqueuer has a `Min_timestamp` of `MAX_TIMESTAMP`, it would never try to CAS n''s rank to the rank of that enqueuer (Line 46 of Procedure 14 and Line 100 of Procedure 19). So, if `refreshNode` actually sets the rank of n' to some non-`DUMMY_RANK` value, the corresponding enqueuer must actually has a non-`MAX_TIMESTAMP` `Min-timestamp` *at some point*. Due to (2), this is constant up until t_1. Therefore, $min\text{-}ts(rank(n', t_{end\text{-}i}), t))$ is constant for any $t \geq t_{end\text{-}i}$ and $k \geq i \geq 1$. $min\text{-}ts(rank(n', t_{end\text{-}0}), t))$ is constant for any $t \geq t_{end\text{-}0}$ if there's a `refreshNode` before t_0. If there's no `refreshNode` before t_0, it is constantly `MAX_TIMESTAMP`. So, $min\text{-}ts(rank(n', t_{end\text{-}i}), t))$ is constant for any $t \geq t_{end\text{-}i}$ and $k \geq i \geq 0$. $(***)$

Combining $(*)$, $(**)$, $(***)$, we obtain the stronger version of the theorem. □

Theorem A.4.4.11 If an enqueue e obtains a timestamp c, finishes at time t_0 and is still **unmatched** at time t_1, then for any subrange T of $[t_0, t_1]$ that does not overlap with a dequeue, $min\text{-}ts(rank(root, t_r), t_s) \leq c$ for any $t_r, t_s \in T$.

Proof We will prove a stronger version of this theorem: Suppose an enqueue e obtains a timestamp c, finishes at time t_0 and is still **unmatched** at time t_1. For every $n_i \in path(e)$, n_0 is the leaf node and n_i is the parent of n_{i-1}, $i \geq 1$. If e starts and finishes its **node-n_i-refresh phase** at $t_{start\text{-}i}$ and $t_{end\text{-}i}$ then for any subrange T of $[t_{end\text{-}i}, t_1]$ that does not overlap with a dequeue d where $n_i \in path(d)$ and d hasn't finished its **node n_i refresh phase**, $min\text{-}ts(rank(n_i, t_r), t_s) \leq c$ for any $t_r, t_s \in T$.

If $t_1 < t_0$ then the theorem holds.

Take r_e to be the rank of the enqueuer that performs e.

Suppose e enqueues an item with the timestamp c into the local SPSC at time $t_{enqueue}$. Because it is still unmatched up until t_1, c is always in the local SPSC during $t_{enqueue}$ to t_1. Therefore, $min\text{-}spsc\text{-}ts(r_e, t) \leq c$ for any $t \in [t_{enqueue}, t_1]$. (1)

Suppose e finishes its **timestamp refresh phase** at $t_{r\text{-}ts}$. Because $t_{r\text{-}ts} \geq t_{enqueue}$, due to (1), $min\text{-}ts(r_e, t) \leq c$ for every $t \in [t_{r\text{-}ts}, t_1]$. (2)

Consider the leaf node $n_0 \in path(e)$. Due to (2), $rank(n_0, t)$ is always r_e for any $t \in [t_{end\text{-}0}, t_1]$. Also due to (2), $min\text{-}ts(rank(n_0, t_r), t_s) \leq c$ for any $t_r, t_s \in [t_{end\text{-}0}, t_1]$.

Consider any non-leaf node $n_i \in path(e)$. We can extend any subrange T to the left until we either:

- Reach a dequeue d such that $n_i \in path(d)$ and d has just finished its **node-n_i-refresh phase**.
- Reach $t_{end\text{-}i}$.

Consider one such subrange T_i.

Notice that T_i always starts right after a **node-n_i-refresh phase**. Due to Theorem A.4.4.9, there's always at least one successful `refreshNode` in this **node-n_i-refresh phase**.

Suppose the stronger version of the theorem already holds for n_{i-1}. That is, if e starts and finishes its **node-n_{i-1}-refresh phase** at $t_{start\text{-}(i-1)}$ and $t_{end\text{-}(i-1)}$ then for any subrange T of $\left[t_{end\text{-}(i-1)}, t_1\right]$ that does not overlap with a dequeue d where $n_i \in path(d)$ and d hasn't finished its **node n_{i-1} refresh phase**, $min\text{-}ts(rank(n_i, t_r), t_s) \leq c$ for any $t_r, t_s \in T$.

Extend T_i to the left until we either:

- Reach a dequeue d such that $n_i \in path(d)$ and d has just finished its **node-n_{i-1}-refresh phase**.
- Reach $t_{end\text{-}(i-1)}$.

Take the resulting range to be T_{i-1}. Obviously, $T_i \subseteq T_{i-1}$.

T_{i-1} satisifies both criteria:

- It's a subrange of $\left[t_{end\text{-}(i-1)}, t_1\right]$.
- It does not overlap with a dequeue d where $n_i \in path(d)$ and d hasn't finished its **node-n_{i-1}-refresh phase**.

Therefore, $min\text{-}ts(rank(n_{i-1}, t_r), t_s) \leq c$ for any $t_r, t_s \in T_{i-1}$.

Consider the last successful `refreshNode` on n_i ending not after T_i starts. Take $t_{s'}$ and $t_{e'}$ to be the start and end time of this `refreshNode`'s CAS-sequence. Because right at the start of T_i, a **node-n_i-refresh phase** just ends, this `refreshNode` must be within this **node-n_i-refresh phase**. (4)

This `refreshNode`'s CAS-sequence must be within T_{i-1}. This is because right at the start of T_{i-1}, a **node-n_{i-1}-refresh phase** just ends and $T_{i-1} \supseteq T_i$, T_{i-1} must cover the **node-n_i-refresh phase** whose end T_i starts from. Combining with (4), $t_{s'} \in T_{i-1}$ and $t_{e'} \in T_i$. (5)

Due to how `refreshNode` is defined and the fact that n_{i-1} is a child of n_i:

- t_{rank} is the time `refreshNode` reads the rank stored in n_{i-1}, so that $t_{s'} \leq t_{rank} \leq t_{e'}$. Combining with (5), $t_{rank} \in T_{i-1}$.
- t_{ts} is the time `refreshNode` reads the timestamp from that rank $t_{s'} \leq t_{ts} \leq t_{e'}$. Combining with (5), $t_{ts} \in T_{i-1}$.
- There exists a time t', $t_{s'} \leq t' \leq t_{e'}$, $min\text{-}ts(rank(n_i, t_{e'}), t') \leq min\text{-}ts(rank(n_{i-1}, t_{rank}), t_{ts})$. (6)

From (6) and the fact that $t_{rank} \in T_{i-1}$ and $t_{ts} \in T_{i-1}$, $min\text{-}ts(rank(n_i, t_{e'}), t') \leq c$.

There shall be no `spsc_dequeue` starting within $t_{s'}$ till the end of T_i because:

- If there's an `spsc_dequeue` starting within T_i, then T_i's assumption is violated.
- If there's an `spsc_dequeue` starting after $t_{s'}$ but before T_i, its dequeue must finish its **node-n_i-refresh phase** after $t_{s'}$ and before T_i. However, then $t_{e'}$ is no longer the end of the last successful `refreshNode` on n_i not after T_i.

Because there's no `spsc_dequeue` starting in this timespan, $min\text{-}ts(rank(n_i, t_{e'}), t_{e'}) \leq min\text{-}ts(rank(n_i, t_{e'}), t') \leq c$.

If there's no dequeue between $t_{e'}$ and the end of T_i whose **node-n_i-refresh phase** hasn't finished, then by Theorem A.4.4.10, $min\text{-}ts(rank(n_i, t_r), t_s)$ is monotonically decreasing for any t_r, t_s starting from $t_{e'}$ till the end of T_i. Therefore, $min\text{-}ts(rank(n_i, t_r), t_s) \leq c$ for any $t_r, t_s \in T_i$.

Suppose there's a dequeue whose **node-n_i-refresh phase** is in progress some time between $t_{e'}$ and the end of T_i. By definition, this dequeue must finish it before T_i. Because $t_{e'}$ is the time of the last successful refresh on n_i before T_i, $t_{e'}$ must be within the **node-n_i-refresh phase** of this dequeue and there should be no dequeue after that. By the way $t_{e'}$ is defined, technically, this dequeue has finished its **node-n_i-refresh phase** right at $t_{e'}$. Therefore, similarly, we can apply Theorem A.4.4.10, $min\text{-}ts(rank(n_i, t_r), t_s) \leq c$ for any $t_r, t_s \in T_i$.

By induction, we have proved the stronger version of the theorem. Therefore, the theorem directly follows. □

Corollary A.4.4.12 Suppose *root* is the root tree node. If an enqueue e obtains a timestamp c, finishes at time t_0 and is still **unmatched** at time t_1, then for any subrange T of $[t_0, t_1]$ that does not overlap with a dequeue, $min\text{-}spsc\text{-}ts(rank(root, t_r), t_s) \leq c$ for any $t_r, t_s \in T$.

Proof Call t_{start} and t_{end} to be the start and end time of T.

Applying Theorem A.4.4.11, we have that $min\text{-}ts(rank(root, t_r), t_s) \leq c$ for any $t_r, t_s \in T$.

Fix t_r so that $rank(root, t_r) = r$. We have that $min\text{-}ts(r, t) \leq c$ for any $t \in T$.

$min\text{-}ts(r, t)$ can only change due to a successful `refreshTimestamp` on the enqueuer node with rank r. Consider the last successful `refreshTimestamp` on the enqueuer node with rank r not after T. Suppose that `refreshTimestamp` reads out the minimum timestamp of the local SPSC at $t' \leq t_{start}$.

Therefore, $min\text{-}ts(r, t_{start}) = min\text{-}spsc\text{-}ts(r, t') \leq c$.

We will prove that after t' until t_{end}, there's no `spsc_dequeue` on r running.

Suppose the contrary, then this `spsc_dequeue` must be part of a dequeue. By definition, this dequeue must start and end before t_{start}, else it violates the assumption of T. If this `spsc_dequeue` starts after t', then its `refreshTimestamp` must finish after t' and before t_{start}. But this violates the assumption that the last `refreshTimestamp` not after t_{start} reads out the minimum timestamp at t'.

Therefore, there's no `spsc_dequeue` on r running during $[t', t_{end}]$. Therefore, $min\text{-}spsc\text{-}ts(r, t)$ remains constant during $[t', t_{end}]$ because it is not `MAX_TIMESTAMP`.

In conclusion, $min\text{-}spsc\text{-}ts(r, t) \leq c$ for $t \in [t', t_{end}]$.

We have proved the theorem. □

Theorem A.4.4.13 Given a rank r. If within $[t_0, t_1]$, there's no uncompleted enqueue on rank r and all matching dequeues for any completed enqueues on rank r has finished, then $rank(n, t) \neq r$ for every node n and $t \in [t_0, t_1]$.

Proof If n doesn't lie on the path from root to the leaf node that is attached to the enqueuer node with rank r, the theorem obviously holds.

Due to Corollary A.4.4.4, there can only be one enqueue and one dequeue at a time at an enqueuer node with rank r. Therefore, there is a sequential ordering among the enqueues and a sequential ordering within the dequeues. Therefore, it is sensible to talk about the last enqueue before t_0 and the last matched dequeue d before t_0.

Since all of these dequeues and enqueues work on the same local SPSC and the SPSC is linearizable, d must match the last enqueue. After this dequeue d, the local SPSC is empty.

When d finishes its **timestamp-refresh phase** at $t_{ts} \leq t_0$, due to Theorem A.4.4.8, there's at least one successful `refreshTimestamp` call in this phase. Because the last enqueue has been matched, $min\text{-}ts(r, t) =$ `MAX_TIMESTAMP` for any $t \in [t_{ts}, t_1]$.

Similarly, for a leaf node n_0, suppose d finishes its **node-n_0-refresh phase** at $t_{r\text{-}0} \geq t_{ts}$, then $rank(n_0, t) =$ `DUMMY_RANK` for any $t \in [t_{r\text{-}0}, t_1]$. (1)

For any non-leaf node $n_i \in path(d)$, when d finishes its **node-n_i-refresh phase** at $t_{r\text{-}i}$, there's at least one successful `refreshNode` call during this phase. Suppose this `refreshNode` call starts and ends at $t_{start\text{-}i}$ and $t_{end\text{-}i}$. Suppose $rank(n_{i-1}, t) \neq r$ for $t \in \left[t_{r\text{-}(i-1)}, t_1\right]$. By the way `refreshNode` is defined after this `refreshNode` call, n_i will store some rank other than r. Because of (1), after this up until t_1, r never has a chance to be visible to a `refreshNode` on node n_i during $[n_{i-1}, t]$. In other words, $rank(n_i, t) \neq r$ for $t \in [t_{r\text{-}i}, t_1]$.

By induction, we obtain the theorem. □

Theorem A.4.4.14 All of dLTQueue's complete histories do not have the `VFresh` violation.

Proof Notice that the dequeuer dequeues by first reading the root node's rank and then returns a value by dequeuing from the local SPSC of the corresponding enqueuer. Suppose the SPSC is linearizable, the dequeued value must first be enqueued by some enqueuer. The theorem holds. □

Theorem A.4.4.15 All of dLTQueue's complete histories do not have the `VRepet` violation.

Proof The dequeuer dequeues by dequeuing from the local SPSC of some enqueuer. If two dequeues are dequeuing from two different local SPSC, there's no way for two dequeues to dequeue a value twice. Suppose the SPSC is linearizable, the same statement holds true for when the two dequeues are dequeuing from the same local SPSC. The theorem holds. □

Theorem A.4.4.16 All of dLTQueue's complete histories do not have the `VOrd` violation.

Proof Consider a complete history c and two enqueues e_1, e_2 such that e_1 precedes e_2. Because e_1 precedes e_2, its timestamp c_1 must be strictly smaller than e_2's timestamp c_2. Suppose e_1 finishes at time t_0 and is still unmatched at time t_1. Then, by Corollary A.4.4.12, for any subrange T of $[t_0, t_1]$, for any $t_r, t_s \in T$:

$$min\text{-}spsc\text{-}ts(rank(root, t_r), t_s) \leq c_1 < c_2$$

Therefore, before e_1 is matched, there's no chance the root node can refer to e_2. It follows that e_1 must be matched before e_2. The theorem holds. □

Theorem A.4.4.17 All of dLTQueue's complete histories do not have the `VWit` violation.

Proof Formally, we will prove that there doesn't exist an unmatched dequeue d and a finished unmatched enqueue e by the time d starts.

The only way for a dequeue to return `false` is for it to read out a `DUMMY` rank from the root node. If there's a finished unmatched enqueue by the time a dequeue starts, the root node must store a non-`DUMMY` rank, by Theorem A.4.4.11. Therefore, the theorem holds. □

Theorem A.4.4.18 dLTQueue is wait-free.

Proof This is trivial, as dLTQueue never enters a loop. □

Theorem A.4.4.19 dLTQueue is linearizable.

Proof This follows directly from Theorem A.2.2.1, Theorem A.4.4.14, Theorem A.4.4.15, Theorem A.4.4.16, Theorem A.4.4.17 and Theorem A.4.4.18. □

A.4.5 Progress guarantee

Notice that every loop in dLTQueue is bounded, and no method have to wait for another. Therefore, dLTQueue is wait-free.

A.4.6 Theoretical performance

A summary of the theoretical performance of dLTQueue is provided in Table 6, which is already shown in Table 4. In the following discussion, R means remote operations and L means local operations.

Operations	Time-complexity
`enqueue`	$6\log_2(n)R + 4\log_2(n)L$
`dequeue`	$4\log_2(n)R + 6\log_2(n)L$

Table 6: Theoretical performance summary of dLTQueue. R means remote operations and L means local operations.

For `enqueue`, we consider the procedure Procedure 11. We consider the propagation process, which causes most of the remote operations, while Line 13 and Line 14 are negligible. Notice that the number of node refreshes are proportional to the number of

the level of the trees, which is $O(n)$ for n being the number of processes. Each level of the tree in the worst case needs 2 retries, each retry would have to:

- Read the current node (which is a truly remote operation for enqueue).
- Read the two child nodes (which is 2 truly remote operations for enqueue).
- Read the two `min-timestamp` variables in the two child nodes (which is 2 truly local operations for enqueue).
- Compare-and-swap the current node (which is a truly remote opoeration for enqueue).

In total, each level requires 6 remote operations and 4 local operations. Therefore, enqueue requires about $6\log_2(n)R + 4\log_2(n)L$ operations.

For dequeue, it is similar to enqueue but the other way around, what makes for a remote operation in enqueue is a local operation in dequeue and otherwise. Therefore, dequeue requires about $4\log_2(n)R + 6\log_2(n)L$ operations.

A.5 Theoretical proofs of Slotqueue

In this section, we provide proofs covering all of our interested theoretical aspects in Slotqueue.

A.5.1 Proof-specific notations

As a refresher, Figure 24 shows the structure of Slotqueue.

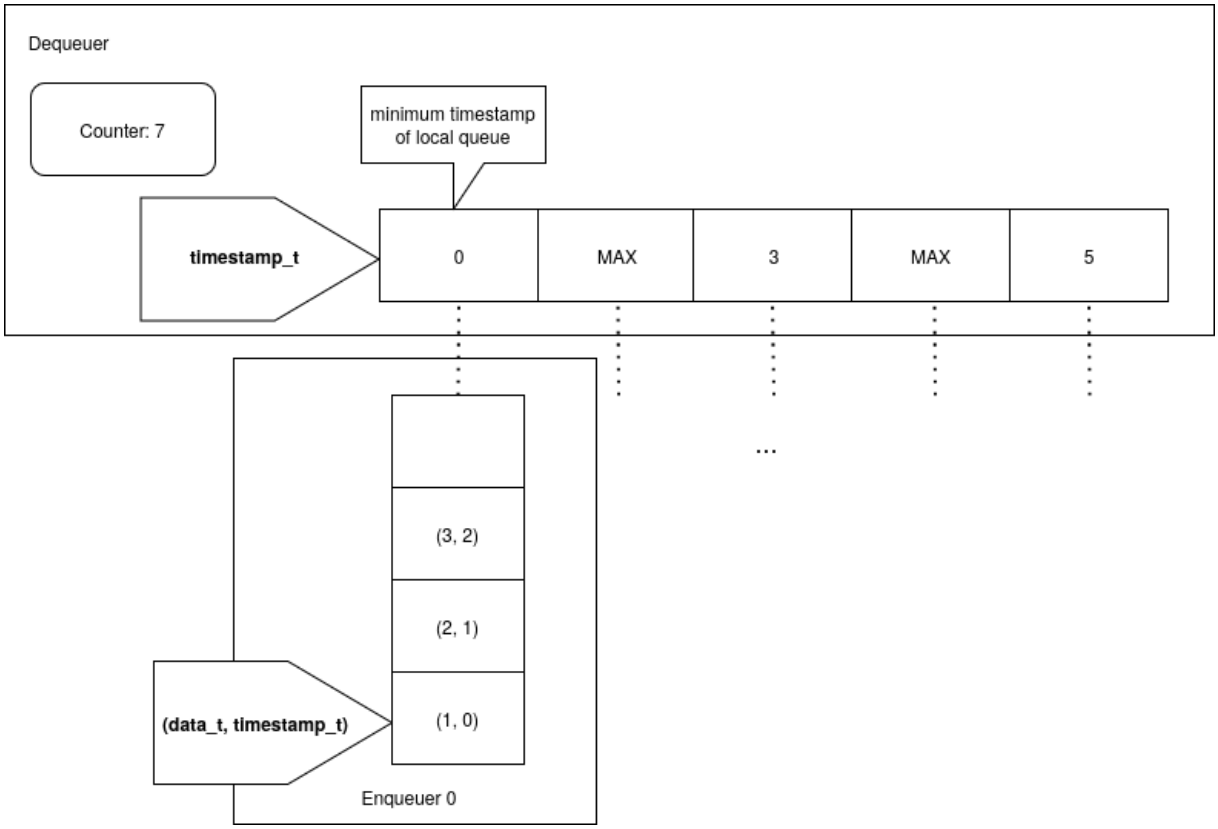


Figure 24: Basic structure of Slotqueue.

Each enqueuer hosts an SPSC that can only accessed by itself and the dequeuer. The dequeuer hosts an array of slots, each slot corresponds to an enqueuer, containing its SPSC's minimum timestamp.

We apply some domain knowledge of Slotqueue algorithm to the definitions introduced in Section A.2.3.

Definition A.5.1.1 A **CAS-sequence** on a slot `s` of an enqueue that affects `s` is the sequence of instructions from Line 12 to Line 17 of its `refreshEnqueue` (Procedure 22).

Definition A.5.1.2 A **slot-modification instruction** on a slot `s` of an enqueue that affects `s` is Line 17 of `refreshEnqueue` (Procedure 22).

Definition A.5.1.3 A **CAS-sequence** on a slot `s` of a dequeue that affects `s` is the sequence of instructions from Line 47 to Line 51 of its `refreshDequeue` (Procedure 25).

Definition A.5.1.4 A **slot-modification instruction** on a slot `s` of a dequeue that affects `s` is Line 51 of `refreshDequeue` (Procedure 25).

Definition A.5.1.5 A **CAS-sequence** of a dequeue/enqueue is said to **observe a slot value of s_0** if it loads s_0 at Line 12 of `refreshEnqueue` or Line 47 of `refreshDequeue`.

The followings are some other definitions that will be used throughout our proof.

Definition A.5.1.6 For an enqueue or dequeue op, $rank(op)$ is the rank of the enqueuer whose local SPSC is affected by op.

Definition A.5.1.7 For an enqueuer whose rank is r, the value stored in its corresponding slot at time t is denoted as $slot(r, t)$.

Definition A.5.1.8 For an enqueuer with rank r, the minimum timestamp among the elements between `First` and `Last` in its local SPSC at time t is denoted as $min\text{-}spsc\text{-}ts(r, t)$.

Definition A.5.1.9 For an enqueue, **slot-refresh phase** refer to its execution of Line 3 - Line 4 of Procedure 21.

Definition A.5.1.10 For a dequeue, **slot-refresh phase** refer to its execution of Line 26 - Line 27 of Procedure 23.

Definition A.5.1.11 For a dequeue, **slot-scan phase** refer to its execution of Line 29 - Line 43 of Procedure 24.

A.5.2 ABA problem

Noticeably, we use no scheme to avoid ABA problem in Slotqueue. In actuality, ABA problem does not adversely affect our algorithm's correctness, except in the extreme case that the 64-bit distributed counter overflows, which is unlikely.

We will prove that Slotqueue is ABA-safe, as introduced in Section A.2.3.

Notice that we only use `CAS`es on:

- Line 17 of `refreshEnqueue` (Procedure 22), which is part of an enqueue.
- Line 51 of `refreshDequeue` (Procedure 25), which is part of a dequeue.

Both `CAS`es target some slot in the `Slots` array.

Theorem A.5.2.1 *(Concurrent accesses on an SPSC and a slot)* Only one dequeuer and one enqueuer can concurrently modify an SPSC and a slot in the `Slots` array.

Proof This is trivial to prove based on the algorithm's definition. □

Theorem A.5.2.2 *(Monotonicity of SPSC timestamps)* Each SPSC in Slotqueue contains elements with increasing timestamps.

Proof Each enqueue would `FAA` the distributed counter (Line 1 in Procedure 21) and enqueue into the local SPSC an item with the timestamp obtained from the counter. Applying Theorem A.5.2.1, we know that items are enqueued one at a time into the

SPSC. Therefore, later items are enqueued by later enqueues, which obtain increasing values by `FAA`-ing the shared counter. The theorem holds. □

Theorem A.5.2.3 A `refreshEnqueue` (Procedure 22) can only change a slot to a value other than `MAX_TIMESTAMP`.

Proof For `refreshEnqueue` to change the slot's value, the condition on Line 15 must be `false`. Then, `new_timestamp` must equal to `ts`, which is not `MAX_TIMESTAMP`. It's obvious that the `CAS` on Line 17 changes the slot to a value other than `MAX_TIMESTAMP`.□

Theorem A.5.2.4 *(ABA safety of dequeue)* Assume that the 64-bit distributed counter never overflows, dequeue (Procedure 23) is ABA-safe.

Proof Consider a **successful CAS-sequence** on slot `s` by a dequeue d. Denote t_d as the value this CAS-sequence observes.

If there's no **successful slot-modification instruction** on slot `s` by an enqueue e within d's **successful CAS-sequence**, then this dequeue is ABA-safe.

Suppose the enqueue e executes the *last* **successful slot-modification instruction** on slot `s` within d's **successful CAS-sequence**. Denote t_e to be the value that e sets `s` $(*)$.

If $t_e \neq t_d$, this CAS-sequence of d cannot be successful, which is a contradiction. Therefore, $t_e = t_d$.

Note that e can only set `s` to the timestamp of the item it enqueues. That means, e must have enqueued a value with timestamp t_d. However, by definition $(*)$, t_d is read before e executes the CAS, so d cannot observe t_d because e has CAS-ed slot `s`. This means another process (dequeuer/enqueuer) has seen the value e enqueued and CAS `s` for e before t_d. By Theorem A.5.2.1, this "another process" must be another dequeuer d' that precedes d because it overlaps with e.

Because d' and d cannot overlap, while e overlaps with both d' and d, e must be the *first* enqueue on `s` that overlaps with d. Combining with Theorem A.5.2.1 and the fact that e executes the *last* **successful slot-modification instruction** on slot `s` within d's **successful CAS-sequence**, e must be the only enqueue that executes a **successful slot-modification instruction** on `s` within d's **successful CAS-sequence**.

During the start of d's successful CAS-sequence till the end of e, `spsc_readFront` on the local SPSC must return the same element, because:

- There's no other dequeue running during this time.
- There's no enqueue other than e running.
- The `spsc_enqueue` of e must have completed before the start of d's successful CAS sequence, because a previous dequeuer d' can see its effect.

Therefore, if we were to move the starting time of d's successful CAS-sequence right after e has ended, we still retain the output of the program because:

- The CAS sequence only reads two shared values: the `rank`th entry of `Slots` and `spsc_readFront()`, but we have proven that these two values remain the same if we were to move the starting time of d's successful CAS-sequence this way.
- The CAS sequence does not modify any values except for the last CAS instruction, and the ending time of the CAS sequence is still the same.
- The CAS sequence modifies the `rank`th entry of `Slots` at the CAS but the target value is the same because inputs and shared values are the same in both cases.

We have proved that if we move d's successful CAS-sequence to start after the *last* **successful slot-modification instruction** on slot `s` within d's **successful CAS-sequence**, we still retain the program's output.

If we apply the reordering for every dequeue, the theorem directly follows. □

Theorem A.5.2.5 *(ABA safety of enqueue)* Assume that the 64-bit distributed counter never overflows, enqueue (Procedure 21) is ABA-safe.

Proof Consider a **successful CAS-sequence** on slot `s` by an enqueue e. Denote t_e as the value this CAS-sequence observes.

If there's no **successful slot-modification instruction** on slot `s` by a dequeue d within e's **successful CAS-sequence**, then this enqueue is ABA-safe.

Suppose the dequeue d executes the *last* **successful slot-modification instruction** on slot `s` within e's **successful CAS-sequence**. Denote t_d to be the value that d sets `s`. If $t_d \neq t_e$, this CAS-sequence of e cannot be successful, which is a contradiction $(*)$.

Therefore, $t_d = t_e$.

If $t_d = t_e =$ `MAX_TIMESTAMP`, this means e observes a value of `MAX_TIMESTAMP` before d even sets `s` to `MAX_TIMESTAMP` due to $(*)$. If this `MAX_TIMESTAMP` value is the initialized value of `s`, it is a contradiction, as `s` must be non-`MAX_TIMESTAMP` at some point for a dequeue such as d to enter its CAS sequence. If this `MAX_TIMESTAMP` value is set by an enqueue, it is also a contradiction, as `refreshEnqueue` cannot set a slot to `MAX_TIMESTAMP`. Therefore, this `MAX_TIMESTAMP` value is set by a dequeue d'. If $d' \neq d$ then it is a contradiction, because between d' and d, `s` must be set to be a non-

`MAX_TIMESTAMP` value before d can be run, thus, e cannot have observed a value set by d'. Therefore, $d' = d$. But, this means e observes a value set by d, which violates our assumption $(*)$.

Therefore $t_d = t_e = t' \neq$ `MAX_TIMESTAMP`. e cannot observe the value t' set by d due to our assumption $(*)$. Suppose e observes the value t' from `s` set by another enqueue/dequeue call other than d.

If this "another call" is a dequeue d' other than d, d' precedes d. By Theorem A.5.2.2, after each dequeue, the front element's timestamp will be increasing, therefore, d' must have set `s` to a timestamp smaller than t_d. However, e observes $t_e = t_d$. This is a contradiction.

Therefore, this "another call" is an enqueue e' other than e and e' precedes e. We know that an enqueue only sets `s` to the timestamp it obtains.

Suppose e' does not overlap with d, then e precedes d. e' can only set `s` to t' if e' sees that the local SPSC has the front element as the element it enqueues. Due to Theorem A.5.2.1, this means e' must observe a local SPSC with only the element it enqueues. Then, when d executes `readFront`, the item e' enqueues must have been dequeued out already, thus, d cannot set `s` to t'. This is a contradiction.

Therefore, e' overlaps with d.

Because e' and e cannot overlap, while d overlaps with both e' and e, d must be the *first* dequeue on `s` that overlaps with e. Combining with Theorem A.5.2.1 and the fact that d executes the *last* **successful slot-modification instruction** on slot `s` within e's **successful CAS-sequence**, d must be the only dequeue that executes a **successful slot-modification instruction** within e's **successful CAS-sequence**.

During the start of e's successful CAS-sequence till the end of d, `spsc_readFront` on the local SPSC must return the same element, because:

- There's no other enqueue running during this time.
- There's no dequeue other than d running.
- The `spsc_dequeue` of d must have completed before the start of e's successful CAS sequence, because a previous enqueuer e' can see its effect.

Therefore, if we were to move the starting time of e's successful CAS-sequence right after d has ended, we still retain the output of the program because:

- The CAS sequence only reads two shared values: the `rank`th entry of `Slots` and `spsc_readFront()`, but we have proven that these two values remain the same if we were to move the starting time of e's successful CAS-sequence this way.
- The CAS sequence does not modify any values except for the last CAS/store instruction, and the ending time of the CAS sequence is still the same.
- The CAS sequence modifies the `rank`th entry of `Slots` at the CAS but the target value is the same because inputs and shared values are the same in both cases.

We have proved that if we move e's successful CAS-sequence to start after the *last* **successful slot-modification instruction** on slot `s` within e's **successful CAS-sequence**, we still retain the program's output.

If we apply the reordering for every enqueue, the theorem directly follows. □

Theorem A.5.2.6 *(ABA safety)* Assume that the 64-bit distributed counter never overflows, Slotqueue is ABA-safe.

Proof This follows from Theorem A.5.2.5 and Theorem A.5.2.4. □

A.5.3 Memory reclamation

Notice that Slotqueue pushes the memory reclamation problem to the underlying SPSC. If the underlying SPSC is memory-safe, Slotqueue is also memory-safe.

A.5.4 Linearizability

We assume all enqueues succeed in this section. Note that a failed enqueue only causes the counter to increment, and does not change the queue state in any other ways.

Lemma A.5.4.7 Only the dequeuer and the enqueuer with rank r can concurrently modify an SPSC and the slot at the r-th index in the `Slots` array.

Proof This lemma is trivial based on how the algorithm is defined. □

Lemma A.5.4.8 Each SPSC in Slotqueue contains elements with increasing timestamps.

Proof Each enqueue would fetch-and-add the distributed counter and enqueue into the local SPSC an item with the timestamp obtained from the counter. Applying Lemma A.5.4.7, we know that items are enqueued one at a time into the SPSC. Therefore, later items are enqueued by strictly later enqueues, which obtain increasing timestamps. The lemma holds. □

Lemma A.5.4.9 If an enqueue e begins its slot-refresh phase at time t_0 and finishes at time t_1, there's always at least one successful `refresh_enqueue` on $rank(e)$ that

does not execute its CAS sequence or one successful `refresh_enqueue` on $rank(e)$ that starts and ends its CAS sequence between t_0 and t_1 or one successful `refresh_dequeue` on $rank(e)$ that starts and ends its CAS sequence between t_0 and t_1.

Proof If one of the two `refresh_enqueues` of e succeeds, then the lemma obviously holds. Consider the case where both fail.

The first `refresh_enqueue` fails because it tries to execute its CAS sequence but there's another `refresh_dequeue` executing its slot modification instruction successfully during the first `refresh_enqueue`'s CAS sequence.

The second `refresh_enqueue` fails because it tries to execute its CAS sequence but there's another `refresh_dequeue` executing its slot modification instruction successfully during the second `refresh_enqueue`'s CAS sequence. This another `refresh_dequeue` must start its CAS sequence after the end of the first successful `refresh_dequeue`, which is after t_0, because there is only one dequeuer, and must end before t_1, because its slot modification instruction takes places during the second `refresh_enqueue`'s CAS sequence. In other words, this another `refresh_dequeue` starts and successfully ends its CAS sequence between t_0 and t_1. □

Lemma A.5.4.10 If an enqueue d begins its slot-refresh phase at time t_0 and finishes at time t_1, there's always at least one one successful `refresh_enqueue` on $rank(e)$ that starts and ends its CAS sequence between t_0 and t_1 or one successful `refresh_dequeue` on $rank(e)$ that starts and ends its CAS sequence between t_0 and t_1.

Proof This lemma is similar to the above lemma. □

Lemma A.5.4.11 Given a rank r, if a successful enqueue e on r obtains the timestamp c completes at t_0 and is still unmatched by $t_1 > t_0$, then $slot(r, t) \leq c$ for any $t \in [t_0, t_1]$.

Proof Because the underlying SPSC queue is linearizable, take $t' < t_0$ to be the time e's `spsc_enqueue` completes successfully. Because e is still unmatched until t_1, the timestamp c must be in the underlying SPSC at any time $t \in [t', t_1]$. Therefore, due to Lemma A.5.4.8, any `spsc_readFront` on rank r's SPSC queue during $[t', t_1]$ must read out a value not greater than c. Consequently, any successful refresh call (`refresh_enqueue` or `refresh_dequeue`) during $[t', t_1]$ must set the slot to some value not greater than c. (1)

At some time after t' and before t_0, e must enter its slot-refresh phase. Due to Lemma A.5.4.9, there must either be a successful refresh call during $[t', t_0]$ or the

enqueue e's `refresh_enqueue` itself must be successful without executing its CAS sequence.

If there is a successful refresh call during $[t', t_0]$. From (1), $slot(r, t) \leq c$ for any $t \in [t_0, t_1]$.

Suppose there is no successful refresh call during $[t', t_0]$. Then, e's `refresh_enqueue` is successful without executing its CAS sequence. According to the code, this is because there is another item in the queue while `refresh_enqueue` is executing. Therefore, $slot(r, t_2) \leq c$ for some time $t_2 \in [t', t_0]$. Combined with (1), we have $slot(r, t) \leq c$ for any $t \in [t_0, t_1]$.

In conclusion, the lemma holds. □

Theorem A.5.4.12 Any complete history h of Slotqueue does not have the `VFresh` violation.

Proof Consider a complete history h. Suppose in h, there exists a dequeue event that returns `true` at time t but no enqueue event matches it at time t. For a dequeue event to return `true`, its call to `spsc_dequeue` must return true. Because the SPSC is linearizable, this dequeue must match some `spsc_enqueue` to this SPSC, which is called by some enqueue. Therefore, this dequeue event must match some enqueue event, a contradiction. The theorem holds. □

Theorem A.5.4.13 Any complete history h of Slotqueue does not have the `VRepet` violation.

Proof Consider a complete history h. Suppose in h, there exists an enqueue event e that matches two dequeue events d_1, d_2 at some time t. This can only happen if d_1 and d_2 both target the same SPSC as e. However, because the SPSC is linearizable, both calls of d_1 and d_2 to `spsc_dequeue` must match different `spsc_enqueue` calls by different enqueues. Therefore, this is a contradiction. The theorem holds. □

Theorem A.5.4.14 Any complete history h of Slotqueue does not have the `VOrd` violation.

Proof Consider a complete history h. Suppose at some time t, there exist enqueue events e_1, e_2 such that $e_1 \underset{h}{\prec} e_2$, e_2 matches d_2 at time t but e_1 is unmatched at time t.

Because $e_1 \underset{h}{\prec} e_2$, e_1 obtains a timestamp smaller than e_2.

If e_1 and e_2 target the same slot, due to the underlying SPSC being linearizable and $e_1 \underset{h}{\prec} e_2$, d_2 cannot match e_2 while e_1 is still unmatched.

Note that d_2's slot-scan phase involves 2 scans.

Suppose e_1 targets the slot at a lower rank than e_2's slot. If d_2 finds e_2 in the first scan, then in the second scan, because $e_1 \underset{h}{\prec} e_2$, d_2 would have seen and prioritized e_1's timestamp, which is a contradiction. Therefore, d_2 must have found e_2 in the second scan. Suppose during the first scan, it finds $e' \neq e_2$. Then, e''s timestamp is larger than e_2's. Because during the second scan, e_1 is not chosen, its slot-refresh phase must finish after e''s, which already finishes in the first scan. Because $e_1 \underset{h}{\prec} e_2$, e_2 must start after e' slot-refresh phase, so it must obtain a larger timestamp than e', which is a contradiction.

The theorem holds. □

Theorem A.5.4.15 Any complete history h of Slotqueue does not have the `VWit` violation.

Proof Consider a complete history h. Suppose there exists a dequeue d starting at time t and returning `false` but there is an enqueue e that finishes before t and is still unmatched at t.

By Lemma A.5.4.11, some slot must contain a timestamp other than `MAX_TIMESTAMP` by t. Therefore, when d performs the slot-scan phase in `read_minimum_rank`, it must see this slot containing a non-`MAX_TIMESTAMP` and return a non-`DUMMY_RANK`. Consequently, d cannot return `false` on line 10.

We claim that inside a dequeue, before the `spsc_dequeue` on line 11, if a slot contains a non-`MAX_TIMESTAMP`, the corresponding SPSC cannot be empty. Consider the successful slot refresh call with the last slot modification instruction targeted at this slot before the `spsc_dequeue` on line 11. Because this slot refresh call sets the slot to non-`MAX_TIMESTAMP`, its `spsc_readFront` must see that the SPSC is non-empty (line 30). From the last refresh call to the current `spsc_dequeue` on line 11, no other `spsc_dequeue` can happen, so this SPSC cannot be empty when line 11 is reached. Therefore, it can never return `false` on line 12.

In conclusion, d cannot return `false`, a contradiction. The theorem holds. □

From the previous theorems, this theorem trivially holds.

Theorem A.5.4.16 Slotqueue is linearizable.

A.5.5 Progress guarantee

Notice that every loop in Slotqueue is bounded, and no method have to wait for another. Therefore, Slotqueue is wait-free.

A.5.6 Theoretical performance

A summary of the theoretical performance of Slotqueue is provided in Table 7, which is already shown in Table 4. By R, we mean remote operations and by L we mean local operations.

Operations	Time-complexity
`enqueue`	$4R + 3L$
`dequeue`	$3R + 2nL$

Table 7: Theoretical performance summary of Slotqueue. R means remote operations and L means local operations.

For `enqueue`, we consider Procedure 21. Line 1 causes 1 truly remote operation, as the distributed counter is hosted on the dequeuer. Line 2, as discussed in the theoretical performance of SPSC, causes $R + L$ operations. In the worst case, two `refreshEnqueue` calls are executed. We then consider each `refreshEnqueue` call. Line 7 causes $R + L$ operations. Most of the time, Line 12 - Line 17 are not executed. Therefore, the two `refreshEnqueue` calls cause at most $2R$ operations. So in total, $4R + 3L$ operations are required.

For `dequeue`, we consider Procedure 23. Line 19 causes most of the remote operations: The double scan of the `Slots` array causes about $2nL$ operations. We consider the truly remote operations. Line 23 causes $R + L$ operations. The double retry on Line 26 - Line 27 each causes L operation (Line 47) and R operation. So in total, $3R + 2nL$ operations are required.